

3D GameStudio



World Definition Language

Referenz-Handbuch

ACKNEX Version 3.9 vom 31. Mai 1998



CONITEC DATENSYSTEME

64807 Dieburg • Dieselstr. 11c • fon 06071-92520 • fax 06071-925233

Copyright © conitec GmbH 1996, 1997, 1998

WDL Interpreter	Volker Kleipa
ACKNEX Engine	Johann Christian Lotter
Sound/Musik	Wladimir Stolipin
World Editor	Wladimir Stolipin / Wolfgang Schulz (original code by B. Wyber & R. Quinet)
World Compiler	Christoph Kirst

Demos & Neuigkeiten finden Sie auf unserer Web-Page
<http://www.conitec.com>.

Als Besitzer einer Commercial- oder Profi-Version haben Sie Anspruch auf Email-Support bei technischen oder sonstigen Fragen. Senden Sie Ihre Frage an **support@conitec.com**, und geben Sie dabei bitte Ihre Version, Kaufdatum und Serien- bzw. Kundennummer an (auf Disk 1 bzw. auf Ihrer Rechnung).

Auch wenn Sie nur eine Lite-Version besitzen, sind Sie bei Problemen nicht alleingelassen: Nützliche Tips, eine Fragen-und-Antworten-Sektion sowie das internationale ACKNEX-Forum finden Sie im **ACKNEX Users Magazine** auf **<http://www.conitec.com/aum>**.

Handbuch und Software unterliegen dem Urheberrechtsschutz. Kein Teil dieses Handbuchs darf ohne ausdrückliches schriftliches Einverständnis von conitec reproduziert, vervielfältigt, gespeichert oder übersetzt werden. Für die in diesem Handbuch angegebenen Daten und Informationen übernehmen wir keine Gewähr; Änderungen, die dem Fortschritt dienen, behalten wir uns vor.

Inhalt

Einführung	1
World Editor	4
Vertex-Modus	14
Wall-Modus	15
Regions-Modus	16
Thing- und Actor-Modus	17
Way-Modus	18
WMP-Format	19
Die ACKNEX 3D Engine	21
World Definition Language	24
WDL-Einführung	24
Schlüsselworte	25
Dateien	29
Vordefinierte Schlüsselworte	32
Paletten	37
Texturen	40
Walls	55
Regionen	61
Things, Actors, Ways	68
Aktionen	80
Synonyme	104
Rollenspiel-Funktionen: Skills	107
Levelwechsel, Spielstände, Multiplayer-Modus	130
Bedieneroberfläche: Views, Panels, Texte, Overlays	132
Publikation eines ACKNEX-Games	142
Probleme & Hinweise	147
Index	151

Einführung

Herzlichen Glückwunsch zum Erwerb von 3D GameStudio! Mit diesem Toolkit können Sie *ohne spezielle Programmier-Vorkenntnisse* professionelle 3D-Demos, Werbe-, Adventure-, Rollen-, Action- oder Renn-Spiele produzieren und diese - wenn Sie wollen - lizenzfrei auf den Markt bringen.

Zur Konstruktion eines Computerspiels müssen Sie eine Skriptdatei (**WDL**) erstellen, die den 'Quellcode' für die Welt mit Angaben über Texturen, Regionen, Things und Actors enthält. Die Topographie Ihrer virtuellen Spielwelt steht in einer weiteren Datei (**WMP**). Alle Dateien können mit einem normalen ASCII-Editor (EDIT, WORDPAD) bearbeitet werden. Die WDL-Datei wird normalerweise mit einem solchen Texteditor erstellt, die WMP-Datei dagegen mit dem Topographie-Editor **WED**.

WED enthält die **ACKNEX** 3D Game Engine, mit der das Spiel getestet werden kann. Ist das Spiel fertig, so können Sie ein **Runtime-Modul** dafür erzeugen, eine separate EXE-Datei, die Sie dann frei vertreiben und publizieren dürfen. WED erzeugt zwei auf Ihr Spiel angepasste Runtime-Module, eines für DOS und eines für Windows95/DirectX.

Es gibt drei GameStudio-Versionen, die Lite-, Commercial- und Profi-Version. Sie unterscheiden sich in bestimmten Features und im Preis. Lesen Sie mehr über die Features und die Lizenzbedingungen im letzten Kapitel dieses Handbuches.

Dieses Handbuch soll nur als Referenz dienen. Zum Kennenlernen der vielfältigen ACKNEX-Features arbeiten Sie am besten das beiliegende Tutorial durch.

Die virtuelle Welt, die Sie mit 3D GameStudio konstruieren können, besteht aus den folgenden Elementen:

Ein **Level** ist ein Teil eines Spiels; er entspricht einer zweidimensionalen 'Landkarte'. Die Grafiken und sonstigen Bestandteile eines Levels werden komplett im RAM bzw. im virtuellen Speicher gehalten; beim Laden eines Levels

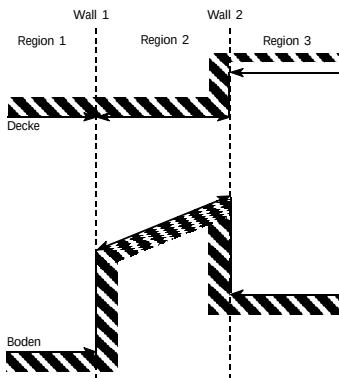
wird die Welt vorberechnet und aufgebaut. Jeder Levelwechsel benötigt daher einige Sekunden Rechenzeit.

Der Level ist in **Regionen** unterteilt. Eine Region wird durch die sie umgebenden Wände und durch ihren Boden und ihre Decke definiert. Wenn Sie von oben auf den Level schauen, hat eine Region die Form eines Polygons. Von der Seite betrachtet sieht sie aus wie eine vieleckige massive Säule, die sich nach oben und unten in die Unendlichkeit erstreckt und in der nur der Raum zwischen Boden und Decke 'herausgemeißelt' ist. Ihr Level ist aus solchen Säulen zusammengesetzt.

Jede Region kann beliebig viele weitere Regionen enthalten. Die Höhen und Neigungen von Boden und Decke einer Region sind frei wählbar; auf diese Weise lassen sich Treppen, Schluchten etc. erstellen (eine Treppenstufe ist eine eigene Region!). Pro Region kann man eine Lichtquelle definieren. Zu Konstruktion von Brücken, schwebenden Objekten, Fahrzeugen oder mehrstöckigen Gebäuden lassen sich beliebig viele Regionen übereinander setzen. Böden und Decken haben je eine Textur und einen Lichtwert; sie können auch mit einem Hintergrund (Sky) versehen werden. Die gleiche Region darf beliebig oft im Level auftauchen.

Die Regionen werden durch vertikale **Wände** (Walls) beliebiger Höhe und Ausdehnung begrenzt. Auf beiden Seiten einer Wand muß folglich jeweils eine Region liegen. Wände werden durch Verbindungslinien zwischen zwei Punkten (**Vertices**) auf der zweidimensionalen Landkarte repräsentiert. Die Wände können

an beliebigen Positionen und in beliebigem Winkel zueinander stehen. Vertikal sind Wände nur entlang der *Höhendifferenzen* von Böden bzw. Decken der Regionen zu ihren beiden Seiten sichtbar. Haben beide Regionen die gleiche Boden- und Deckenhöhe, so ist die Wand unsichtbar.



'Querschnitt' durch drei angrenzende Regionen

Im Bild sehen Sie im Seitenaufriß drei aneinandergrenzende Regionen, die durch zwei Wände begrenzt werden. Die mittlere Region bildet im Spiel eine 'Mauer' mit schräger

Krone, die zwei Räume unterschiedlicher Höhe trennt. Wandtexturen sind nur entlang der durchgezogenen Linien sichtbar.

Stufen, Fenster, Portale, Mauern etc. ergeben sich durch unterschiedliche Boden- oder Deckenhöhen. Böden und Decken können über die Z-Koordinaten ihrer Eck-Vertices zusätzlich geneigt werden (im Bild mit Wall 2 dargestellt). Wird die Decke einer Region mit einer Sky-Textur belegt, so sieht man über der Region den freien Himmel.

An beliebigen Positionen können Sie Gegenstände (**Things**) oder Lebewesen (**Actors**) platzieren. Die Textur eines Things kann je nach Blickwinkel wechseln, so daß das Objekt - obwohl zweidimensional - räumlich wirkt. **Actors** sind selbstbewegende, 'intelligente' Objekte. Sie können Actors im WED einen **Way** zuordnen, den sie dann im Level verfolgen. Das Verhalten und die Eigenschaften eines Actors können in der WDL-Datei festgelegt werden. Hierbei entspricht der Actor einem linearen Automaten, bei dem äußere Reize - **Events** - bestimmte festlegbare **Aktionen** auslösen.

Texturen sind Bitmaps beliebiger Größe, die auf Wände, Boden, Decke oder Objekte aufgespannt werden können. Die Textur kann animiert sein, ihr kann ein Sound zugeordnet werden und sie kann sich während des Spiels verändern, zum Beispiel durch Treffer. Es lassen sich auf einer Wand beliebig viele Texturen einander überlagern, um z.B. Inschriften oder Einschußlöcher darzustellen. Die Texturen werden in Echtzeit je nach den Lichtverhältnissen schattiert. Es stehen 256 Farben zur Verfügung, inklusive einer Transparenz-Farbe für Löcher oder Fenster.

In den folgenden Kapiteln sind die Befehle und Elemente des Editors und der World Definition Language **WDL** in Form einer Referenzliste beschrieben. Wenn Sie aber sofort mit der Entwicklung eines Game-Levels loslegen wollen, so lesen Sie das **Tutorial**. Dort ist der Aufbau eines Minimal-Levels Schritt für Schritt erklärt.

Wenn Sie es noch nicht getan haben, so ist jetzt der richtige Augenblick, die erste Diskette (#1) ins Laufwerk zu stecken, das Disketten-Verzeichnis zu öffnen und auf das **setup**-Icon zu doppelklicken.

World Editor

Der World Editor **WED.EXE** (bzw. **WEDS.EXE** in der 'Lite'- und **WEDC.EXE** in der 'Commercial'-Version) ist die integrierte Oberfläche zum Erstellen und Bearbeiten der virtuellen Welt. Der WED interpretiert die WDL-Datei und gegebenenfalls eine bereits vorhandene WMP-Datei; er kann Runtime-Module generieren und (mit der Profi-Version) Ihre Game-Dateien in eine einzige komprimierte Resource-Datei (.WRS) kompilieren.

WED ist ein DOS-Programm, läuft aber auch unter Windows 95. Der World Editor wird mit folgendem DOS-Befehl gestartet:

WED [*WED-Optionen*] *name*[.WDL] [*name*.WMP] [*ACKNEX-Optionen*]

Wird der WMP-Dateiname weggelassen, so lädt WED die in der WDL-Datei angegebene WMP-Datei. Existiert diese Datei nicht, so wird sie neu angelegt.

Folgende WED-Kommandozeilen-Optionen beeinflussen das Verhalten von WED (die ACKNEX-Optionen werden im nächsten Kapitel besprochen):

- S** startet in SVGA-Auflösung mit 800x600 Bildpunkten.
- I** startet in SVGA-Auflösung mit 1024x768 Bildpunkten
- VESA** benutzt zur SVGA-Grafikdarstellung das langsamere, aber kompatiblere VESA-BIOS. Starten Sie WED mit dieser Option, wenn die Darstellung auf Ihrer SVGA-Karte verzerrt erscheint (z.B. Matrox Mystique).
- VGA** Benutzen Sie diese Option, wenn Sie keine VESA-SVGA-kompatible Grafikkarte besitzen. Allerdings werden in diesem Modus die Boden- und Wand-Texturen nicht dargestellt.

-RUN Startet das Game direkt, ohne Umweg über den World Editor. Kann zum Starten eines compilierten Levels (WRS-Datei) verwendet werden.

Batch-Optionen (nur Profi-Version):

WED -C *name[.WDL] [name[.WDL]...]*

Compiliert alle angegebenen Level *name.WDL* zur Resource *name.WRS*.

WED -CT *name[.WDL] [name[.WDL]...]*

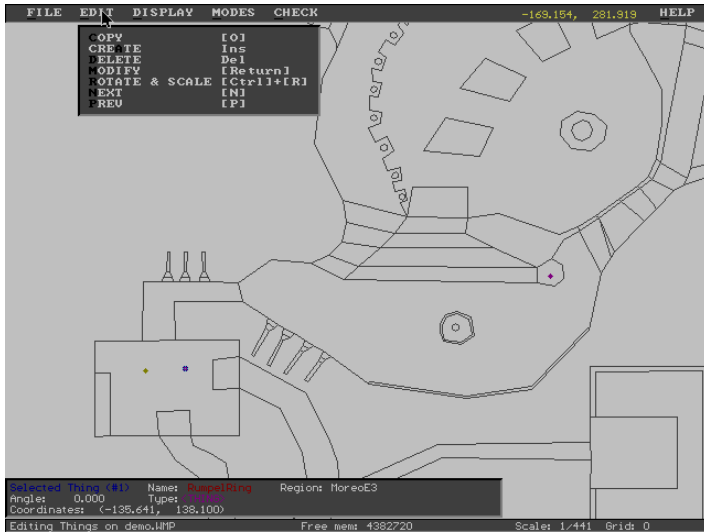
Wie **-C**, jedoch werden auch die Dateien mitcompiliert, die in der WDL-Datei durch **IFDEF** bzw. **IFNDEF** (s.u.) übersprungen werden. Auf diese Weise lassen sich mehrere Versionen eines Levels in eine einzige Resource-Datei compilieren.

WED -X *name[.WDL] verzeichnis*

Kopiert selektiv sämtliche Dateien, die für *name.WDL* benötigt werden, in das angegebene Verzeichnis. Sinnvoll, um die erforderlichen Dateien von nicht benötigten zu separieren.

WED -XT *name[.WDL] verzeichnis*

Wie **-X**, jedoch werden auch die Dateien mitkopiert, die in der WDL-Datei durch **IFDEF** bzw. **IFNDEF** (s.u.) übersprungen werden.



Nach dem Start zeigt WED die WMP-Datei als 'Landkarte' mit weißen Linien für Wände, grünen Kreuzen für Things, roten Kreuzen für Actors und einem blauen Kreuz für die Startposition des Players. Die Vergrößerung ist anfangs so eingestellt, daß die gesamte Landkarte auf den Bildschirm paßt.

Die Arbeit ist ähnlich wie mit einem CAD-Programm: Objekte werden durch Berühren mit der Maus selektiert und per Linksklick markiert. Das Info-Panel am unteren Bildrand gibt Auskunft über den aktuellen Modus und über die Parameter des gerade selektierten bzw. des zuletzt markierten Objekts.

Das WED-Menü enthält die folgenden Funktionen:

~~FILE~~ Untermenü

OPEN WMP [Ctrl-O]		Lädt eine neue WMP-Datei basierend auf der aktuellen WDL-Datei. Existiert diese Datei nicht, wird sie neu angelegt.
SAVE WMP	[F2], [Ctrl-S]	Speichert die aktuelle WMP-Datei mit Sicherheits-Backup (.BAK)
SAVE AS	Shift-[F2]	Speichert die aktuelle WMP-Datei unter einem neuen Namen.
PUBLISH	[Ctrl-P]	Generiert das DOS- und das Windows 95- Runtime-Modul (WRUN.EXE, WWRUN.EXE) für den aktuellen Level. Benutzen Sie hierzu den ersten Level des Spiels. Das Spiel kann dann zusammen mit diesem Runtime-Modul lizenzfrei verkauft werden; die im letzten Kapitel für die lizenzfreie Publikation aufgeführten Einschränkungen sind zu beachten. WWRUN.EXE benötigt zusätzlich die Dateien WWRUN.MDF und WWRUN.WDF im gleichen Verzeichnis.
COMPILE & PUBLISH	[F9]	Wie PUBLISH, compiliert jedoch zusätzlich eine World Resource-Datei (.WRS) aus allen zum Level zugehörigen Dateien (nur Profi-Version). Zeigt zugleich die 'Magic Key'-Nummer an, die für lizenzpflichtige Module benötigt wird. Der "All IFDEFs"-Schalter im Compile-Dialog sorgt dafür, dass auch die Dateien mitcompiliert werden, die in der WDL-Datei durch IFDEF bzw. IFNDEF übersprungen werden. Das Resource- Runtime-Modul für DOS ist VRUN.EXE, das für Windows 95 WVRUN.EXE.

EXIT	[Alt-F4], [Ctrl-Q], [F10]	Beendet das Programm.
------	---------------------------------	-----------------------

MODE	[Alt-M]	Modus-Untermenü
-------------	----------------	------------------------

VERTICES	[V]	Schaltet in den Vertex-Modus.
----------	-----	-------------------------------

WALLS	[W]	Schaltet in den Wall-Modus.
-------	-----	-----------------------------

REGIONS	[R]	Schaltet in den Region-Modus.
---------	-----	-------------------------------

THINGS	[T]	Schaltet in den Thing/Actor-Modus.
--------	-----	------------------------------------

WAYS	[Y]	Schaltet in den Way-Modus.
------	-----	----------------------------

WALK- THROUGH	[Shift-^] [Shift- ~]	Startet das Game. Im laufenden Game können Sie dann mit der gleichen Tastenkombination ständig zwischen Game und Editor hin- und herschalten, um Ihre Topographie auszutes-ten. Wird im WED der Player versetzt (blaues Kreuz), so wird die neue Player-Position simultan ins Game übernommen. Auch Änderungen von Wänden, Things und Actors werden Sie sofort im Game wiederfinden.
------------------	--------------------------	--

EDIT	[Alt-E]	Editier-Untermenü
-------------	----------------	--------------------------

UNDO	[BkSp], [Ctrl-Z]	Macht successive die letzten Operationen rückgängig.
------	---------------------	--

UNDO ALL	[Ctrl-U]	Macht sämtliche Änderungen seit dem letzten Speichern rückgängig.
----------	----------	---

COPY	[Ctrl-V]	Kopiert das selektierte Objekt mit sämtlichen Objekt-Eigenschaften, Offsets etc. um 1 Rasterabstand nach rechts unten verschoben.
------	----------	---

CREATE	[Ins], Linksklick	Erzeugt ein neues Objekt, abhängig vom aktuellen Modus.
DELETE	[Del]	Löscht das selektierte Objekt, mit Sicherheitsabfrage.
MODIFY	[Return], Rechts- klick	Verändert die Eigenschaften (Name etc.) des selektierten Objekts bzw. der markierten Objekte.
IMPORT	[Ctrl-I]	Fügt Vertices und Objekte aus einer externen .WMP-Datei in die Landkarte ein. Mit einer File-Scrollbox läßt sich die .WMP-Datei im aktuellen Verzeichnis auswählen. Die Objekt-namen müssen sämtlich in der aktuellen .WDL-Datei definiert sein!
EXPORT	[Ctrl-E]	Speichert alle markierten Objekte mitsamt ihren Vertices als externe .WMP-Datei im aktuellen Verzeichnis.
ROTATE & SCALE	[Ctrl-T]	Rotiert und skaliert das selektierte Objekt bzw. die markierten Objekte um einen vorgebbaren Winkel und Prozentwert.
NEXT	[Tab]	Selektiert das nächste Objekt.
PREV	[Shift-Tab]	Selektiert das vorige Objekt.
WALL	[Alt-W]	Wall-Untermenü (nur im Wall-Modus)
SPLIT	[S]	Teilt die selektierte Wand und erzeugt einen neuen Vertex in der Mitte.
JOIN	[J]	Verbindet mehrere markierte aneinander-grenzende Wände und löscht die gemeinsamen Vertices. Die neue Wand erhält die Eigen-schaften der ersten alten Wand.

FLIP	[F]	Ändert die Orientierung der selektierten Wand bzw. der markierten Wände durch Vertauschen des Anfangs- und Endvertex und der rechten und linken Region. Dies spiegelt die Wandtextur und ist für Funktionen erforderlich (z.B. ALIGN), die die rechte und linke Region der markierten Wände unterschiedlich behandeln.
RESET	[Z]	Setzt die horizontale und vertikale Verschiebung der Wand-Textur (X- und Y-Offset) bei allen markierten Wänden auf 0 zurück.
ALIGN HOR	[A]	Paßt die Texturen der rechten Seiten mehrerer markierter Wände durch Verändern der X-Offsets 'nahtlos' horizontal an. Hierzu muß eine zusammenhängende Kette von Wänden selektiert sein. Die Orientierung einzelner Wände kann ggf. mit FLIP geändert werden. Ausgehend von der ersten Wand der Kette werden die X-Offsets aller angrenzenden Wände so angepaßt, daß die Textur jeder Wand dort beginnt, wo die Textur der vorhergehenden Wand endet.
ALIGN CEILING	[C]	Paßt die Textur einer oder mehrerer markierter Wände durch Verändern der Y-Offsets vertikal an die Decke der Region an. Nach Durchführung der Operation liegen die Oberkanten der Texturen genau auf Deckenhöhe der rechten Region der ersten markierten Wand.

ALIGN FLOOR	[L]	Paßt die Textur einer oder mehrerer markierter Wände durch Verändern der Y-Offsets vertikal an den Boden der Region an. Nach Durchführung der Operation liegen die Unterkanten der Texturen genau auf Bodenhöhe der rechten Region der ersten markierten Wand.
LENGTH	[Ctrl-L]	Gibt der markierten Wand eine definierte Länge, indem der zweite Vertex der Wand entsprechend verschoben wird. Bei Aufruf dieser Funktion darf nur <u>eine</u> Wand markiert sein. In einem Zahlenfenster läßt sich die gewünschte Wandlänge in Steps direkt eingeben.
RECTANGLE	[Ctrl-R]	Erzeugt an der Cursorposition ein Rechteck aus vier Walls mit definierbarer Seitenlänge. In einem Zahlenfenster läßt sich die gewünschte Länge in Steps direkt eingeben.
POLYGON	[Ctrl-Y]	Erzeugt an der Cursorposition ein n-seitiges Polygon aus n Walls. Die Anzahl der Seiten läßt sich in einem Zahlenfenster eingeben.
DISPLAY	[Alt-D]	Display-Untermenü
GRID	[G]	Schaltet das blaue Raster ein bzw. aus. Bei eingeschaltetem Raster lassen sich Vertices, Things und Actors nur auf Rasterpunkte platzieren.
GRID+	[+], [Num+]	Verdoppelt den Maßstab des Rasters.
GRID-	[-], [Num-]	Halbiert den Maßstab des Rasters.

ZOOM+	[Num*]	Erhöht den Maßstab des Bildes (auch mit den Tasten [0]..[9].
ZOOM-	[Num/]	Reduziert den Maßstab des Bildes.
SCROLL	[Scroll Lock]	Schaltet das Autoscroll-Feature ein bzw. aus. Im ausgeschalteten Zustand wird der Bildausschnitt nur mit den Pfeiltasten bewegt.

CHECK	[Alt-C]	Check-Untermenü
--------------	----------------	------------------------

KEYWORDS	[Ctrl-K]	Überprüft, ob alle in der aktuellen WMP-Datei vorkommenden Schlüsselworte auch in der WDL-Datei definiert sind.
STATISTICS	[Ctrl-H]	Zeigt die Anzahl aller Objekte des aktuellen Levels.
SCAN REGIONS	[O]	Überprüft alle Regionen und schaltet in den Regionsmodus um. Erforderlich, wenn z.B. durch Einzug neuer Wände Regionen erzeugt wurden.
CHECK WALLS	[Ctrl-W]	Überprüft, ob alle Wände konsistente Regionszuweisungen besitzen. 'Verdächtige' Wände werden markiert.
DUPLICATES	[Ctrl-D]	Markiert alle Regionen, Walls, Things oder Actors, die den gleichen Namen wie das selektierte Objekt haben.

HELP	[Alt-H]	Hilfe-Untermenü
-------------	----------------	------------------------

ABOUT	Zeigt die Versionsnummer.
-------	---------------------------

Befindet sich der Mauszeiger über einem Objekt, so wird dieses automatisch mit einem gelben Rahmen selektiert. Klicken mit der linken Maustaste markiert das selektierte Objekt (grüner Rahmen). Sie können einen Rahmen ziehen, um mehrere Objekte zu markieren, oder diese mit gedrückter **[Shift]**-Taste nacheinander anklicken. Ziehen mit gedrückter linker Maustaste verschiebt das selektierte Objekt bzw. alle markierten Objekte.

In sämtlichen Modi kann mit den Tasten **[0]..[9]** der Bildausschnitt, in dem sich der Mauszeiger befindet, in festen Schritten heran- bzw. weggezoomt werden. Mit der **[TAB]**-Taste können Objekte nacheinander selektiert werden.

1. Vertex-Modus

Dieser Modus wird mit der **[V]**-Taste bzw. über das Menü eingeschaltet. Sie können Vertices - Eckpunkte für Wände - plazieren, verschieben oder löschen sowie die Vertices durch Linien verbinden. Zu beachten ist, daß der Abstand zwischen einem Vertex und einer davon unabhängigen Wand mindestens 0.25 Steps betragen muß. Ist die betreffende Wand auch aus weiter Entfernung sichtbar, muß der Abstand entsprechend höher gewählt werden. Zu enge Wand/Vertex-Abstände erzeugen sichtbare Fehler im Bild.

Beim Neigen von Regionen mittels der Z-Koordinaten der Eckpunkte ist zu beachten, daß die Boden- und Deckenfläche der Region nicht 'verbogen' werden darf und der Neigungswinkel 75° nicht wesentlich überschreiten sollte. Geneigte Flächen werden nicht sehr präzise dargestellt, daher sind diffuse Texturen hierfür am geeignetsten.

Folgende Maus- und Tastenfunktionen sind im Vertex-Modus aktiv:

Linksklick auf Hintergrund plaziert einen Vertex. Der Vertex erscheint als grünes Kreuz.

Linksklick auf Vertex markiert den Vertex. Ist gleichzeitig die **[Shift]**-Taste oder die **[Ctrl]**-Taste gedrückt, bleiben alle weiteren markierte Vertices markiert; ansonsten werden sie unmarkiert. Um den markierten Vertex erscheint ein grüner Rahmen.

Links Ziehen auf Hintergrund erzeugt einen Rahmen zum Markieren mehrerer Vertices.

Links Ziehen auf Vertex markiert und verschiebt den Vertex. Ist gleichzeitig die **[Shift]**-Taste oder die **[Ctrl]**-Taste gedrückt, so werden auch alle weiteren markierte Vertices verschoben; ansonsten werden sie unmarkiert.

Rechtsklick auf Vertex zum Editieren der Z-Koordinate (Höhe) des Vertex; mit unterschiedlichen Vertex-Höhen können Regionen geneigt werden.

[Del] löscht alle markierten Vertices.

[Ins] verbindet alle markierten Vertices in der Reihenfolge des Markierens mit Wall-Linien und schaltet dann in den Wall-Modus. Existieren links und rechts der neuen Walls bereits definierte Regionen, so werden diese automatisch den Walls zugewiesen.

[Shift]-[Ins] verbindet alle markierten Vertices, falls es drei oder mehr sind, in der Reihenfolge des Markierens mit einem geschlossenen Polygonzug aus Wall-Linien und schaltet dann in den Wall-Modus.

2. Wall-Modus

Dieser Modus wird mit der **[W]**-Taste oder über das Menü eingeschaltet. Hier können Sie ganze Liniengruppen plazieren, löschen, verschieben, rotieren oder skalieren. Jeder Wall **muß** ein Name sowie eine rechte und linke Region zugeordnet werden. Dabei ist zu beachten, daß die so gebildeten Regionen auf **allen** Seiten geschlossen sein müssen: Berühren sich verschiedene Regionen ohne Wall dazwischen, führt dies zu Fehlern bei der Bilddarstellung!

Linksklick auf Wall markiert die Wall. Ist gleichzeitig die **[Shift]**-Taste oder die **[Ctrl]**-Taste gedrückt, bleiben alle weiteren markierte Walls markiert; ansonsten werden sie unmarkiert. Um die markierte Wall erscheint ein grüner Rahmen.

Links Ziehen auf Hintergrund erzeugt einen Rahmen zum Markieren mehrerer Walls.

Links Ziehen auf Wall markiert und verschiebt die Wall zusammen mit den zugehörigen Vertices. Ist gleichzeitig die [Shift]-Taste oder die [Ctrl]-Taste gedrückt, so werden auch alle weiteren markierten Walls verschoben; ansonsten werden sie unmarkiert.

Rechtsklick auf Wall läßt eine Scrollbox erscheinen, in der der Wall ein Wall-Schlüsselwort aus dem WDL-Script zugewiesen werden kann.

[Del] löscht alle markierten Walls mitsamt ihren Vertices.

3. Regions-Modus

Hier können Sie zusammenhängenden Gebieten eine Regionsdefinition sowie Boden- und Deckenhöhen zuweisen. Wenn Sie durch Einfügen von Wänden neue Regionen erzeugt haben, schalten Sie zum Zuweisen mit **[O] (SCAN REGIONS)** in den Regions-Modus, andernfalls genügt **[R]**.

Linksklick auf Region markiert die Region. Ist gleichzeitig die **[Shift]**-Taste oder die **[Ctrl]**-Taste gedrückt, bleiben alle weiteren markierten Regionen markiert; ansonsten werden sie unmarkiert. Um die markierte Region erscheint ein grüner Rahmen.

Links Ziehen auf Hintergrund erzeugt einen Rahmen zum Markieren mehrerer Regionen.

Links Ziehen auf Region markiert und verschiebt die Region zusammen mit den zugehörigen Walls und Vertices. Ist gleichzeitig die **[Shift]**-Taste oder die **[Ctrl]**-Taste gedrückt, so werden auch alle weiteren markierten Regionen verschoben; ansonsten werden sie unmarkiert.

Rechtsklick auf Region läßt eine Scrollbox erscheinen, in der der Region ein Region-Schlüsselwort aus dem WDL-Script zugewiesen werden kann. Die Höhen können entweder verändert oder als Default-Höhen aus der Regionsdefinition übernommen werden.

[Del] löscht alle markierten Regionen mitsamt ihren Walls und Vertices.

4. Thing- und Actor-Modus

Hier können Sie Things und Actors plazieren, ihnen die Anfangswinkel zuweisen sowie bestimmte Positionen - z.B. die Startposition des Players - markieren.

Linksklick auf Hintergrund erzeugt ein Objekt (Thing oder Actor). Das Objekt erscheint als Kreis mit grünem Kreuz.

Linksklick auf Objekt markiert das Objekt. Ist gleichzeitig die **[Shift]**-Taste oder die **[Ctrl]**-Taste gedrückt, bleiben alle weiteren markierten Objekte markiert; ansonsten werden sie unmarkiert. Um das markierte Objekt erscheint ein grüner Rahmen.

Links Ziehen auf Hintergrund erzeugt einen Rahmen zum Markieren mehrerer Objekte.

Links Ziehen auf Objekt markiert und verschiebt das Objekt. Ist gleichzeitig die **[Shift]**-Taste oder die **[Ctrl]**-Taste gedrückt, so werden auch alle weiteren markierten Objekte verschoben; ansonsten werden sie unmarkiert.

Rechtsklick auf Objekt läßt eine Scrollbox erscheinen, in der dem Objekt die Player-Startposition oder ein Thing- bzw. Actor-Schlüsselwort aus dem WDL-Script zugewiesen werden kann.

[Ins] erzeugt ein Objekt, wie Linksklick.

[Del] löscht alle markierten Objekte.

5. Way-Modus

Dieser Modus wird mit der **[Y]** Taste bzw. über das Menü eingeschaltet. Sie können eine aus beliebig vielen Eckpunkten (Waypoints) bestehende geschlossene Wegstrecke für einen Actor eingeben. Die Wegstrecke erscheint auf dem Bildschirm als hellblauer gestrichelter Linienzug, der die Waypoints miteinander verbindet. Folgende Maus- und Tastenfunktionen sind in diesem Modus aktiv:

Linksklick auf Hintergrund erzeugt einen neuen Waypoint an der Cursorposition im aktuellen Way. Der Waypoint erscheint als blaues Kreuz und wird automatisch zwischen den beiden nächstliegenden verbundenen Waypoints bzw. hinter den letzten markierten Waypoint in den Way eingebunden.

Linksklick auf Waypoint markiert den Waypoint. Ist gleichzeitig die **[Shift]**-Taste oder die **[Ctrl]**-Taste gedrückt, bleiben alle weiteren markierten Waypoints markiert; ansonsten werden sie unmarkiert. Um den markierten Waypoint erscheint ein blauer Rahmen.

Links Ziehen auf Hintergrund erzeugt einen Rahmen zum Markieren mehrerer Waypoints.

Links Ziehen auf Waypoint markiert und verschiebt den Waypoint. Ist gleichzeitig die **[Shift]**-Taste oder die **[Ctrl]**-Taste gedrückt, so werden auch alle weiteren markierte Waypoints verschoben; ansonsten werden sie unmarkiert.

Rechtsklick auf Waypoint zum Zuweisen eines Way-Namens. Hierbei erscheint eine Pop-Up-Liste aller in der WDL definierten Way-Namen, von denen einer ausgesucht werden kann.

[Ins] erzeugt einen neuen Way und generiert zugleich an der Cursorposition den ersten Waypoint.

[Del] löscht alle markierten Waypoints aus dem Way, wobei die restlichen Waypoints verbunden bleiben.

6. WMP-Format

Die vom WED generierte WMP-Datei ist eine einfache Textdatei, die mit jedem ASCII-Editor verändert werden kann. Sie enthält eine Liste aller Koordinaten und Objekte in der angegebenen Reihenfolge im folgenden Format:

VERTEX *x y z*;

Vertex-Definition. *x*, *y* und *z* sind Fixkommazahlen mit drei Nachkommastellen und entsprechen der X- und Y-Koordinate und der Höhe des Vertex in *Steps*.

REGION *name f c*;

Region-Definition; *name* ist das Schlüsselwort der Region aus der WDL-Datei, *f* ist die Boden- und *c* die Deckenhöhe.

WALL *name v1 v2 rr rl sx sy*;

Wand-Position. *name* ist das Schlüsselwort für den zugewiesenen Wandtyp aus der WDL-Datei. *v1* und *v2* sind die fortlaufenden Nummern der Vertices für die beiden Eckpunkte. *rr* und *rl* sind die fortlaufenden Nummern der rechts und links von der Wand liegenden Regionen (bezogen auf die Blickrichtung von *v1* nach *v2*). *sx* und *sy* sind die horizontalen und vertikalen Textur-Offsets in Pixeln, die mit der ALIGN-Funktion verändert werden können.

THING *name x y a r*;

Thing-Position. *name* ist das Schlüsselwort für die Thing-Definition aus der WDL-Datei, *x* und *y* die Koordinaten des Things in *Steps*, *a* der Winkel des Things in Einheiten von 0...360° und *r* die Region, in der sich das Thing befindet.

ACTOR *name* *x* *y* *a* *r*;

Actor-Position. *name* ist das Schlüsselwort für die Actor-Definition aus der WDL-Datei, *x* und *y* die Koordinaten der Actor-Anfangsposition in *Steps*, *a* der Winkel des Actors in Einheiten von 0...360° und *r* die Region, in der sich der Actor bei Spielbeginn befindet.

PLAYER_START *x* *y* *a* *r*;

Anfangsposition, -Winkel und Anfangsregion des Players.

WAY *name* *x1* *y1* *x2* *y2*... *xn* *yn*;

Way-Definition. *name* ist das Schlüsselwort des Way, darauf folgen die Koordinatenpaare für die Waypoints.

Die ACKNEX 3D Engine

Die Engine ist der 'Kern' der Software, welcher für den Bildaufbau sorgt und das Game startet. Sowohl WED wie auch die Runtime Module enthalten die ACKNEX Engine. Sie wird mit der WED-Kommandozeilenoption **-RUN** oder mit der Menü-Option **WALKTHROUGH (IF11)** vom WED aus aktiviert.

Außer dem Dateinamen sind folgende Parameter in der WED-Kommandozeile zulässig:

- WMP *name*** Verwendet die Topographie aus der Datei *name.WMP*, auch wenn in der WDL mit MAPFILE eine andere Datei vorgegeben wurde.
- DIR *name*** Benutzt das angegebene Verzeichnis für die Spielstände, auch wenn in der WDL ein anderes Verzeichnis vorgegeben wurde.
- D *name*** Definiert das Schlüsselwort *name* für spätere Auswertung in der WDL-Datei via **IFDEF** (s.u.)
- NODE *n*** Nummer des Netzknoten (0..1) für Multiplayerspiele (nur Commercial- und Profi-Version).
- COM *n*** Nummer der seriellen Schnittstelle (1..4) für die Kommunikation bei Multiplayerspielen. Wird dieser Parameter weggelassen, so wird die Verbindung über ein IPX-Netzwerk versucht. Die Schnittstelle darf nicht von der Maus belegt sein und muß über einen freien Interrupt verfügen. COM1 und COM3 wird INT4, COM2 und COM4 INT3 zugeordnet.
- IPX** Benutzt ein Local Area Netzwerk (LAN) mit IPX-Protokoll, anstelle einer seriellen Verbindung, für die Multiplayer-Kommunikation (Windows, nur Profi-Version).
- OS** Keine Soundeffekte.
- OM** Keine Musik.
- OCD** Keine Erkennung von Audio-CDs.
- OT** Testlauf ohne Sound, Musik, CD-Audio und Timer.

- RO** Benutzt keine FM-Sounds, sondern die Wavetable via MPU401, wenn vorhanden, für die Musikausgabe.
- NJ** Der Joystick wird nicht benutzt. Diese Option ist sinnvoll, wenn es z.B. mit der DOS-Engine unter Windows 95 Probleme mit dem Joystick gibt (plötzliches 'Zucken' des Players).
- NM** Die Maus wird nicht benutzt.
- C** Check-Modus; es wird kein Spiel gestartet, sondern nur die Syntax der WDL- und WMP-Dateien überprüft.
- En** Anzahl der Fehlermeldungen. Normalerweise werden bei fehlerhaften WDL-Dateien bis zu 6 Fehlermeldungen ausgegeben und dann auf einen Tastendruck gewartet.
- W1** Start mit erweitertem Syntax-Check und Warnmeldungen für potentielle Fehler in der WDL-Datei.
- W2** Wie **-W1**, zusätzlich werden alle unbenutzten Objekte und Definitionen in der WDL-Datei angezeigt.
- SST** Startet das Spiel im Single-Step-Modus. In diesem Modus muss nach jedem Bildaufbau auf die **[S]**-Taste gedrückt werden.
- GOD** Startet das Spiel im God-Modus. In diesem Modus kann der Player im Level zu Testzwecken massive Wände und Objekte durchschreiten (Vorsicht - Absturzgefahr!).
- NC** Kein **CLIP_DIST**-Check.
- WND** Startet im Fenster- statt im Fullscreen-Modus.
- 3D_SIMEYE** Startet im 3D Steroscopic-Modus mit Unterstützung der SimuIEyes® Shutter-Brille (nur Profi-Windows-Version; wird von manchen älteren Video-Karten / DirectX-Treibern nicht unterstützt).

Während des Spiels können mit Tastenkombinationen bestimmte Debugging-Modi ein- oder abgeschaltet werden (im Runtime-Modul sind diese Modi inaktiv):

[Ctrl]-[Alt]-[G] Schaltet den God-Modus (s.o.) ein/aus.

[Ctrl]-[Alt]-[C]	Schaltet den CLIP_DIST -Check ein/aus.
[Ctrl]-[Alt]-[S]	Schaltet den Einzelschrittmodus ein/aus. In diesem Modus friert das Bild ein, mit der Taste [S] läuft das Game um jeweils ein Bild weiter.
[Ctrl]-[Alt]-[End]	Bricht das Game ab.
[Alt]-[Return]	Schaltet zwischen Fullscreen- und Fenster-Modus um.
[F11]	Schaltet zurück zum WED.

Außerdem sind folgende Tasten (sofern sie nicht per WDL umdefiniert wurden) im Spiel aktiv:

[F2]	Speichert den Spielstand in die Datei "TEST_0.SAV"
[F3]	Lädt den letzten Spielstand
[F5]	Schaltet den Motion Blur um.
[F6]	Speichert Screenshots in die Datei "SHOT_n.PCX"
[F10]	Beendet das Spiel.
[F12]	Schaltet 3stufig Musik und Sound ab.
[TAB]	Schaltet die Karte um.
[0]	Aktiviert eine Default-Playerbewegung, mit der der Player sich auch vertikal bewegen kann (im Runtime-Modul inaktiv).

World Definition Language

1. WDL-Einführung

Die Eigenschaften und Objekte der virtuellen Welt werden durch ein Script in WDL, der World Definition Language, beschrieben. Besteht die Spielwelt aus mehreren unabhängigen Spielfeldern (Leveln), so kann es für jeden Level ein eigenes WDL-Script geben.

Es gibt verschiedene Typen von WDL-Elemente, wie Regionen, Wände, Actors, Things, Texturen, Bitmaps. Übergeordnete Elemente wie Wände können untergeordnete Elemente wie Texturen enthalten, welche wiederum Bitmaps und Sounds enthalten. Jedem Element oder Objekt lassen sich bestimmte **Eigenschaften** zuordnen. Bei bestimmten Objekten werden durch **Events**, d.h. durch Einwirkung auf das Objekt, **Aktionen** ausgelöst. Diese Aktionen wiederum können die Eigenschaften von Objekten verändern.

Raum- und Zeiteinheiten der Welt sind der *Step* und der *Tick*. Ein *Step* entspricht etwa einem Bildschirmdurchmesser; ein *Tick* entspricht der Zeit zwischen zwei Bildzyklen (Frames) auf einem 486er-PC, d.h. etwa einer Sechzehntelsekunde. **Winkel** werden in Radiant (0..6.28) angegeben und gegen den Uhrzeigersinn gezählt, wobei 0 der positiven Richtung der X-Achse des Koordinatensystems entspricht.

Folgende Sonderzeichen gelten in der WDL:

...;	Semikolon beendet Zuweisung
..., ...	Kommata trennen Parameter
{...}	Geschweifte Klammern fassen Parameter zusammen
"..."	Zwischen Anführungszeichen stehen Texte
<...>	Zwischen spitzen Klammern stehen Dateinamen (ohne Pfad!)
#...	Kommentar bis zum Zeilenende
//...	Kommentar bis zum Zeilenende
/*...*/	Kommentarblock

2. Schlüsselworte

Alle Elemente der Welt und deren Eigenschaften werden über Schlüsselworte (*Keywords*) angesprochen. Schlüsselworte spielen in der WDL eine ähnliche Rolle wie Variablennamen in einer Programmiersprache. Einige Schlüsselworte sind vordefiniert. Die Schlüsselworte für Dateien, Objekte, Texturen usw. können jedoch vom Anwender selbst definiert werden.

Ein Schlüsselwort wird durch eine Zeile im WDL-Script definiert, die etwa so aussieht:

TYP *Keyword Wert*;

oder

TYP *Keyword { ... mehr Keywords... }*

Keyword ist das Schlüsselwort selbst, ein beliebiger Name aus bis zu 30 Buchstaben. Der Name darf keine Umlaute oder Sonderzeichen - ausgenommen das Unterstreichungszeichen `_` - enthalten und nicht mit einer Zahl beginnen. Auch darf man nicht verschiedenen Objekten das gleiche Schlüsselwort zuweisen. **TYP** steht für den Schlüsselwort-Typ, *Wert* für den zugewiesenen Inhalt des Schlüsselworts, z.B. eine Zahl, ein bzw. mehrere weitere Schlüsselworte oder eine ganze Liste von Schlüsselworten zwischen geschweiften Klammern `{...}`. Die Definition darf sich über mehrere Zeilen erstrecken und wird erst durch das Semikolon abgeschlossen. Für selbst-definierte Schlüsselworte gibt es folgende Typen (in alphabetischer Reihenfolge):

ACTION für eine Echtzeit-Aktion, d.h. eine Liste von Anweisungen, die während des Spiels durch ein Ereignis (Event) ausgelöst wird und Eigenschaften von Walls, Things, Actors oder Regionen verändern kann,

ACTOR	für ein belebtes Objekt, einen linearen Automaten mit definierbarem Aktions- und Reaktionsverhalten,
BMAP	für eine Bitmap (Grafikdatei) in den Formaten .PCX, .WEX, .LBM oder .BBM oder für einen rechteckigen Ausschnitt aus einer solchen Bitmap,
FLIC	für eine Animationsdatei im .FLI-, .WEX- oder FLC-Format (nur Profi-Version),
FONT	für einen Zeichensatz zur Darstellung von Zahlen und Texten,
MAPFILE	für eine Topographie-Datei,
MODEL	für ein texturiertes 3D-Modell im .MDL-Format,
MUSIC	für eine Song-Datei im Format .CMF oder .MID,
MSPRITE	für den Mauszeiger,
OVERLAY	für ein Bildschirm-Overlay zur Darstellung von Cockpits, Waffen oder Werkzeugen über dem 3D-Fenster,
OVLY	für eine zu einem Overlay zugehörige Bitmap,
PANEL	für ein Bedienungspanel mit Anzeige-Elementen,
REGION	für eine von Wänden begrenzte Region im Spielfeld,
SKILL	für eine Eigenschaft, die als Zahlenwert ausgedrückt werden kann,
SOUND	für eine Soundeffekt-Datei im Format .WAV bzw. .WEX,

STRING	für einen kurzen Text, z.B. für Beschreibungen, Menüs oder Dialoge, in 'Rohform',
SYNONYM	für einen 'Platzhalter',
TEXT	für einen formatierten Text,
TEXTURE	für eine animierte Textur, die Böden, Decken, Walls, Things oder Actors zugewiesen werden kann,
THING	für ein Werkzeug, eine Waffe oder ein sonstiges unbelebtes Objekt,
WALL	für eine Wand oder Schwelle, die Regionen voneinander trennt,
WAY	für den Weg, den ein Actor im Level zurücklegt,
VIEW	für ein 3D-Fenster in die virtuelle Welt.

Der *Wert*, der dem so definierten Schlüsselwort zugewiesen werden kann, richtet sich nach dem Typ des Schlüsselworts. Folgende Werte sind bei Zuweisungen möglich:

<i>Zahl</i>	Ordnet dem Schlüsselwort eine Zahl zu. Alle Zahlen sind ganze Zahlen mit bis zu 5 Stellen (z.B 12345) oder Festkommazahlen mit bis zu drei Nachkommastellen (z.B. -12345.678).
<i>Zahl,...Zahl</i>	Ordnet dem Schlüsselwort eine Liste aus mehreren Zahlen zu, die durch Kommata getrennt sind.
<i>Keyword</i>	Ordnet dem Schlüsselwort ein anderes, bereits vorher definiertes Schlüsselwort zu.

Keyword, ... Keyword

Ordnet dem Schlüsselwort eine Liste aus mehreren bereits vorher definierten Schlüsselworten zu.

"Text"

Ordnet dem Schlüsselwort eine Buchstabenfolge zu. Zeilenumbrüche werden als "\n" geschrieben.

<Filename> Ordnet dem Schlüsselwort den Inhalt einer Datei zu. Die Dateiart ergibt sich aus der Extension. Gültige Extensionen sind *<.LBM>*, *<.BBM>*, *<.PCX>* für Bitmaps oder Paletten, *<.FLI>*, *<.FLC>* für Animationen, *<.MDL>* für 3D Model files, *<.MID>* für Songs, *<.IBK>* für Instrumente, *<.WAV>* für Sound-Effekte und *<.WEX>* für 'externe' Grafik-, Sound- oder Animations-Dateien außerhalb der Resource. Alle in spitzen Klammern stehenden Dateien werden vom Compiler zur World Resource zusammengebunden.

{...}

Ordnet dem Schlüsselwort eine komplexe Beschreibung zu, die aus einer Liste weiterer Schlüsselwort-Zuweisungen besteht. Diese Beschreibung kann sich über beliebig viele Zeilen erstrecken und wird mit der Klammer abgeschlossen.

Im folgenden sind die fest vordefinierten Schlüsselworte in GROSSBUCHSTABEN, die Anwender-definierbaren Schlüsselworte, Parameter oder Zahlen *kursiv* geschrieben. Diejenigen Schlüsselworte, deren Werte auch während des Spiels durch *Actions* verändert werden können, sind mit einem Pfeil (►) gekennzeichnet. Diejenigen Schlüsselworte, deren Werte sich von selbst ändern und in *Actions* ausgewertet, jedoch nicht vordefiniert werden können, sind mit einem Rechteck (#) markiert.

3. Dateien

Schlüsselworten können Dateien zugewiesen werden, um Bitmaps, Animationen, Sounds und Songs zu definieren. Die Namen der Dateien, die zum fertigen Game-Level gehören und mitcompiliert werden sollen, müssen in der WDL in spitzen Klammern (<...>) ohne Pfad angegeben werden. Es gibt folgende Datei-Schlüsselworte:

BMAP *Keyword, < Filename >;*

Ordnet dem Schlüsselwort eine Bitmap (Grafikdatei) in 256 Farben in den Formaten .PCX, .LBM oder .BBM zu. Eine .PCX-Datei kann auch in .WEX umbenannt werden; in diesem Fall wird sie von der Engine zuerst als separate Datei außerhalb der Resource gesucht.

BMAP *Keyword, < Filename >,x,y,dx,dy;*

Alternativ: Ordnet dem Schlüsselwort einen rechteckigen Ausschnitt einer Grafikdatei zu. Die Koordinaten x,y bezeichnen die obere linke Ecke des Ausschnitts, dx,dy jeweils Breite und Höhe (Angaben in Pixeln). Werden die Koordinaten weggelassen, so wird dem Schlüsselwort die gesamte Datei zugeordnet.

OVLY *Keyword, < Filename >;*

OVLY *Keyword, < Filename >,x,y,dx,dy;*

Wie **BMAP**; ordnet jedoch dem Schlüsselwort ein Overlay zu. Overlays dienen der Darstellung von Cockpits, Waffen, Mauszeigern o.ä. auf dem Bildschirm. Sie belegen etwa den zwei- bis dreifachen Speicherplatz einer Bitmap, können dafür aber wesentlich schneller gezeichnet werden. Die horizontale Pixelanzahl einer Overlay-Bitmap muß durch 4 teilbar sein.

FONT *Keyword, < Filename >, width, height;*

Ordnet dem Schlüsselwort einen Zeichensatz zu. *width, height* ist die Breite und Höhe der einzelnen Zeichen in Pixeln. Alle Zeichen müssen die gleichen Abmessungen haben. Die Bitmap *Filename* enthält entweder 11 Zeichen - die Zahlen 0..9 und das Leerzeichen - für numerische Anzeigen, oder 128 oder 256 Zeichen des PC- bzw. ASCII-Zeichensatzes. Die Zeichen können mehrzeilig übereinander in der Bitmap stehen; die Anzahl der Zeichen und ihre Anordnung ergibt sich automatisch aus der Buchstabengröße sowie der Größe der Bitmap. Die Bitmap muß exakt entweder 11mal, 128mal oder 256mal so groß sein wie ein einzelnes Zeichen.

MODEL *Keyword, < Filename >;*

ordnet dem Schlüsselwort ein 3D-Polygon-Modell im Format .MDL zu. Sie können MDL-Dateien mit Freeware bzw. Shareware Tools wie z.B. QMe oder Meddle erzeugen und damit auch DXF-Meshes importieren. Zur Anpassung der MDL-Skin auf Ihre Level-Palette benötigen Sie im QMe-Hauptverzeichnis eine Palettendatei PALETTE.LMP im 768-Byte-Format (z.B. mit PCX2PAL erzeugen), für Meddle hingegen ein .PAL file im JASC-Format (mit Paintshop Pro erzeugen).

Das MDL-File muss folgenden Einschränkungen genügen: Nicht mehr als 1024 Faces (je weniger, desto besser); nur eine Skin, die an die Level-Palette angepaßt sein muss; als Polygone sind nur Dreiecke zulässig; Polygone dürfen einander nicht durchdringen; bestimmte 'kritische' Polygone (z.B. lange, spitze Objekte) müssen eventuell unterteilt werden, wenn sie durch andere Polygone hindurchscheinen oder verdeckt werden.

SOUND *Keyword, < Filename >;*

ordnet dem Schlüsselwort eine Sound-Datei im Format .WAV zu. Eine .WAV-Datei kann auch in .WEX umbenannt werden; in diesem Fall wird sie von der Engine zuerst als separate Datei außerhalb der Resource gesucht. Die Sample-Rate kann 11 oder 22 kHz betragen; die DOS-Engine (inklusive WED) akzeptiert nur 8-Bit Sounds, die Windows-Engine auch 16-Bit Sounds.

MUSIC *Keyword, < Filename >;*

ordnet dem Schlüsselwort eine Song-Datei im Format .MID zu. Die intern definierten Instrumente entsprechen dem General Midi-Standard.

FLIC *Keyword, < Filename >;*

ordnet dem Schlüsselwort eine Animationsdatei im .FLI- oder .FLC-Format zu. Eine .FLI-Datei kann auch in .WEX umbenannt werden; in diesem Fall wird sie von der Engine zuerst als separate Datei außerhalb der Resource gesucht. Dies ist bei sehr großen .FLI-Dateien sinnvoll, da diese als separate Datei etwas schneller abgespielt werden.

4. Vordefinierte Schlüsselworte

Mit Hilfe der folgenden Schlüsselworte, die am Beginn der WDL-Datei vor allen anderen Definitionen stehen müssen, werden die Grundeinstellungen für Grafik, Sounds und Verzeichnisse definiert:

VIDEO *Keyword;*

Stellt die Bildschirmauflösung ein. Es sind folgende Schlüsselworte für die Bildschirmauflösung vordefiniert:

320x200 VGA-Auflösung 320x200 Bildpunkte.

X320x240 VGA-Auflösung 320x240 Bildpunkte.

X320x400 VGA-Auflösung 320x400 Bildpunkte.

S640x480 SVGA-Auflösung 640x480 Bildpunkte (ab Commercial-Version).

S800x600 SVGA-Auflösung 800x600 Bildpunkte (Profi-Version).

Die niedrigen Auflösungen sind für schnelle Action-Spiele, die hohen 'S'-Auflösungen dagegen z.B. für Adventures oder kommerzielle Applikationen gedacht. Bei der Windows-Version ist zu beachten, dass die 'X'-Auflösungen von manchen DirectX-Treibern nicht unterstützt werden.

NEXUS *Zahl;*

Stellt die Größe des Nexus, d.h. der internen Datenstruktur zur Bilddarstellung ein. Die Nexus-Größe hängt von der Anzahl der im Bild hintereinander sichtbaren Objekte, Wände und Regionen ab. Je größer der Nexus, desto komplexere Räume können dargestellt werden, desto mehr Speicherplatz ist jedoch auch erforderlich. Der Vorgabewert für den Nexus ist 14. Ein für eine Szenerie zu kleiner Nexus wird mit der Fehlermeldung "nexus too small" angezeigt.

CLIP_DIST *Zahl;*

Vorgabewert der **CLIP_DIST** für Regionen (normalerweise 1000 Steps). Nur solche Wände und Objekte, die vollständig innerhalb der **CLIP_DIST**-Entfernung von der Playerposition liegen, werden beim Bildaufbau berücksichtigt. In den einzelnen Regionen lassen sich individuelle, von dem hier angegebenen Wert abweichende **CLIP_DISTs** definieren, mit denen man den Bildaufbau in komplexen Levels um bis zu 30% beschleunigen kann. Wände außerhalb der **CLIP_DIST** werden monochrom mit Palettenfarbe 1 dargestellt. Eine zu kleine **CLIP_DIST** kann Bildfehler erzeugen, kann aber auch manchmal sinnvoll sein - z.B. in Regionen, wo alles außerhalb einer bestimmten Entfernung im Dunkeln oder im Nebel verschwimmt.

LIGHT_ANGLE *Zahl;*

Mit diesem Parameter kann die Richtung zu einer unendlich weit entfernten Lichtquelle im Bereich 0..2 π vorgegeben werden. Alle Wände mit einer Textur-**ALBEDO** (s.u.) reflektieren Licht aus der angegebenen Richtung gemäß ihrer Orientierung. Der Wert 0 entspricht dabei Osten (Sonnenaufgang), der Wert 3.145 Westen (Sonnenuntergang).

DITHER *Zahl (0 oder 1);*

Mit diesem Parameter (Vorgabe = 1 = an) kann der 'Dithering'-Effekt ab- oder eingeschaltet werden. Durch Dithering wird ein sanfterer Übergang zwischen Licht- und Schattenbereichen erreicht.

IBANK *< Filename > ;***DRUMBANK** *< Filename > ;*

Lädt die Instrumentenbänke, die für die Midi-Songs verwendet werden. Filename ist eine ADLIB-Instrumentendatei im .IBK-Format. Die Perkussions-Instrumente werden über Midi-Kanal 10 abgespielt. Wird keine **DRUMBANK** angegeben, so stehen nur 5 Standard-Drums zur Verfügung; wird keine **IBANK** angegeben, so benutzt das Programm interne general midi-kompatible Instrumente.

MIDI_PITCH *Zahl;*

Anzahl der Oktaven, um die der Pitchbend-Parameter in Midi-Songs die Tonhöhe verändert; Wertebereich 1..12, Vorgabe 2.

INCLUDE *< Filename > ;*

Liest weitere WDL-Definitionen aus der separaten WDL-Datei *< Filename >* und fährt anschließend mit dem Auswerten der ursprünglichen WDL-Datei fort. Auf diese Weise lassen sich z.B. in einem Spiel die allen Leveln gemeinsamen Definitionen in gesonderten WDL-Dateien zusammenfassen, die dann von jeder Level-WDL per **INCLUDE**-Schlüsselwort gelesen werden. Es sind beliebig viele **INCLUDE**-Schlüsselworte zulässig.

BIND *< Filename > ;*

Die angegebene Datei wird vom Compiler zum Spiel dazugebunden, z.B. falls sie in einem anderen Level benötigt wird. Es können beliebig viele **BIND**-Dateien angegeben werden.

MAPFILE *< Filename > ;*

Definiert die WMP-Topographie-Datei des Anfangs-Levels. Dieses Schlüsselwort muß in der Haupt-WDL stehen, es kann nicht per **INCLUDE** gelesen werden.

SAVEDIR *"Dirname";*

Definiert das Verzeichnis, in dem die Spielstände gespeichert werden können. Existiert das Verzeichnis nicht, so wird es beim Speichern des ersten Spielstandes angelegt. Zu beachten ist, daß Backslashes ("\\") in C-Notation, d.h. doppelt (z.B. "C:\\games\\mygame") angegeben werden müssen. Der hier angegebene Verzeichnisname kann bei VRUN.EXE zur Laufzeit durch den Kommandoparameter **-DIR** ersetzt werden.

PATH *"Dirname";*

Alle Zusatz-Dateien - Bitmaps, Sounds, Mini - werden von WRUN zuerst im aktuellen Verzeichnis und dann in dem hier angegebenen Verzeichnis gesucht (auch hier Backslashes mit "\\\" angeben!). Es können bis zu 16 **PATH**-Schlüsselworte angegeben werden, die in der angegebenen Reihenfolge durchsucht werden.

DEFINE *NewName, [OldName];*

Mit diesem Schlüsselwort kann man in der WDL-Datei Namen, weitere Schlüsselworte und Parameter umbenennen (für C-Programmierer ist dies nichts Neues). Das Umbenennen hat zwar keine Auswirkung auf die Funktion der betreffenden Werte; insbesondere umfangreiche WDL-Aktionen werden jedoch deutlich besser 'lesbar', wenn den Objekt-Skills oder Zahlen aussagekräftige Namen gegeben werden.

Beispiel:

```
DEFINE MyNumber, -3456;
DEFINE HitEffect, SKILL3;
ACTION kill {
    SET  RESULT, MY.HitEffect;
    SET  MyActor.HitEffect, MyNumber;
}
```

Zeichen wie "{ } , ; () < > " können nicht umdefiniert werden, und **DEFINES** dürfen nicht verschachtelt werden.

UNDEF *Name;*

Ein per **DEFINE** angegebenes Schlüsselwort kann mit **UNDEF** für alle nachfolgenden WDL-Zeilen wieder 'undefiniert' werden.

IFDEF *Name;***IFNDEF** *Name;***IFELSE;****ENDIF;**

Mit diesen Schlüsselworten können WDL-Zeilen abhängig von **DEFINES** oder Start-Optionen (**-d**) 'ausgeblendet' werden. Alle Zeilen zwischen den Schlüsselworten **IFDEF** und **ENDIF** werden ignoriert, falls der **IFDEF**-Parameter vorher nicht per **DEFINE** oder Startoption **-d** definiert wurde. Alle Zeilen zwischen **IFNDEF** und **ENDIF** werden ignoriert, wenn der **IFDEF**-Parameter vorher definiert wurde. Das Schlüsselwort **IFELSE** kehrt die Wirkung des vorhergehenden **IFDEF** oder **IFNDEF** um.

Beispiel:

```
DEFINE hires; // oder WRUN -d hires
IFDEF hires;
    VIDEO  S640x480; // SVGA high resolution
IFELSE;      // no hires
```

```
VIDEO    X320x400; // VGA mid resolution
ENDIF;
```

5. Paletten

Da die einzelnen Pixel einer 256-Farben-Bitmap noch nicht die endgültigen Farben enthalten, sondern nur Farbnummern, ist eine *Palette* erforderlich, um diesen Nummern sichtbare Farben zuzuordnen. Ihr Paint-Programm speichert die Palette in der Grafik-Datei mit ab. Sie könnten also theoretisch jeder Textur eine eigene Palette geben. Dies wird jedoch 'psychedelische' Effekte hervorrufen, wenn die Textur-Paletten von der Level-Palette abweichen. Achten Sie darauf, daß alle Texturen im Level die gleiche Farbpalette benutzen. Hierzu bietet jedes Paint-Programm einen Menüpunkt 'Palette anpassen' bzw. 'Palette laden'.

Farb-Paletten sind nicht nur für die Farbdarstellungen der Texturen, sondern auch für das *Shading* zuständig, d.h. für das Aufhellen, Abdunkeln oder 'Vernebeln' von Texturen je nach Umgebungslicht und Entfernung von Player. Hierzu müssen die Farben innerhalb der Palette in Gruppen (Shading-Bereiche) abnehmender Helligkeit (oder zunehmender Neblichkeit) sortiert werden. In weiter Entfernung oder tiefer Dunkelheit werden alle Texturen in der zweiten Farbe der Palette dargestellt.

Die ersten drei Farben in der Palette haben eine Sonderbedeutung: Die erste Farbe ist für Transparenz, die zweite für Dunkelheit und weite Entfernungen, die dritte für Beleuchtung zuständig. Beim Erstellen von Grafiken ist es daher nützlich, der Farbe 0 irgendeinen erkennbaren Farbton zu geben, um sie vom 'echten' Schwarz zu unterscheiden. Die zweite Farbe sollte - je nach dem Effekt, den man für weite Entfernungen erzielen will - schwarz oder neblig-weiß sein, die dritte weiß oder hellgelb, je nach dem vorherrschenden Farbton der Lichtquellen.

Es ist immer nur eine Palette aktiv; jedoch lassen sich Paletten jederzeit wechseln oder sanft überblenden. Die in der WDL-Datei zuletzt definierte Palette wird bei Spielbeginn als Level-Palette benutzt. Mit folgender Zuweisung wird eine 256-Farbenpalette aus einer Grafikdatei im Format .PCX, .BBM oder .LBM geladen und die Shading-Bereiche definiert:

PALETTE *Keyword* { ... }

Dies weist dem Schlüsselwort eine Palettendefinition zu. Eine Palettendefinition kann - in der angegebenen Reihenfolge - zwischen den geschweiften Klammern die folgenden Schlüsselwort-Zuweisungen enthalten:

PALFILE < *Filename* > ;

Bestimmt die Grafikdatei (PCX, LBM oder BBM), deren Farben für die Palette verwendet werden. Unter Windows muß die erste Farbe dieser Palette (0) immer schwarz sein, die letzte (255) immer weiß.

RANGE *Start, Length;*

Shading-Bereich in der Grundpalette. Ein Shading-Bereich ist eine zusammenhängende Gruppe gleicher Farben mit kontinuierlich abnehmender Helligkeit. *Start* gibt die Nummer der Startfarbe an (1..254), *Length* die Anzahl der Farben in der Gruppe (bis zu 255). Die erste Farbe dieser Gruppe muß die hellste, die letzte die dunkelste sein, die fast der **INFINITY**-Farbe entsprechen sollte. Es können bis zu 24 Shadingbereiche pro Palette definiert werden. Wenn keine Shadingbereiche angegeben werden, ist das Shading ausgeschaltet. Farben, die in keinem Shading-Bereich liegen, werden nicht abgedunkelt; sie können daher zur Darstellung von Lichtquellen verwendet werden.

FLAGS *Keyword1, Keyword2...;*

Hier kann eine Liste von Schlüsselworten (Flags) angegeben werden, die die Eigenschaften der Palette bestimmen. Folgende Flags lassen sich 'setzen':

HARD

Wird dieses Flag angegeben, so wird das Shading 'hart' durchgeführt. Der Übergangsbereich von hell nach dunkel ist kleiner, die Farben werden nicht umgerechnet, sondern lediglich verschoben. Dadurch bleiben jedoch auch abgedunkelte Texturen kontrastreich, Details sind besser erkennbar.

AUTORANGE

Dieses Flag bewirkt ein automatisches Shading sämtlicher Palettenfarben, die sich in irgendeinem der oben angegebenen **RANGEs** befinden, auch wenn sie nicht sortiert sind. Die Farben werden nach ihrer Ähnlichkeit mit der **INFINITY**-Farbe - der zweiten Farbe in der Palette - abgeschattiert. Damit kann ein Shading- oder Nebel-Effekt auch mit völlig unsortierten Paletten erreicht werden. Allerdings ergibt manuelles Sortieren der Farben in differenzierten Farb-**RANGEs** im allgemeinen ein besseres und vor allem besser kontrollierbares Schattieren.

BLUR

Ist dieses Flag gesetzt, so erfolgt der Motion Blur-Effekt 'softer'. Es wird in der Bewegung eine leichte Unschärfe des 3D-Bildes erzeugt, die das 'Pixeln' kontrastreicher Texturen abmildert. Außerdem lassen sich Texturen semitransparent (**DIAPHANOUS**) darstellen. Nachteil: Spielstart und Palettenwechsel dauern pro **BLUR**-Palette ca. 2 Sekunden länger. Sind alle Level-Paletten einander ähnlich, genügt es, nur der Haupt-Palette das Flag **BLUR** zu geben.

Beispiel:

```
PALETTE my_pal {  
    PALFILE    <raw.pcx>;    // unsorted palette with 2nd color = black  
    RANGE      16,224;        // shade colors from #16 up to #240  
    FLAGS      BLUR,AUTORANGE; // don't bother about range sorting  
}
```

6. Texturen

Texturen bestimmen das Aussehen der Welt. Es gibt Texturen für Wände, Böden und Decken, für Hintergründe (Sky-Texturen), Things, Actors und Panels, die alle folgendermaßen definiert werden:

TEXTURE *Keyword* { ... }

Dies weist dem Schlüsselwort eine Texturdefinition zu. Eine Texturdefinition kann - in der angegebenen Reihenfolge - zwischen den geschweiften Klammern wiederum die folgenden Schlüsselwort-Zuweisungen enthalten:

SIDES *Zahl*;

Anzahl der 'Seiten' der Textur. Jede Seite entspricht einer Bitmap oder - bei animierten Texturen - mehreren Bitmaps. Je nach dem Objekt, dem die Textur zugewiesen wurde, unterscheidet sich die Bedeutung der Seiten. Normalerweise wechseln die Seiten-Bitmaps je nach dem Winkel, unter dem das betreffende Objekt betrachtet wird. Damit erhalten Things und Actors ein räumliches Aussehen. Thing- bzw. Actor-Texturen können beliebig viele **SIDES** haben, die im Uhrzeigersinn gezählt werden; die erste Seite entspricht der Frontalansicht.

WALL-Texturen haben normalerweise nur eine Seite. Bei zweiseitigen Wandtexturen wechseln die Bitmaps je nach der Blickrichtung, unter der die Wand betrachtet wird: Vom ersten Vertex der Wand her bis zur Mitte ist die erste Seite, von der Mitte bis zum Endvertex die zweite Seite sichtbar. Auf diese Weise lassen sich z.B. räumliche Elemente der Textur besser darstellen, indem der ersten Seite eine Linksperspektive, der zweiten eine Rechtsperspektive zugeordnet wird. Hat die Wandtextur vier Seiten, so sind die beiden ersten ihrer rechten bzw. Vorderseite, die beiden letzten ihrer linken bzw. Rückseite zugeordnet.

Mehr als vier Seiten für eine Wandtextur ist manchmal bei mehrstöckigen Regionen (s.u.) sinnvoll: In der obersten Region werden die ersten vier Wandseiten, in der darunterliegenden **BELOW**-Region die nächsten vier Seiten dargestellt und so weiter.

SKY-Texturen können beliebig viele Seiten haben; diese werden in gleichen Winkelbereichen auf das 360°-Panorama verteilt. Bei z.B. einer zehneitigen **SKY**-Textur nimmt jede Bitmap also einen Bereich von 36° ein. Damit ergibt

sich die horizontale Skalierung von Sky-Texturen aus der Größe der Bitmaps und der Anzahl der Sides. Vertikal sind **SKY**-Texturen nicht skaliert, d.h. ein Pixel einer solchen Textur entspricht einem Bildschirmpixel.

CYCLES *Zahl;*

Anzahl der Phasen bei animierten Texturen, default = 1.

FRAME *Zahl;*

Start-Frame bei animierten **MODEL**-Texturen, zwischen 1 und der Anzahl der Frames im **MDL**-File (default = 1).

BMAPS *Bitmap1, Bitmap2...;*

Liste von Schlüsselworten, denen die Bitmap-Dateien der Textur zugewiesen wurden. Alle Bitmaps einer Wand-, Boden- oder Decken-Textur müssen gleich groß sein; Thing- und Actor-Texturen dagegen dürfen Bitmaps unterschiedlicher Größe enthalten. Die Anzahl der Bitmaps ist nicht limitiert, muß aber dem Produkt aus (**SIDES** * **CYCLES**) entsprechen. Zuerst kommen in der **BMAPS**-Liste alle Phasen der Vorderseite des Objekts, dann die Phasen der weiteren Seiten im Uhrzeigersinn um das Objekt herum.

Anstelle eines Bitmap-Schlüsselworts ist in der Liste auch das Schlüsselwort **NULL** erlaubt, allerdings nur bei Texturen für Things, Actors oder transparente Walls. Die betreffende Textur-Phase oder -Seite wird dann nicht dargestellt, so als sei das Objekt unsichtbar. Bei Wänden muss mindestens eine Bitmap der Textur nicht-**NULL** sein.

Die maximale Breite oder Höhe einer Bitmap ist 1024 Pixel. Abgesehen davon kann die Bitmap für eine Thing- oder Actor-Textur beliebige Abmessungen haben. Bitmaps im LBM-Format müssen eine geradzahlige Breite haben. Wall-Texturen dürfen horizontal beliebig groß sein, vertikal muß die Anzahl der Pixel einer Zweierpotenz entsprechen, also z.B. 64, 128 oder 256 Pixel. Boden- und Deckentexturen müssen quadratisch sein; es sind hier nur die Abmessungen 64x64, 128x128 oder 256x256 Pixel erlaubt.

Bei Hintergrund-Bitmaps (Flag **SKY**, s.u.) ist die horizontale Größe beliebig, die vertikale Größe hängt vom maximal sichtbaren Hintergrund-Bildausschnitt (u.a. abhängig von Blickwinkel **PLAYER_TILT**, s.u.) ab. Normalerweise berührt der untere Rand der Bitmap den Horizont. Sky-Texturen können ebenso wie andere Texturen per Objekt-Parameter **OFFSET_X** und **OFF-**

SET_Y (s.u.) horizontal und vertikal verschoben werden; zusätzlich lassen sie sich mit den vordefinierten Skills **SKY_OFFS_X** und **SKY_OFFY_Y** verschieben. Werden im Bild die Ränder einer Sky-Textur oben oder unten überschritten, so wird der Rest des Bildausschnitts automatisch mit der angrenzenden Pixel-Zeile der Textur gefüllt; darum sollte die erste und die letzte Pixelreihe einer Sky-Textur monochrom sein.

FLIC *Flic*;

Textur-FLIC-Animation, alternativ zu **BMAPS** (nur Profi-Version). *Flic* ist das vorher definierte Schlüsselwort für eine FLI/FLC-Datei. Die Größe der Textur ergibt sich aus den Abmessungen dieser Animation. Diese Abmessungen unterliegen den gleichen Bedingungen, die unter **BMAPS** angegeben sind. Die Flic-Palette muß der Level-Palette entsprechen. Die Animation wird ständig wiederholt, es sei denn, das **ONESHOT**-Flag (s.u.) ist gesetzt.

TITLE *String*;

Mit dem Parameter **TITLE** kann einer Textur ein String zugewiesen werden, der anstelle einer **BMAP** mit dem Textur-**FONT** dargestellt wird. Der String kann als Schlüsselwort oder auch direkt (**TITLE** "Ein Titel!"); angegeben werden. Die Abmessungen des **TITLE** müssen den Einschränkungen der Textur genügen; z.B. muss bei einzeiligen **TITLEs** auf Wänden die Buchstabenhöhe eine Zweierpotenz sein. Hat eine Textur einen **TITLE**, so darf sie nicht animiert sein, und ihr dürfen keine **BMAPs** zugewiesen werden. Die **TITLE**-Textur darf auch via **ATTACH** über anderen Texturen erscheinen.

Beispiel:

```
TEXTURE title_tex {  
    FONT    my_font;  
    TITLE   "That's a title!";  
}
```

MODEL *Model;*

Animiertes 3D Modell, alternativ zu **BMAPS**. *Model* ist das vorher definierte Schlüsselwort für eine MDL-Datei. Eine MODEL-Textur ist nur für Things oder Actors zulässig. Die Größe der Textur ergibt sich aus den Positionen der äußeren Vertices des Model in Pixeln. Die Palette der Skin-Bitmap muß der Level-Palette entsprechen. Nur ein einziger **DELAY**-Wert kann angegeben werden. Die Model-Animation wird ständig wiederholt, es sei denn, das **ONESHOT**-Flag (s.u.) ist gesetzt. Wenn nur ein Teil der MDL-Frames für die Animation benutzt werden soll, kann der Startframe mit dem Textur-Parameter **FRAME** und die Anzahl der Phasen mit **CYCLES** angegeben werden. Beispiel:

```
TEXTURE hero3d_tex {  
    MODEL  hero_md1;  
    DELAY  2;  
    SOUND  hero_snd;  
}
```

DELAY *Zahl, Zahl...;*

Liste von Phasendauern in *Ticks* für jede Animationsphase. Die Anzahl der Delay-Zahlen entspricht dem Wert von **CYCLES**. Wird diese Liste nicht angegeben, so erfolgt die Animation mit einem Tick pro Phase. Je größer der **DELAY**-Wert, desto besser werden die Auswirkungen unterschiedlicher Bildaufbaugeschwindigkeiten 'ausgebügelt'.

MIRROR *Flag, Flag....;*

Liste von Spiegel-Flags (0 oder 1) für jede Textur-Seite. Ist das Spiegel-Flag auf 1 gesetzt, so werden alle Bitmaps der betreffenden Seite spiegelbildlich (seitenverkehrt) dargestellt. Die Anzahl der Flags muß dem Wert von **SIDES** entsprechen. Wird diese Liste nicht angegeben, so werden alle Bitmaps seitenrichtig dargestellt.

OFFSET_X *Zahl, Zahl...;***OFFSET_Y** *Zahl, Zahl...;*

Individuelle horizontale und vertikale Verschiebung der Bitmaps einer animierten Thing- oder Actor-Textur in Pixeln. Die Anzahl der Zahlen entspricht der Anzahl der **BMAPS**. Beispiel:

```
TEXTURE animthing_tex {  
    CYCLES    4;  
    BMAPS     thing_map, thing_map, thing_map, thing_map;  
    DELAY     6, 6, 6, 6;  
    OFFSET_X  5, 10, 20, 40;  
    OFFSET_Y  5, 10, 20, 40;  
}
```

RANDOM *Zahl*;

Maximalwert für einen zufallsabhängigen Verzögerungsfaktor (0..1), der zu jeder Phase der Texturanimation addiert wird.

► **SCALE_X** *x*;

► **SCALE_Y** *y*;

Skalierung der Textur in horizontaler und in vertikaler Richtung in Pixeln pro Step. Bei Boden- und Deckentexturen gibt der Wert *x* die Skalierung in X-Richtung, der Wert *y* die Skalierung in Y-Richtung des Koordinatensystems an. Je kleiner die angegebenen Werte, desto größer wird die Textur dargestellt; je größer die Werte, desto höher die Pixel-Auflösung in der Nähe des Objekts. Wird dieses Schlüsselwort weggelassen, so wird die Textur in beiden Richtungen mit 16 Pixels pro Step skaliert. Bei **SKY**-Texturen hat dieses Schlüsselwort keine Bedeutung.

► **AMBIENT** *Zahl*;

Leuchtkraft der Textur (-1 .. +1, default 0). Bei 0 wird die Wandtextur nur durch das Player-Licht und durch die Grundhelligkeit der Region erleuchtet; bei 1 leuchtet die Textur selbst hell. Negative Ambients ´schlucken´ Licht aus der Umgebung; sie können eingesetzt werden, um Schattenbereiche an den Wänden zu modellieren.

► **ALBEDO** *Zahl*;

Reflektionskraft der Textur (0..1, default 0) für aus Richtung **LIGHT_ANGLE** bzw. von einer separaten Lichtquelle (s. **GENIUS**) einfallendes Licht. Gebäude, Säulen oder Wandvorsprünge mit einer hohen Textur-**ALBEDO** (ca. 0.3) wirken räumlicher.

RADIANCE *Zahl*;

Mit diesem Textur-Parameter (0..1, default 0) läßt sich das Rückstrahlverhalten von Texturen bestimmen. Texturen mit einer **RADIANCE** > 0 nehmen bei hohen Lichtwerten die dritte Farbe der Palette an.

► **SOUND** *Sound;*

Schlüsselwort, dem eine Sounddatei (.VOC oder .WAV) zugewiesen wurde. Bei Bodentexturen ertönt dieser Sound rhythmisch beim Gehen, bei Deckentexturen ist der Sound - sofern **SLOOP** gesetzt ist - ständig in der gesamten Region hörbar.

► **SVOL** *Zahl;*

Maximallautstärke des Sound (0..1). Wird hier nichts angegeben, so wird der Sound maximal mit der Lautstärke 0.5 abgespielt.

► **SDIST** *Zahl;*

Kritische horizontale Distanz in *Steps*, innerhalb der der Sound hörbar ist (default 100). Der Sound wird um so lauter, je näher man dem Objekt kommt, dem die Textur zugewiesen wurde. Bei Stereo-Soundkarten kann man die Richtung zum Objekt anhand der unterschiedlichen Lautstärke aus dem rechten und linken Kanal lokalisieren. Nahe der Textur wird der Sound mit der mit **SVOL** eingestellten Maximallautstärke gespielt.

► **SVDIST** *Zahl;*

Kritische vertikale Distanz in *Steps*, innerhalb der der Sound hörbar ist (default 0, d.h. inaktiv). Nur wirksam bei Werten > 0 und bei Things oder Actors. Befindet sich der Player höher oder tiefer als Objekt-Höhe plus **SVDIST**, ist der Sound nicht hörbar.

SCYCLES *Flag, Flag....;*

Liste von Sound-Flags (0 oder 1) für jede Animationsphase. Ist das Sound-Flag auf 1 gesetzt, so wird der Textur-Sound bei der betreffenden Phase abgespielt. Die Anzahl der Flags entspricht dem Wert von **CYCLES**. Wird diese Liste nicht angegeben, so beginnt der Sound mit der ersten Phase.

►ATTACH *Textur;*

Der Original-Textur einer Wand, eines Things oder Actors wird die angegebene Textur überlagert, um z.B. Inschriften, Einschußlöcher oder Schatten darzustellen. Die **ATTACH**-Textur erscheint dabei transparent über oder (im Fall von Schatten) unter der Original-Textur. Bei Walls kann auch der neuen Textur wiederum mit **ATTACH** eine weitere Textur zugewiesen werden.

Auf diese Weise lassen sich beliebig viele Texturen auf der gleichen Wand überlagern - beispielsweise eine ganze Reihe von Einschußlöchern. Dabei ist jedoch zu beachten, daß die Textur-'Kette' nicht geschlossen sein darf: Keine Textur darf sich - direkt oder indirekt - selbst **ATTACH**en! Things und Actors kann nur eine einzige **ATTACH**-Textur zugewiesen werden, wobei der **ATTACH**-Parameter des Actors selbst benutzt werden muss.

ATTACH-Texturen können durchscheinend (**DIAPHANOUS**-Flag, s.u.) oder halbtransparent (**GHOST**) dargestellt werden. Sie dürfen animiert sein, wenn auch die Originaltextur animiert ist. Dabei muss die Anzahl der **CYCLES** bei der Original- und der **ATTACH**-Textur übereinstimmen. Die **DELAY**- und **RANDOM**-Werte der Originaltextur sind für die Animation ausschlaggebend. Soll nur die **ATTACH**-Textur animiert sein, so kann der Originaltextur eine 'Dummy-Animation' identischer Bitmaps zugewiesen werden.

►POS_X *Zahl;*

►POS_Y *Zahl;*

Eine **ATTACH**-Textur erscheint auf der mit diesen Schlüsselworten angegebenen Pixelposition über der Original-Textur, bezogen auf die obere linke Ecke. Sind **POS_X** und **POS_Y** beide 0 (default), so erscheint sie genau oben links.

Hinweis: Um eine **ATTACH**-Textur völlig unabhängig von Wand-Höhen und -Skalierungen exakt in die Mitte einer Wand zu platzieren, wird ihre Position folgendermaßen berechnet (Faustregel):

POS_X = **OFFSET_X** + **SIZE_X/2** - **WA/2**,

POS_Y = **HW** - **OFFSET_Y** - **HA/2** - (**CEIL_HGT+FLOOR_HGT**)***SCALE_Y/2**

POS_X, POS_Y: Position der **ATTACH**-Textur in Pixeln,

WA, HA: Breite und Höhe der **ATTACH**-Textur in Pixeln,

OFFSET_X, OFFSET_Y: Offsets der Wand in Pixeln,

HW: Höhe der Wand-Textur in Pixeln,

SCALE_Y: Skalierungsfaktor der Wand-Textur,
SIZE_X: Länge der Wand in Pixeln,
CEIL_HGT, FLOOR_HGT: Decken- und Bodenhöhe.

Bei **PORTCULLIS**- oder **FENCE**-Wänden (s.u.) ist die Berechnung einfacher: Bei einer Wand-**POSITION** von 0 bezieht sich der vertikale Nullpunkt einer **ATTACH**-Textur auf die Unterkante der Wand, d.h. die Attach-Textur muss mit einem negativen **POS_Y**-Wert nach oben verschoben werden, um sichtbar zu sein. Bei **POSITION**=1 bezieht sich der vertikale Nullpunkt der **ATTACH**-Textur auf die Oberkante der Wand.

Die folgenden Parameter bestimmen die Reaktion von Objekten, denen die betreffende Textur gegeben wurde, auf Berührungen oder Anklicken mit der Maus. Mit dem Skill **TOUCH_RANGE** (s.u.) kann die maximale Entfernung, bei der das Objekt noch auf die Maus reagiert, in Steps angegeben werden. Objekte, deren **IMMATERIAL**-Flag gesetzt ist, sind für die Maus 'unsichtbar'. Auch **ATTACH**-Texturen können auf die Maus reagieren, wobei jedoch nur die letzte (oberste) **ATTACH**-Textur relevant ist.

► **TOUCH** *String;*

Der angegebene Textstring erscheint auf dem Mauszeiger, sobald dieser das betreffende Objekt im Bild berührt, der Skill **TOUCH_MODE** (s.u.) einen Wert von 1 oder höher enthält und das Objekt näher ist als der Skill **TOUCH_DIST**. Siehe auch Synonym **TOUCH_TEXT** (s.u.).

FONT *Font;*

Zeichensatz für den **TOUCH**- oder den **TITLE**-String.

► **IF_TOUCH** *Action;*

Die angegebene Aktion wird ausgeführt, sobald der Mauszeiger die Textur berührt.

► **IF_RELEASE** *Action;*

Die angegebene Aktion wird ausgeführt, sobald der Mauszeiger die Textur wieder verläßt.

►IF_KLICK Action;

Die angegebene Aktion wird anstelle der 'allgemeinen' **IF_KLICK**-Aktion ausgeführt, sobald auf die Textur links geklickt wird. Mit den Skills **HIT_X** und **HIT_Y** (s.u.) lassen sich innerhalb der Aktion die Pixelkoordinaten der 'Klickstelle' auswerten.

FLAGS Keyword1,Keyword2...;

Hier kann eine Liste von Schlüsselworten (Flags) angegeben werden, die die Eigenschaften der Textur bestimmen. Folgende Flags lassen sich 'setzen':

ONESHOT

Wird dieses Flag gesetzt, so ist die Textur nicht permanent animiert. Sie zeigt normalerweise die erste Animationsphase. Durch Setzen des Flags **PLAY** des zugehörigen Objekts (Wall, Thing, Actor oder Region) kann die Textur für einen Zyklus animiert werden, sie bleibt danach auf der letzten Phase stehen. Ein eventueller Textursound wird einmal abgespielt.

GHOST

Wird dieses Flag gesetzt, so erscheinen Things, Actors und transparente Wände mit dieser Textur halbtransparent gedithered.

DIAPHANOUS

Wie **GHOST**, jedoch werden die Texturen transparenter Objekte oder Wände durchscheinend (nicht gedithered) dargestellt. Allerdings funktioniert dieses Flag nur mit einer **BLUR**-Palette und erhöht die Rechenzeit für die betreffenden Texturen um ca. 30%.

BEHIND

Die Attach-Textur erscheint hinter der Originaltextur (z.B. bei Things oder Actors).

SHADOW

Die Attach-Textur erscheint immer auf Bodenhöhe (z.B. bei Things oder Actors). Mit einer dunklen **DIAPHANOUS**-Textur lassen sich auf diese Weise Actor-Schatten generieren. Beispiel:

```
TEXTURE shadow_tex {  
    BMAPS    shadow_map;    // flat black spot (2nd color)  
    FLAGS    DIAPHANOUS,BEHIND,SHADOW;  
}  
ACTOR guy {  
    TEXTURE  guy_tex;  
    ATTACH   shadow_tex;  
}
```

LIGHTMAP

Das Flag **LIGHTMAP** bewirkt bei **ATTACH**-Texturen eine Licht- und Schatten-Modulation von WALL-Oberflächen, z.B. zum Generieren von Schatten- oder Lichtverläufen. Die **LIGHTMAP**-Bitmap darf nur die Farben #0..#15 und #241..#255 enthalten (sonst: Programmabbruch!). Die Farbe #0 ist transparent, die Farben #1..#15 dunkeln die WALL-Oberfläche in 16 Stufen ab, während die Farben #255..#241 sie aufhellen. Beim Aufhellen entspricht die hellste Farbe (#241) der Farbe #2 der Palette, die weiss oder hellgelb sein sollte. Beispiel:

```
BMAP torch_map,<torchlight.pcx>;  
TEXTURE torch_tex {  
    BMAPS    torch_map;  
    FLAGS    LIGHTMAP;  
    POS_X    40;  
    POS_Y    10;  
}  
WALL stone_wall {  
    TEXTURE  stone_tex;  
    ATTACH   torch_tex;  
}
```

SKY

Wird dieses Flag gesetzt, so erfolgt für die Textur keine Perspektiv-, sondern eine Parallaxprojektion. Sie wird damit wie eine Hintergrund-Kulisse dargestellt (für Berge, Himmel, Horizont usw.). Die Hintergrund-Textur wird nicht gezoomt, so daß sie immer in unendlicher Entfernung erscheint. **SKY**-Texturen können Wänden, Böden und Decken zugewiesen werden; sie gehen an den Berührungsstellen 'nahtlos' und unsichtbar ineinander über.

WIRE

Nur bei **MODEL**-Texturen: Stellt das **MODEL** als Drahtgitter in der **MAPCOLOR** dar.

CLUSTER

Nur bei **MODEL**-Texturen: Stellt nur die Eckpunkte der Polygone des **MODEL**s dar.

NO_CLIP

Dieses Flag bewirkt, daß Thing- oder Actor-Texturen nicht an Boden oder Decke einer Region abgeschnitten werden.

CLIP

Dieses Flag bewirkt, daß Thing- oder Actor-Texturen an Boden oder Decke einer Region abgeschnitten werden. Setzen Sie dieses Flag, um zu verhindern, dass Actors z.B. durch die Decken von BELOW-Regionen hindurch zu sehen sind.

SLOOP

Wird dieses Flag gesetzt, so wird der Textursound kontinuierlich in einer Schleife abgespielt, unabhängig von der Animationsphase. Lautstärke und Sound-Richtung werden dabei permanent nachgeregelt. Da dies insbesondere bei Wand-Texturen relativ viel Rechenzeit kostet und außerdem in Hörweite der Textur einen der Soundkanäle ständig belegt, sollte dieses Flag sparsam eingesetzt werden.

CONDENSED

Bei Setzen dieses Flags wird der Maus-Textstring (**TOUCH**, s.u.) horizontal um 1 Pixel gestaucht. Besonders bei schräggestellten Fonts sieht der Text dadurch häufig besser aus.

NARROW

Wie **CONDENSED**, jedoch wird der Textstring stärker gestaucht.

SAVE

Dieses Flag muß in der Texturdefinition gesetzt werden, wenn während des Spiels Parameter der Textur per Aktion verändert werden. Die Textur wird dann beim Speichern des Spielstands (**SAVE**-Anweisung, s.u.) abgespeichert.

Beispiel: Definition einer mehrseitigen Sky-Textur für eine Hügellandschaft mit 2 Türmen im Osten.

```
BMAP hills <huegel.lbbm>;
BMAP tower <turm.lbm>;

TEXTURE towers_landscape {
    SIDES 10;           // Eine Bitmap pro 36° Blickfeld
    BMAPS tower, hills, tower, hills, hills, hills, hills, hills, hills, hills;
    FLAGS SKY;
}
```

Weiteres **Beispiel:** Definition einer mehrseitigen, animierten Textur. Es soll ein schwimmender Fisch dargestellt werden.

```
BMAP PIRAN_0_1 <fish1.bbm>; // Piranha in 4 Phasen animiert,
BMAP PIRAN_0_2 <fish2.bbm>; // und in 5 Ansichten gezeichnet,
BMAP PIRAN_0_3 <fish3.bbm>; // hier von vorne
BMAP PIRAN_0_4 <fish4.bbm>;
BMAP PIRAN_45_1 <fishvr1.bbm>; // von vorne rechts (45 Grad)
BMAP PIRAN_45_2 <fishvr2.bbm>;
BMAP PIRAN_45_3 <fishvr3.bbm>;
BMAP PIRAN_45_4 <fishvr4.bbm>;
BMAP PIRAN_90_1 <fishr1.bbm>; // von rechts (90 Grad)
BMAP PIRAN_90_2 <fishr2.bbm>;
BMAP PIRAN_90_3 <fishr3.bbm>;
BMAP PIRAN_90_4 <fishr4.bbm>;
BMAP PIRAN_135_1 <fishhr1.bbm>; // von hinten rechts...
BMAP PIRAN_135_2 <fishhr2.bbm>;
BMAP PIRAN_135_3 <fishhr3.bbm>;
BMAP PIRAN_135_4 <fishhr4.bbm>;
BMAP PIRAN_180_1 <fishh1.bbm>; // von hinten.
```

```
BMAP   PIRAN_180_2   <fishh2.bbm>;
BMAP   PIRAN_180_3   <fishh3.bbm>;
BMAP   PIRAN_180_4   <fishh4.bbm>;
SOUND  PIRAN_SND     <shwssh.voc>;

TEXTURE  PIRAN_TEX {
    SCALE_X    20;           // Maßstab 20 Pixel pro Step
    SCALE_Y    20;
    SIDES      8;           // Objekt hat 8 Seiten
    CYCLES      4;           // und 4 Phasen
    BMAPS      PIRAN_0_1,PIRAN_0_2,PIRAN_0_3,PIRAN_0_4,
               PIRAN_45_1,PIRAN_45_2,PIRAN_45_3,PIRAN_45_4,
               PIRAN_90_1,PIRAN_90_2,PIRAN_90_3,PIRAN_90_4,
               PIRAN_135_1,PIRAN_135_2,PIRAN_135_3,PIRAN_135_4,
               PIRAN_180_1,PIRAN_180_2,PIRAN_180_3,PIRAN_180_4,
    // von hier an gespiegelt
               PIRAN_135_1,PIRAN_135_2,PIRAN_135_3,PIRAN_135_4,
               PIRAN_90_1,PIRAN_90_2,PIRAN_90_3,PIRAN_90_4,
               PIRAN_45_1,PIRAN_45_2,PIRAN_45_3,PIRAN_45_4;
    MIRROR      0,0,0,0,0,1,1,1; // Letzte 3 Seiten gespiegelt
    DELAY      3,3,2,2; // Phasendauern in Ticks
    SCYCLES     1,0,0,0; // Sound startet mit der ersten Phase
    RANDOM      0.2;       // Zufallsdauer für die Delay-Phasen.
    SOUND       PIRAN_SND;
    SDIST       30;        // Sound erst unter 30 Steps Entfernung hörbar
}
```

Hinweise zur Gestaltung von Texturen

Zur Produktion von Bitmaps können Sie zwischen verschiedenen Verfahren wählen, je nach der Atmosphäre, die Sie im Spiel erzeugen wollen. Sie können Bitmaps mit einem Paintprogramm zeichnen, mit einem 3D-Programm rendern oder nach Modellen einscannen. Auch in den beiden letzten Fällen werden Sie die Bitmaps in der Regel von Hand nachbearbeiten und vor allem auf Ihre Level-Palette anpassen müssen.

Bitmaps für Things, Actors und Wände - jedoch nicht Sky-, Panel- und Overlay-Bitmaps - werden gezoomt. Da hierbei entfernungsabhängig einzelne Pixelreihen der Textur unterdrückt werden, sollten benachbarte Einzelpixel keinen großen Farb- und Helligkeitskontrast aufweisen (bei eingescannten oder gerenderten Bitmaps ergibt sich dies in der Regel automatisch). Dithering,

Gitterstrukturen, dünne Linien etc. erzeugen Flimmer- und Moiréeffekte und sind zu vermeiden. Sind scharfe Linien und kontrastreiche Kanten erforderlich, so sollten diese zumindest auf einer Seite weich in den Untergrund übergehen. Wand- und Bodentexturen sollen normalerweise an allen Seiten anreihbar sein. Ausnahme sind höchstens solche Texturen, die eine Wand vollständig und ohne Wiederholung bedecken. Eine **SKY**-Textur sollte vertikal mindestens so groß sein, daß ihr oberer oder unterer Rand niemals im Blickfeld erscheinen kann; andernfalls sollten die Ränder, da sie bei Überschreitung des Blickfelds gedehnt wird, einfarbig sein. Auf jeder horizontalen Position kann nur eine **SKY**-Textur dargestellt werden; eine **SKY**-Wand und eine angrenzende **SKY**-Decke dürfen keine unterschiedlichen Bitmaps, **SIDES** oder horizontalen **OFFSET**-Werte haben. Thing- und Actor-Bitmaps sollten mit mindestens einem Pixel 'Luft' (Farbe 0) auf allen Seiten ausgeschnitten werden.

Bei den Abmessungen von Bitmaps sind bestimmte Gesetzmäßigkeiten zu beachten (s.o.) Da aber die Texturen skaliert werden können, bestimmen ihre Bitmap-Abmessungen nicht ihre Größe im Spiel. Generell sollte eine Textur-Bitmap um so größer sein, je detailreicher die Textur ist. Things und Actors, an die man nahe herankommt, sollten groß angelegt werden, Boden- und Deckentexturen entsprechend kleiner. Im allgemeinen sollten Texturen, die nur von weitem sichtbar sind, mit **SCALE_X/Y**-Faktoren unter 10 skaliert werden, und solche, an die man dicht herankommt, mit einem Faktor über 20. Werden während des Spiels Textur-Skalierungen, Lichtwerte oder Lichtquellen durch Aktionen verändert, muß bei allen betroffenen Objekten (Walls, Things oder Actors) die Textur mit einer **SET**- oder **SET_ALL**-Anweisung (s. Kapitel 'Aktionen') neu gesetzt werden.

Beim Generieren von **LBM**- oder **BBM**-Bitmaps mit dem Grafikprogramm Deluxe Paint ist zu beachten, daß die Breite der Bitmaps geradzahlig sein muß. Auch darf beim Speichern der Button 'old' im Menüpunkt **SAVE AS** nicht aktiviert sein. Viele Konvertierprogramme (GWS oder Paintshop Pro) erzeugen nur das 'old'-Format bei LBM's. Dieses Format wird nicht unterstützt; benutzen Sie stattdessen **PCX**.

Hinweise zu Sounds

Die verwendeten **WAV**-Sounds müssen im 8-Bit (DOS) bzw. 8- oder 16-Bit (Windows 95) Mono Format abgespeichert sein; für den Stereo-Effekt sorgt ACKNEX selbst. Die **WAV**-Dateien sollten keine Spezial-Events wie *Loop-Points*, *Regions* oder *Play-Lists* enthalten. Für normale Sounds genügt im allgemeinen eine Sampling-Rate von 11 kHz, hochfrequente Sounds (Zischen, Wasserrauschen oder -Plätschern) können in 22 kHz gesampled werden.

7. Walls

Vertikale Walls (Wände) verlaufen entlang der Verbindungslinien zwischen den Vertices; sie trennen die Regionen des Spielfelds voneinander. Ihre Eigenschaften werden in der Wand-Definition festgelegt:

WALL *Keyword* { ... }

Das so definierte Wand-Schlüsselwort kann dann später im WED einer Verbindungslinie zugewiesen werden. Eine Wanddefinition kann - in der angegebenen Reihenfolge - die folgenden Schlüsselwort-Zuweisungen enthalten:

- ▶ **TEXTURE** *Texture*; Textur für die Wand; bis zu vier Seiten. Bei zweiseitigen Texturen wechselt die Texturseite je nachdem, welcher Vertex der Wand dem Betrachter näher ist; vierseitige Texturen unterscheiden zusätzlich zwischen rechter und linker Seite der Wand.
- ▶ **ATTACH** *Texture*; Überlagerte Textur für die Wand; nur eine Seite. Die Anzahl der Phasen (**CYCLES**) muss entweder der der Wand-Textur entsprechen oder 1 sein.
- ▶ **OFFSET_X** *Zahl*; Diese beiden Parameter verschieben die Wandtextur um die angegebene Anzahl von **OFFSET_X** Pixeln nach links und um **OFFSET_Y** Pixel nach unten. **OFFSET_X** darf nicht negativ werden; wird dieses Schlüsselwort in der Definition angegeben, so darf die Wandtextur mit der ALIGN-Funktion des WED nicht noch zusätzlich verschoben werden. Bei Sky-Texturen kann mit **OFFSET_Y** der Abstand des Horizonts vom unteren Rand der Bitmap angegeben werden.
- ▶ **OFFSET_Y**
- ▶ **CYCLE** *n*; Augenblickliche Animationsphase der Textur.

► POSITION	<i>Zahl;</i>	Die Bedeutung dieses Schlüsselwortes ist abhängig von den Flags (s.u.).
MAP_COLOR	<i>n;</i>	Modus (0..1, Vorgabe 1) zum Zeichnen der Wand auf der Landkarte. Bei Zuweisung von 0 wird keine Wand gezeichnet, bei 1 wird die Wand in einer Default-Farbe (COLOR_WALLS , COLOR_BORDER - s.u.) gezeichnet.
► DIST	<i>Zahl;</i>	Grenzdistanz des nächsten Vertex der Wand zum Player, Vorgabe=0. Bei Überschreiten der Grenzdistanz kann eine IF_NEAR- oder IF_FAR-Aktion (s.u.) ausgelöst werden. Ist die DIST 0, wird eine IF_NEAR-Aktion immer bei Berühren oder Durchqueren der Wand ausgelöst. Ist die DIST größer als 0, jedoch kleiner als die halbe Länge der Wand, wird die IF_NEAR-Aktion nahe dem Zentrum der Wand nicht ausgelöst, da nur die Distanz zum nächsten Eckpunkt relevant ist!
► SKILL1... ► SKILL8	<i>Zahl;</i>	Acht 'Universalparameter', die auf die Eigenschaften der Wand keinen Einfluß haben. Sie können jedoch von Aktionen verändert und ausgewertet werden.

Die folgenden Parameter werden automatisch gesetzt, können jedoch in Aktionen ausgewertet werden:

#DISTANCE	<i>Zahl</i>	Ungefähre ($\pm 20\%$) Distanz des Players zum nächsten Vertex der Wand; nur gültig für Wände innerhalb der CLIP_DIST .
#LENGTH	<i>Zahl</i>	Länge der Wand in Steps.
#SIZE_X	<i>Zahl</i>	Länge der Wand in Pixeln, abhängig von der Textur-Skalierung.

#LEFT	<i>Region</i>	Die Regionen auf der linken bzw. rechten Seite der Wand.
#RIGHT		
►X1, ►Y1, ►Z1	<i>Zahl</i>	Position von Vertex1 der Wand; kann zum Verschieben oder Rotieren von Wänden während des Spiels per Aktion verändert werden. Dabei dürfen sich jedoch Wände nicht kreuzen! Wenn sich die Länge der Wand verändert, muss eine SHAKE -Anweisung (s.u.) durchgeführt werden.
►X2, ►Y2, ►Z2	<i>Zahl</i>	Position von Vertex2 der Wand.

Mit den folgenden Schlüsselworten lassen sich Aktionen angeben, die durch bestimmte mit der Wand zusammenhängende Ereignisse (Events) ausgelöst werden.

►IF_NEAR	<i>Action;</i>	Diese Aktion wird ausgelöst, sobald der Player sich dem nächstgelegenen Vertex der Wand bis auf eine Grenzdistanz (DIST) genähert hat. Ist die DIST =0, (default), wird die Aktion bei jeder Berührung bzw. Durchquerung der Wand ausgelöst.
►IF_FAR	<i>Action;</i>	Diese Aktion wird ausgelöst, sobald sich der Player vom nächstgelegenen Vertex der Wand über die kritische Distanz (DIST) hinaus entfernt hat.
►IF_HIT	<i>Action;</i>	Diese Aktion wird ausgelöst, wenn der Player die Wand mit der SHOOT -Anweisung trifft oder wenn die Wand von einem explodierenden Objekt (s.u.) getroffen wird.
►EACH_CYCLE	<i>Action;</i>	Diese Aktion wird nach jedem Animationszyklus der Textur bzw. am Ende einer ONESHOT -Animation ausgelöst.

- **EACH_TICK** *Action*; Diese Aktion wird nach jedem Bildaufbau (d.h. ca. jede 1/16 Sekunde) ausgelöst.

Mit dem folgenden Schlüsselwort kann eine ganze Liste von weiteren Schlüsselworten (Flags) 'gesetzt' werden, die die Eigenschaften der Wand bestimmen. Flags haben in Aktionen den Wert 1 (= gesetzt) oder 0 (= nicht gesetzt).

- FLAGS** *Flag1, Folgende Flags lassen sich in dieser Liste
Flag2...;* angeben bzw. in Aktionen auswerten oder
 verändern:
- **INVISIBLE** Wird dieses Flag gesetzt, so ist die Wand unsichtbar, jedoch undurchdringlich. Zu beiden Seiten unsichtbarer Wände muß die gleiche Region liegen.
- **PASSABLE** Wird dieses Flag gesetzt, so ist die Wand für den Player und für Actors passierbar (durchlässig).
- **IMMATERIAL** Die Wand wird nach Setzen dieses Flags von **SHOOT**-Anweisungen und Mausklicks nicht mehr beeinflußt.
- **IMPASSABLE** Wird dieses Flag gesetzt, so ist die Wand auf ihrer ganzen Höhe und Länge undurchdringlich, auch wenn sie nicht sichtbar ist. Ist zugleich das **PASSABLE**-Flag gesetzt, so ist die Wand zwar für den Player, nicht aber für Actors passierbar.
- #VISIBLE** Dieses Flag ist automatisch gesetzt, solange die Wand vom Player gesehen wird. Es kann in Aktionen ausgewertet werden.
- **SEEN** Dieses Flag ist automatisch gesetzt, sobald die Wand irgendwann einmal vom Player gesehen wurde. Es wird z.B. von der Automap-Funktion ausgewertet.

- **BERKELEY** Wird dieses Flag gesetzt, so ist die Wand nur solange vorhanden, solange man sie sieht oder sich innerhalb ihrer Grenzdistanz befindet. Das Flag spart Rechenzeit in Leveln mit vielen animierten oder mit Sound belegten Wandtexturen.
- **TRANSPARENT** Wird dieses Flag gesetzt, so ist die Farbe 0 in der Wandtextur durchsichtig, z.B. für Gitter oder Zäune. Zu beachten ist, daß zu beiden Seiten durchsichtiger Wände die gleiche Region liegen muß.
- **PLAY** Wird dieses Flag gesetzt, so wird die Wandtextur für einen Zyklus animiert; sie bleibt danach auf der letzten Phase stehen. Bei der betreffenden Textur muß hierzu das Flag **ONESHOT** gesetzt sein. Am Ende der Animation setzt sich das Flag **PLAY** automatisch auf 0 zurück, ggf. wird eine **EACH_CYCLE**-Aktion ausgelöst.
- **CURTAIN** Ist dieses Flag gesetzt, so erstreckt die Wall sich vom Boden bis zur Decke ihrer Region, unabhängig von den Regionshöhen.
- **PORTCULLIS** Dieses Flag 'heftet' die Wandtextur an die Ober- oder Unterkante der Wand; sinnvoll z.B. für Wände, die sich vertikal verschieben, wie Aufzüge oder Falлтüren. Wird das Flag gesetzt, so wird die Wand-Textur automatisch entweder an der Unterkante (bei **POSITION**=0) oder an der Oberkante (bei **POSITION**=1) der Wand ausgerichtet, so daß sie sich mit der Wand mitbewegt.

- FENCE** Wird bei transparenten Walls dieses Flag gesetzt, so endet die Wall nicht erst unter der Decke, sondern schon ab ihrer Textur-Oberkante. Der Wall-Parameter **POSITION** gibt dabei in Steps die Eindringtiefe der Wall in den Boden an; eine negative **POSITION** läßt sie ueber dem Boden 'schweben'. Durch Verändern von **POSITION** kann die transparente Wall wie ein Gatter angehoben oder abgesenkt werden. Bei **FENCE**-Walls ist das Flag **PORTCULLIS** automatisch gesetzt.
- **SENSITIVE** Nach Setzen dieses Flags löst die betreffende Wand ihre **IF_NEAR**-Aktion bereits dann aus, wenn sie vom Player erblickt wird.
- **FRAGILE** Wird dieses Flag gesetzt, kann die **IF_HIT**-Aktion der Wand durch eine **EXPLODE**-Anweisung (s.u.) ausgelöst werden.
- FAR** Wird dieses Flag gesetzt, so ist die Wand auch außerhalb der **CLIP_DIST** sichtbar; sinnvoll für weit entfernte **SKY**-Wände, um durch Verringern der **CLIP_DIST** Rechenzeit zu sparen. Wände mit Flag **FAR** dürfen nicht an **BELOW**-Regionen angrenzen.
- SAVE** Wird dieses Flag gesetzt, so werden alle Eigenschaften der Wand beim Spielstand-Speichern (**SAVE**-Anweisung, s.u.) mitgespeichert.
- **FLAG1...**
► **FLAG8** Acht 'Universalflags', die auf die Eigenschaften der Wand keinen Einfluß haben. Sie können jedoch von Aktionen verändert und ausgewertet werden.

Hinweis: Aufgrund der begrenzten Rechengenauigkeit bei der Texturdarstellung dürfen Wände maximal 200 Steps lang und maximal 1000 Steps hoch sein. Längere Wände müssen entsprechend unterteilt werden.

8. Regionen

Regionen sind durch Polygone (Wände) begrenzte Abschnitte im Spielfeld, die beliebig vertikal übereinander gestapelt werden können. Ihre Eigenschaften werden in der Region-Definition festgelegt:

REGION *Keyword* { }

Der mit diesem Schlüsselwort definierte Region-Prototyp kann dann im WED der rechten oder linken Seite von Walls zugewiesen werden. Eine Regionsdefinition enthält - in der angegebenen Reihenfolge - die folgenden Schlüsselwort-Zuweisungen:

- | | | |
|---|-----------------|---|
| ► FLOOR_TEX | <i>Texture;</i> | Weist dem Boden der Region eine Textur zu. Der Textursound ertönt beim Gehen in der Region, entweder rhythmisch zur WAVE - oder WALK -Periode (s.u.), je nachdem, ob das SLOOP -Flag der Textur gesetzt ist oder nicht. Tonhöhe und Lautstärke können durch Skills (PSOUND_VOL , s.u.) verändert werden. |
| ► CEIL_TEX | <i>Texture;</i> | Weist der Decke der Region eine Textur zu. Der Sound der Deckentextur ertönt bei Betreten der Region; ist das SLOOP -Flag gesetzt, hört man den Sound in der Region ständig, unabhängig von der Bewegung des Players. |
| ► FLOOR_HGT
► CEIL_HGT | <i>Zahl;</i> | Höhe des Bodens bzw. der Decke der Region in Steps; bei geeigneten Regionen addieren sich zu diesen Werten noch die Höhen (Z -Werte) der Eckpunkte. Die Höhendifferenz zwischen Boden und Decke sollte 1000 Steps nicht überschreiten. |

►FLOOR_OFFSETS_X ►FLOOR_OFFSETS_Y ►CEIL_OFFSETS_X ►CEIL_OFFSETS_Y	<i>Zahl;</i>	Verschiebung der Boden- oder Deckentextur in Richtung der X- oder Y-Koordinatenachse in Pixeln, bezogen auf den Koordinatenursprung des Spielfelds.
►FLOOR_ANGLE ►CEIL_ANGLE	<i>Zahl;</i>	Texturwinkel der Boden- und Deckentexturen; Bereich 0..2 π , default 0. Dieser Winkel entspricht einer Drehung der Textur um das Drehzentrum GENIUS (s.u.). Allerdings benötigen gedrehte Boden- und Decken-Texturen mehr Rechenzeit und sind nur bei un-geneigten Regionen möglich.
BELOW	<i>Region;</i>	Zuweisung einer vorher definierten, tieferliegenden Region zur Darstellung von Balkons, Brücken, mehrstöckigen Gebäuden oder schwebenden Objekten oder zur Definition von Unterwasser-Regionen. Der tieferen Region darf ebenfalls eine BELOW -Region zugeordnet sein. Im Level werden alle über BELOW verbundene Regionen übereinander gestapelt dargestellt.
#TOP	<i>Region;</i>	Die oberste Region einer Kette von BELOW -Regionen; andernfalls die Region selbst. Dieser Parameter wird bei Game-Start automatisch gesetzt, kann aber in Aktionen ausgewertet werden.

- **GENIUS** *Object;* Zuweisung eines Things oder Actors, der alternativ zu **LIGHT_ANGLE** als Lichtquelle für die Wände der Region (s. **ALBEDO**) und gleichzeitig als Drehzentrum für die **ROTATE**-Anweisung dient.
Das Objekt darf **INVISIBLE** sein, muß aber, um statt des globalen **LIGHT_ANGLE** als Lichtquelle zu dienen, einen Textur-**AMBIENT** > 0 haben. Es darf auch außerhalb der Region liegen und gleichzeitig mehreren Regionen zugeteilt werden.
- **AMBIENT** *Zahl;* Helligkeit der Region im Bereich 0..1 (16 Stufen, default 0). Die **AMBIENT**-Werte von Textur und Region ergeben zusammen mit der Player-Lichtquelle (**PLAYER_LIGHT**) und der Entfernung zum Player die Helligkeit einer Textur.
- **CLIP_DIST** *Zahl;* Alle Wände in größerer als der angegebenen Entfernung (in *Steps*) von der Playerposition in dieser Region werden beim Bildaufbau nicht berücksichtigt. Regionen, deren Inneres nie sichtbar ist - wenn sich z.B. Boden und Decke berühren - sollten eine **CLIP_DIST** von 0 erhalten.
- **SKILL1...**
► **SKILL8** *Zahl;* Acht 'Universalparameter', die auf die Eigenschaften der Region zunächst keinen Einfluß haben. Sie können jedoch innerhalb von Aktionen verändert und ausgewertet werden.
- **IF_ENTER** *Action;* Diese Aktion wird ausgelöst, sobald der Player die Region betritt. Hierzu muss sich der Player jedoch mindestens einen Framezyklus lang in der Region aufhalten, was besonders bei schmalen Regionen und schneller Player-Bewegung zu beachten ist.

►IF_LEAVE	Action;	Diese Aktion wird ausgelöst, sobald der Player die Region verläßt.
►IF_DIVE	Action;	Diese Aktion wird ausgelöst, sobald die Augenhöhe des Players (PLAYER_Z) die Bodenhöhe erreicht oder unterschreitet.
►IF_ARISE	Action;	Diese Aktion wird ausgelöst, sobald die Augenhöhe des Players die Deckenhöhe erreicht oder überschreitet.
►EACH_CYCLE	Action;	Diese Aktion wird am Ende jeder Textur-Animationsphase bzw. am Ende einer ONESHOT -Animation ausgelöst.
►EACH_TICK	Action;	Diese Aktion wird pro Bildzyklus einmal ausgeführt.
FLAGS	Flag1, Flag2...;	Hier kann eine Liste von Schlüsselworten (Flags) gesetzt werden, die die Eigenschaften der Region bestimmen. Folgende Schlüsselworte sind möglich:
FLOOR_ASCEND CEIL_ASCEND FLOOR_DESCEND CEIL_DESCEND		Wird eines dieser Flag gesetzt, so werden die Eckpunkte von Boden oder Decke um die Z -Werte ihrer Vertices angehoben (ASCEND) oder abgesenkt (DESCEND). Auf diese Weise lassen sich geneigte Flächen darstellen (s. Anmerkungen unten).
#VISIBLE		Dieses Flag ist automatisch gesetzt, solange Boden, Decke oder Wände der Region sichtbar sind. Es kann in Aktionen ausgewertet werden.
►SEEN		Dieses Flag ist automatisch gesetzt, sobald die Region irgendwann einmal vom Player gesehen wurde.
SAVE		Wird dieses Flag gesetzt, so werden alle Eigenschaften der Region beim Spielstand-Speichern mitgespeichert.

SAVE_ALL

Wenn dieses Flag gesetzt ist, werden sämtliche Walls und Things innerhalb dieser Region mit dem Spielstand mitgespeichert. Das Flag ist sinnvoll, wenn die Region in Echtzeit bewegt wird, z.B. durch **ROTATE**.

#HERE

Dieses Flag ist automatisch gesetzt, solange sich der Player in dieser Region befindet.

•BASE

Dieses Flag bewirkt, daß sich die Bodenhöhe des Players (**PLAYER_HGT**) beim Betreten der Region nicht ändert. Sinnvoll bei schmalen Löchern oder Schlitzten im Boden, in die der Player nicht 'hineinfallen' soll.

STICKY

Wenn dieses Flag gesetzt ist, wird der Player bei **SHIFT**- oder **ROTATE**-Anweisungen mit dem Boden der Region mitbewegt.

•PLAY

Wird dieses Flag gesetzt, so werden Boden- und Decken-Textur für einen Zyklus animiert; sie bleiben danach auf der letzten Phase stehen. Bei den Texturen selbst muß hierzu das Flag **ONESHOT** gesetzt sein. Am Ende der Animation setzt sich das Flag **PLAY** automatisch auf 0 zurück, ggf. wird die Aktion **EACH_CYCLE** (s.o.) ausgeführt.

•FLAG1...**•FLAG8**

Acht 'Universalflags', die auf die Eigenschaften der Region zunächst keinen Einfluß haben. Sie können jedoch innerhalb von Aktionen verändert und ausgewertet werden.

Hinweise zu **BELOW**-Regionen:

- # Der Player und alle Things und Actors ohne **GROUND**-Flag werden bei Spielstart automatisch auf die höchste Region an ihrer Position platziert.
- # **FAR**-, **PORTCULLIS**- und **FENCE**-Wände sind an und innerhalb von **BELOW**-Regionen nicht möglich.

Unterwasser-Regionen:

- # Entspricht die **CEIL_HGT** einer **BELOW**-Region genau der **FLOOR_HGT** der darüberliegenden Region, so läßt sich die **BELOW**-Region als 'Unterwasser'-Region einsetzen. Der vordefinierte Skill **PLAYER_DEPTH** (s.u.) enthält bei solchen **BELOW**-Regionen die Differenz zwischen den Bodenhöhen und damit die 'Wassertiefe'; in anderen Regionen ist er 0. **PLAYER_DEPTH** kann z.B. benutzt werden, um per Aktion (s.u.) zum Waten die **PLAYER_SIZE** zu reduzieren. Hat die obere Region eine **IF_DIVE**- und die untere eine **IF_ARISE**-Aktion, so sind Boden bzw. Decke für den Player durchdringlich.

Beispiel:

```
REGION under_water {
    FLOOR_HGT -5;
    CEIL_HGT  1;
    FLOOR_TEX mud_tex;
    CEIL_TEX  underwater_tex;
    IF_ARISE  water_arise;
}
REGION water {
    FLOOR_HGT 1;    // same as under_water's CEIL_HGT!
    FLOOR_TEX water_tex;
    CEIL_HGT  20;
    CEIL_TEX  sky_tex;
    BELOW     under_water;
    IF_DIVE   water_dive;
}
ACTION water_dive { FADE_PAL blue_pal,0.5; }
ACTION water_arise { FADE_PAL blue_pal,0; }
```

Einschränkungen für Schräg-Regionen (**FLOOR_ASCEND**, **CEIL_ASCEND**, **FLOOR_DESCEND**, **CEIL_DESCEND**):

- # Auf geneigte Böden und Decken werden die Texturen vertikal projiziert, so daß sie trotz der Neigung nahtlos aneinander anschließen. Wegen der Projektionsverzerrungen sollte der Winkel nicht zu steil werden; außerdem sollten möglichst kontrastarme Texturen verwendet werden. Da die Darstellung geneigter Flächen Rechenzeit kostet, ist es empfehlenswert, sie sparsam einzusetzen.
- # Bei großen Boden- oder Decken-Bitmaps (256x256 Pixel) sollte der **SCALE_X/Y**-Faktor der Textur den Wert 32 nicht überschreiten, sonst 'schwimmt'

die Textur u.U. in der Entfernung. Kleinere Bitmaps dürfen entsprechend feiner skaliert werden.

- # Bei dem Zuweisen von Höhen (Z-Werten) an Vertices darf die Boden- und Deckenfläche der Region nicht 'verbogen' werden.
- # Es dürfen nicht mehrere identische Regionen mit unterschiedlicher Steigung an die gleiche **BELOW**-Region angrenzen. In diesem Fall muß für jede Schräge eine separate Region definiert werden.
- # Walls, die Schrägen voneinander trennen, sollten mit den Flags **INVISIBLE** und **PASSABLE** unsichtbar gemacht werden.
- # Die Topographie ist so zu wählen, dass nicht direkt aus einem 90°-Winkel seitlich auf eine Schräge geblickt wird, sonst können treppenförmige Verzerrungen auftreten.
- # Der Boden einer Schräge darf nicht in die Decke eindringen (0.01 Steps Sicherheitsabstand einhalten).
- # Der **AMBIENT** einer schrägen Region ist so zu wählen, daß der aus Textur-**AMBIENT** und **PLAYER_LIGHT** resultierende Lichtwert von Böden und Decken nie negativ wird.

9. Things, Actors, Ways

Things sind 'eindimensionale' Gegenstände, die - im Gegensatz zur Wall - nur einen Vertex besitzen. Ihre Eigenschaften werden in der Thing-Definition festgelegt:

THING *Keyword* { }

Ein **Actor** unterscheidet sich von einem Thing nur durch komplexeres Aktions- und Reaktionsverhalten.

ACTOR *Keyword* { }

Der auf diese Weise definierte Actor kann im WED an irgendeiner Stelle des Spielfelds plaziert werden. Außerdem kann man ihm einen **Way**, d.h. eine Liste von räumlichen Punkten (**Waypoints**) zuteilen. Eine solche Way-Definition besteht einfach aus dem Schlüsselwort **WAY**, gefolgt von einem Way-Namen, dem dann per WED die Waypoint-Liste zugeordnet wird:

WAY *Keyword*;

Der Actor, dem dieser Way zugewiesen wurde, 'wandert' zyklisch von einem zum nächsten Waypoint. Dabei findet keine Kollisionserkennung statt, es sei denn, das Actor-Flag **CAREFULLY** (s.u.) ist gesetzt.

Eine Thing- oder Actor-Definition enthält in der angegebenen Reihenfolge die folgenden Schlüsselwort-Zuweisungen:

- **TEXTURE** *Texture*; Die Textur, entweder eine Bitmap- oder eine Model-Textur. Bei mehrseitigen Bitmap-Texturen wechselt die Texturseite je nach dem Winkel, unter dem das Thing oder der Actor betrachtet wird.

- **ATTACH** *Texture;* **ATTACH**-Textur für das Objekt. Die Anzahl der Phasen (**CYCLES**) und Seiten (**SIDES**) muss entweder der Objekt-Textur entsprechen oder 1 sein. Things und Actors darf nur eine einzige **ATTACH**-Textur zugewiesen werden; der Textur-Parameter **ATTACH** selbst wird ignoriert.
- **CYCLE** *n;* Augenblickliche Animationsphase der Textur.
- **OVERLAY** *Overlay;* Weist dem Thing oder Actor ein Schlüsselwort für ein Overlay (s.u.) zu, z.B. zum Aufnehmen des Objekts mit der Maus oder zur Darstellung in einem Inventory.
- Beispiel:**
 SET MY.INVISIBLE,1; // disappear
 SET MSPRITE,MY.OVERLAY; // appear
Mit diesen Zeilen in einer Aktion, die durch Textur-**IF_KLICK** ausgelöst wird, läßt sich der Gegenstand auf den Mauszeiger 'nehmen'.
- **TARGET** *Keyword;* Weist einem Actor ein Ziel oder eine Primitiv-Verhaltensweise zu. Durch Kombination bzw. event-gesteuertes Wechseln dieser **TARGETs** kann der Actor ein komplexes Verhalten zeigen. Folgende Ziele lassen sich angeben:
- NULL** Der Actor bewegt sich nicht.
- MOVE** Der Actor bewegt sich geradeaus in Richtung seines **ANGLE**. Mit dem Parameter **SPEED** (s.u.) wird seine horizontale Geschwindigkeit vorgegeben. **VSPEED** bestimmt den Tangens des vertikalen Winkels, mit dem sich der Actor auf- oder abwärts bewegt; mit **VSPEED** = 1 bewegt sich der Actor z.B. im Winkel von 45° aufwärts. Ist sein **CAREFULLY**-Flag (s.u.) gesetzt, wird bei jedem Überqueren einer Regionsgrenze seine **IF_ARRIVED**-Aktion ausgelöst.

- BULLET** Wie **MOVE**; jedoch wird beim Auftreffen auf ein Hindernis die **IF_HIT**-Aktion des Actors ausgelöst. Das Flag **CAREFULLY** muß zur Kollisionserkennung gesetzt sein.
- STICK** Der Actor versucht mit seiner Maximalgeschwindigkeit **SPEED** seine augenblickliche Entfernung und seinen Winkel zum Player beizubehalten. Mit den Parametern **REL_ANGLE** und **REL_DIST** (s.u.) kann man den vom Actor angestrebten Winkel zur Player-Blickrichtung und die angestrebte Distanz zum Player nachträglich einstellen. Bei Setzen des **TARGET STICK** werden **REL_ANGLE** und **REL_DIST** automatisch auf die aktuellen Werte gesetzt.
- Ist das **CAREFULLY**-Flag gesetzt, verändern Kollisionen des Actors mit Wänden die Skills **IMPACT_VX**, **IMPACT_VY** und **IMPACT_VROT** (s.u.), ähnlich wie Player-Kollisionen. Durch **STICK**-Actors kann dem Player eine räumliche 'Struktur' gegeben werden, so daß er z.B. nur seitwärts in schmale Gänge hineinpaßt.
- FOLLOW** Der Actor bewegt sich mit **SPEED** auf den Player zu und versucht - wenn **VSPEED** > 0 ist - auch seine Höhe der des Players anzupassen. Die maximale Vertikalgeschwindigkeit wird dabei durch ein positives **VSPEED** vorgegeben. Ist **VSPEED** negativ, so bewegt sich der Actor maximal mit **SPEED** aufwärts und mit dem Betrag von **VSPEED** abwärts. Ist das **CAREFULLY**-Flag gesetzt, wird bei jedem Überqueren einer Regionsgrenze die **IF_ARRIVED**-Aktion ausgelöst.
- REPEL** Wie **FOLLOW**; der Actor bewegt sich jedoch vom Player weg, wobei sich seine Höhe nicht verändert.
- VERTEX** Der Actor bewegt sich auf die Position zu, die seinen Zielkoordinaten **TARGET_X** und **TARGET_Y** (s.u.) entspricht. Bei Erreichen dieser Zielposition wird die **IF_ARRIVED**-Aktion ausgelöst.

HOLD

Nur für polygonale (MDL-)Actors: Der Actor erscheint direkt vor dem Player, unabhängig von dessen Position und Winkel. 3D-Waffen oder Werkzeuge können auf diese Weise definiert werden. Die Ausrichtung des Actors wird von seinem **ANGLE**, seine Position relativ zum Player von seinen **HEIGHT**, **TARGET_X** und **TARGET_Y** Parametern bestimmt. **Beispiel:**

```
MODEL sword_mod,<sword.mdl>;
TEXTURE sword_tex {
    SCALE_X      32;
    SCALE_Y      32;
    MODEL        sword_mod;
}
ACTOR sword_arm {
    TEXTURE      sword_tex;
    TARGET       HOLD;
    ANGLE        3.14;
    TARGET_X     1;
    TARGET_Y     2;
    HEIGHT       -1;
}
```

Die angegebene Parameter-Reihenfolge muss eingehalten werden. Es darf nur ein **HOLD**-Actor sichtbar sein. Es können mehrere definiert und gewechselt werden, doch die unbenutzten müssen **INVISIBLE** gesetzt werden.

NODE1
NODE2

Der Actor wird von dem über **NODE** mit der angegebenen Nummer angeschlossenen Player 'ferngesteuert' (nur Commercial- und Profi-Version). Solange noch keine Kommunikation mit dem Node zustandegekommen ist, ist der Actor unsichtbar. Entspricht die Nummer (1 oder 2) *n* dem aktuellen Node (Befehlszeilenoption **-NODE**), 'erbt' der Player beim Spielstart die Position und Blickrichtung von diesem Actor.

Way		Angabe eines Way-Schlüsselworts weist dem Actor diesen Way zu. Der Actor begibt sich sofort mit SPEED zum ersten Waypoint. Das Schlüsselwort WAYPOINT (s.u.) wird auf den Wert 1 gesetzt. VSPEED wird ignoriert. Bei jedem Erreichen eines Waypoints wird die IF_ARRIVED -Aktion ausgelöst.
Wall, Thing, Actor		Angabe des Schlüsselworts oder Synonyms für eine Wand, ein Thing oder einen weiteren Actor weist dem Actor den ersten Vertex bzw. die Position dieses Objekts als Ziel zu, welches er mit SPEED und VSPEED zu erreichen versucht (wie bei FOLLOW). Gibt es mehrere Objekte mit dem gleichen Schlüsselwort, bewegt er sich auf das mit der niedrigsten WED-Indexnummer zu. Beim Erreichen des Ziels wird die IF_ARRIVED -Aktion ausgelöst.
► WAYPOINT	<i>n;</i>	Weist dem Actor als Ziel einen Positionspunkt innerhalb des Way zu. Der Actor bewegt sich direkt auf die betreffende Position zu. Während des Spiels läßt sich unter diesem Schlüsselwort die Nummer des nächsten anzusteuernenden Waypoints auswerten.
► TARGET_X ► TARGET_Y	<i>Zahl;</i>	Hat ein Actor als TARGET WAY , FOLLOW oder ein Objekt, so enthalten diese Parameter die Koordinaten seines aktuellen Ziels. Bei TARGET VERTEX können hier seine Zielkoordinaten angegeben werden.
► REL_ANGLE ► REL_DIST	<i>Zahl;</i>	Winkel und Entfernung des Actors mit TARGET STICK zum Player; kann nachträglich verändert werden, um z.B. den Actor um den Player herumzubewegen.

► HEIGHT	<i>Zahl;</i>	Höhe der 'Füße' des Things oder Actors, entweder absolut (bei gesetztem GROUND -Flag, s.u.) oder relativ zur Bodenhöhe seiner Region. Die Perspektive von Things mit relativer HEIGHT wirkt manchmal besser, wenn sie z.B. mit HEIGHT -0.25 ein Stück in den Boden hinein versetzt werden.
► ANGLE	<i>Zahl;</i>	Winkel des Objekts in Radian (0..6.28). Wird auf den per WED angegebenen Winkel addiert und kann durch eine Aktion verändert werden.
► SPEED	<i>Zahl;</i>	Horizontale Geschwindigkeit des Actors in Steps pro Tick, Vorgabe = 0.
► VSPEED	<i>Zahl;</i>	Vertikale Geschwindigkeit des Actors, abhängig vom TARGET entweder als Steigung (Tangens des vertikalen Bewegungswinkels) oder direkt in Steps pro Tick, Vorgabe = 0.
ASPEED	<i>Zahl;</i>	Mit diesem Parameter (default=0=inaktiv) kann die maximale Drehgeschwindigkeit in Radian pro Tick vorgegeben werden. Insbesondere mehrseitige und MODEL -Actors wirken natürlicher, wenn sie sich auf ihren WAYPOINTS nicht ruckartig, sondern in einigen Phasen drehen (ASPEED =0.2).
MAP_COLOR	<i>n;</i>	Farbnummer (0..255, Vorgabe 1) zum Darstellen des Things bzw. Actors auf der Landkarte. Bei Zuweisung von 0 wird das Objekt nicht gezeichnet; bei Zuweisung von 1 wird das Objekt in einer Default-Farbe (Skills COLOR_ACTORS , COLOR_THINGS - s.u.) gezeichnet.

► DIST	<i>Zahl;</i>	Grenzdistanz vom Objekt-Zentrum zum Player, Vorgabe=0. Bei Überschreiten der Grenzdistanz kann eine IF_NEAR- bzw. IF_FAR-Aktion (s.u.) ausgelöst werden. Bei DIST=0 wird IF_NEAR nur bei direkter Berührung des Objekts ausgelöst. Hinweis: Ist die DIST > 0, aber kleiner als die Größe des Objekts (plus Player-Breite), müsste der Player zum Auslösen der IF_NEAR-Aktion in das Objekt eindringen!
► SKILL1... ► SKILL8	<i>Zahl;</i>	Acht 'Universalparameter', die auf die Eigenschaften des Objekts zunächst keinen Einfluß haben. Sie können jedoch innerhalb von Aktionen verändert und ausgewertet werden.
#DISTANCE		Angenäherte (+ / 20%) Distanz des Players zum Zentrum des Objekts; nur gültig bei Objekten innerhalb der CLIP_DIST.
#SIZE_X #SIZE_Y		Horizontale und vertikale Größe des Objekts in Steps; ergibt sich aus Maßen und Skalierung der Textur.
► X , ► Y	<i>Zahl;</i>	Vertex-Position des Things oder Actors; kann zum 'Beamen' verändert werden. Wenn sich dadurch seine Höhe über dem Boden verändert, muss ein SHAKE (s.u.) durchgeführt werden.
► REGION	<i>Region;</i>	Region des Actors, muß ggf. bei Verändern der Position neu zugewiesen werden. Die Region von Things darf nicht verändert werden, sonst funktioniert die Kollisionserkennung nicht mehr!

Mit den folgenden Schlüsselworten lassen sich Aktionen angeben, die durch bestimmte mit dem Thing oder Actor zusammenhängende Ereignisse (Events) ausgelöst werden. Die Aktionen werden nur dann durchgeführt, wenn sich das Objekt entweder innerhalb der **CLIP_DIST** befindet oder aber das Flag **LIBER** (s.u.) gesetzt hat.

- | | | |
|---------------------|----------------|--|
| ► IF_NEAR | <i>Action;</i> | Diese Aktion wird ausgelöst, sobald der Player sich dem Thing oder Actor bis auf eine Grenzdistanz (DIST) genähert hat. Ist keine Grenzdistanz definiert, so wird die Aktion bei jeder Berührung ausgelöst. |
| ► IF_FAR | <i>Action;</i> | Diese Aktion wird ausgelöst, sobald sich der Player über die Grenzdistanz DIST hinaus entfernt hat. |
| ► IF_ARRIVED | <i>Action;</i> | Diese Aktion wird ausgelöst, sobald der Actor mit gesetztem CAREFULLY -Flag eine Regionsgrenze überschritten oder sein Zielobjekt oder den nächsten Waypoint erreicht hat. |
| ► IF_HIT | <i>Action;</i> | Diese Aktion wird ausgelöst, wenn das Objekt mit der SHOOT - oder EXPLODE -Anweisung getroffen wird oder wenn das Objekt mit einem Hindernis kollidiert (s. BULLET , FRAGILE). |
| ► EACH_CYCLE | <i>Action;</i> | Diese Aktion wird nach jedem Animationszyklus der Textur bzw. am Ende einer ONESHOT -Animation ausgelöst. |
| ► EACH_TICK | <i>Action;</i> | Diese Aktion wird nach jedem Bildzyklus (d.h. ca. jede 1/16 Sekunde) ausgelöst. |

Mit dem folgenden Schlüsselwort kann eine ganze Liste von weiteren Schlüsselworten (Flags) 'gesetzt' werden, die die Eigenschaften des Objekts bestimmen. Flags haben bei Auswertung in Aktionen den Wert 1 (= gesetzt) oder 0 (= nicht gesetzt).

FLAGS	<i>Flag1, Flag2...;</i>	Folgende Flags lassen sich in dieser Liste angeben bzw. in Aktionen auswerten oder verändern:
#THING #ACTOR		Das THING -Flag ist bei Things und Actors immer automatisch gesetzt, das ACTOR -Flag ist immer bei Actors gesetzt. Mit diesen beiden Flags läßt sich in Aktionen die Art eines Objekt-Synonyms bestimmen.
► INVISIBLE		Wird dieses Flag gesetzt, so ist das Thing oder der Actor unsichtbar und passierbar, so als sei er nicht vorhanden. Der betreffende Actor bewegt sich nicht, und Events werden nicht ausgelöst. Bei Things oder Actors, die zwar per WDL definiert, aber nicht im Level platziert sind, ist das INVISIBLE -Flag automatisch gesetzt.
► PASSABLE		Wird dieses Flag gesetzt, so ist das Objekt für den Player passierbar (durchlässig).
#VISIBLE		Dieses Flag ist automatisch gesetzt, solange das Thing bzw. der Actor vom Player aus sichtbar ist. Es kann in Aktionen ausgewertet werden.
► BERKELEY		Wird dieses Flag gesetzt, so ist das Objekt inaktiv (INVISIBLE), solange man es nicht sieht und sich außerhalb seiner Grenzdistanz DIST befindet. Das Flag spart Rechenzeit in Levels mit vielen animierten Things und Actors.
► LIBER		Ist dieses Flag gesetzt, so führt der Actor Bewegungen und Aktionen auch dann aus, wenn er sich außerhalb der CLIP_DIST befindet. Das kostet allerdings Rechenzeit... Sinnvoll ist dieses Flag bei Objekten, deren Verhalten unabhängig von der Player-Position sein soll, z.B. bei patroillierenden Wachen, EACH_TICK -Zeitbomben oder Geschossen.

FAR

Actors, die nicht sichtbar und weiter als ihre **DIST** vom Player entfernt sind, führen aus Rechenzeitgründen normalerweise nur jedes 6. Frame eine **TARGET**-Bewegung aus (s. Skill **SKIP_FRAMES**). Ist dies für bestimmte Actors nicht erwünscht, so kann ihr **FAR**-Flag gesetzt werden; in diesem Fall bewegen sie sich immer kontinuierlich, unabhängig von **SKIP_FRAMES**.

►GROUND

Wird dieses Flag gesetzt, so bezieht sich die Höhe des Things oder Actors (**HEIGHT**) nicht auf die Höhe des Bodens ihrer Region, sondern auf die Null-Höhe des Levels. Objekte mit **GROUND** werden in Höhe des Bodens und der Decke ihrer Region 'abgeschnitten', wobei Neigungen nicht berücksichtigt werden. Auf diese Weise können Objekte z.B. in Wasser ein- oder auftauchen.

Things und Actors ohne **GROUND** werden immer vollständig dargestellt, auch wenn sie in Teile des Bodens und der Decke eindringen. An Wänden bzw. Regionsgrenzen werden ihre eindringenden Bereiche allerdings abgeschnitten. Befinden sie sich bei Spielstart in einer mehrstöckigen Region, so werden sie in die oberste Region platziert.

Wird das **GROUND**-Flag nachträglich durch eine Aktion verändert, so wird die **HEIGHT** automatisch umgerechnet.

CANDELABER

Nur wirksam, wenn **GROUND** nicht gesetzt wurde: Das Objekt wird nicht an den Boden, sondern an die Decke seiner Region angepaßt. Sinnvoll zur Darstellung von herabhängenden Objekten (Stalaktiten oder Leuchten).

►SEEN

Dieses Flag ist automatisch gesetzt, sobald das Objekt irgendwann einmal vom Player gesehen wurde. Es wird z.B. von der Automap-Funktion ausgewertet.

- ▶ **MOVED** Dieses Flag wird automatisch gesetzt, wenn sich der Actor im letzten Bildzyklus bewegt hat. Es kann in Aktionen ausgewertet werden und muß dann manuell zurückgesetzt werden.

- ▶ **PLAY** Wird dieses Flag gesetzt, so wird die Textur für einen Zyklus animiert; sie bleibt danach auf der letzten Phase stehen. Bei der betreffenden Textur muß hierzu das Flag **ONESHOT** gesetzt sein. Am Ende der Animation setzt sich das Flag **PLAY** automatisch auf 0 zurück, ggf. wird eine **EACH_CYCLE**-Aktion ausgelöst.

- ▶ **IMMATERIAL** Das betreffende Thing oder der Actor wird nach Setzen dieses Flags von **SHOOT**-Anweisungen und Mausclicks nicht mehr beeinflussen.

- ▶ **SENSITIVE** Nach Setzen dieses Flags wird bei Actors die **IF_HIT**-Aktion ausgelöst, wenn diese mit einem Hindernis kollidieren. Ausserdem lösen **SENSITIVE**-Objekte ihre **IF_NEAR**-Aktion bereits dann aus, wenn sie vom Player erblickt werden.

- ▶ **FRAGILE** Wird dieses Flag gesetzt, kann die **IF_HIT**-Aktion des Actors oder Things durch eine **EXPLODE**-Anweisung (s.u.) ausgelöst werden.

- ▶ **CAREFULLY** Wird dieses Flag gesetzt, führt der Actor auf seinem Weg eine Kollisionserkennung durch, weicht Wänden und anderen Actors aus, erkennt Regionswechsel und kann dadurch - wenn das Flag **GROUND** nicht gesetzt ist - seine Höhe der Regionshöhe anpassen. Dies kostet allerdings Rechenzeit.

MASTER

Wenn bei einem Actor die Flags **MASTER** und **CARE-FULLY** gesetzt sind, löst er - wie der Player - **IF_LEAVE**, **IF_ENTER** und **IF_NEAR**Aktionen aus. Bitte beachten: **IF_NEAR** mit einer **DIST** > 0

funktioniert hier nicht, der Actor muss die Wand bzw. das Objekt direkt durchqueren oder berühren, um die Aktion auszulösen.

SAVE

Wird dieses Flag gesetzt, so werden alle Eigenschaften des Thing beim Spielstand-Speichern mitgespeichert. Das **SAVE**-Flag ist bei Actors automatisch gesetzt.

►FLAG1...**►FLAG8**

Acht 'Universalflags', die auf die Eigenschaften des Objekts zunächst keinen Einfluß haben. Sie können jedoch innerhalb von Aktionen verändert und ausgewertet werden.

Hinweis zu Things und Actors:

Da der Bitmap-Wechsel via **SIDES** bei stillstehenden Things und Actors unnatürlich wirkt, sollten nur bewegte und animierte Actors mehrseitig sein.

10. Aktionen

Aktionen geben den Objekten der Welt das Verhalten pseudo-intelligenter linearer Automaten. Es gibt 'globale' Aktionen, die zu bestimmten Zeitpunkten oder z.B. bei Betätigen einer Taste ausgelöst werden, und objektbezogene Aktionen, die durch **Objekt-Events** wie **IF_NEAR**, **IF_FAR** usw. ausgelöst werden. Eine Aktion besteht aus einer Liste von Anweisungen, die nacheinander ausgeführt werden. Da **ACKNEX** eine Multitasking-Engine ist, können wie im wahren Leben beliebig viele Aktionen gleichzeitig ablaufen. Die Anweisungen stellen eine Art einfache Programmiersprache dar, mit der auf vielfältige Weise das Spielgeschehen beeinflußt werden kann.

ACTION Keyword { }

Es sind folgende Anweisungen innerhalb einer Aktion möglich:

SET [Object1.]Keyword1, [Object2.]Keyword2;

Eine häufig gebrauchte Anweisung: **SET** weist einem beliebigen Schlüsselwort, einem Objekt-Parameter oder Skill (s.u.) einen neuen Wert oder ein anderes Schlüsselwort zu. Auf diese Weise lassen sich auch Aktionen selbst während des Spiels 'umschalten' oder - durch Zuweisung des Schlüsselwortes **NULL** - 'abschalten'. Für *Object1/2* wird das Schlüsselwort oder Synonym (s.u.) der Region, der Wand, des Actors, Things oder der Liste angegeben, deren Parameter verändert werden sollen. Dieses Objekt muß entweder im Level platziert oder zumindest in der WDL-Datei vor der betreffenden Aktion definiert worden sein.

Existieren mehrere Objekte mit dem angegebenen Schlüsselwort im Level, so verändert die Zuweisung nur das jeweils erste per WED platzierte Objekt (s. **SET_ALL**). Für *Keyword1* kann auch ein **FLAG**-Schlüsselwort angegeben werden, das durch die Zuweisung von 0 deaktiviert, durch die Zuweisung von 1 aktiviert wird.

Beispiele:

```
SET tank.CEIL_TEX,water_tex; // Verändert die Region tank.
SET bumper.FLAG3,1; // Aktiviert FLAG3 des Things bumper.
SET IF_S, NULL; // Deaktiviert die S-Taste.
```

SET_ALL *[Object1.]Keyword1, [Object2.]Keyword2;*

Wie **SET**; allerdings werden von **SET_ALL** sämtliche per WED platzierte Objekte oder Regionen mit dem angegebenen Schlüsselwort oder Synonym beeinflusst, nicht nur das erste. Positionen (**X,Y,Z, X1,Y1,Z1, X2,Y2,Z2**) von Objekten lassen sich mit **SET_ALL** allerdings nicht verändern.

ACCEL *Skill1, Zahl / Skill2;*

Addiert die Zahl oder Skill2 zu Skill1, jedoch auf eine spezielle Weise, die einer Beschleunigung ähnelt. Das erste Argument entspricht dabei einer Geschwindigkeit, die sich um den Wert des zweiten Arguments verändert. Trägheits- und Reibungsfaktoren lassen sich über die vordefinierten Skills **INERTIA** (Default = 1) und **FRICTION** (Default = 0.5) einstellen.

RULE *Skill / [Object.]Keyword = Ausdruck;*

Weist dem angegebenen Skill oder Objekt-Parameter das Resultat eines mathematischen Ausdrucks zu, in welchem Zahlen, weitere Skills, Objekt-Parameter, Klammern, die Grundrechenarten und diverse mathematische Funktionen vorkommen dürfen. Folgende mathematische Funktionen sind innerhalb des Ausdrucks zulässig:

SIN(x),COS(x),TAN(x) ▶ trigonometrische Funktionen

ASIN(x),ACOS(x) ▶ inverse trigonometrische Funktionen

LOG(x),LOG10(x),LOG2(x) ▶ Logarithmus von x

RANDOM(x) ▶ Zufallszahl zwischen 0 und x

SQRT(x) ▶ Quadratwurzel von x

SIGN(x) ▶ -1 wenn ($x < 0$), 1 wenn ($x > 0$), 0 wenn ($x = 0$)

ABS(x) ▶ Betrag von x

INT(x) ▶ Ganzzahliger Wert von x

EXP(x) ▶ e hoch x

Außer den Grundrechen-Operatoren + - * / werden noch folgende Operatoren unterstützt (nur für ganzzahlige Werte):

% ▶ Modulo (ganzzahliger Rest bei Division)

| ▶ Bitweise ODER-Verknüpfung

^ ▶ Bitweise Exclusive ODER-Verknüpfung

& ▶ Bitweise UND-Verknüpfung

Außerdem lassen sich Rechenoperatoren mit dem "="-Zeichen verknüpfen:

+= ▶ addiert den Ausdruck auf den ersten Parameter

-= ▶ subtrahiert den Ausdruck vom ersten Parameter

***=** ▶ multipliziert den ersten Parameter mit dem Ausdruck

/= ▶ dividiert den ersten Parameter durch den Ausdruck

Aufgrund der begrenzten Rechengenauigkeit (Festkommaarithmetik mit 3 Nachkommastellen) sollten Zahlen und Zwischenwerte den Betrag 0.01 nicht

unterschreiten. Ungültige Operationen - z.B. die Quadratwurzel aus einer negativen Zahl - führen zum Abbruch mit Fehlermeldung!

Beispiele:

```
RULE x = (a + 1) * b / c;  
RULE z = x % 10;  
RULE actor.ANGLE = ASIN(3*x + 0.5);  
RULE x += 1;           // Skill x um 1 erhöhen  
RULE x += 2*TIME_CORR; // Skill x zeitkorrigiert um 2 erhöhen
```

Zu beachten ist, daß **RULE** sich nur auf numerische Werte bezieht, während alle anderen Parameter - Flags, Texturen, Aktionen usw. - mit **SET** oder **SET_ALL** verändert oder zugewiesen werden müssen!

IF (Ausdruck) { Anweisungen... }**IF (Ausdruck) { Anweisungen... } ELSE { Anweisungen... }**

führt die Anweisungen zwischen dem ersten Paar geschweiften Klammern nur dann aus, wenn der *Ausdruck* zwischen den runden Klammern wahr (d.h. ungleich Null) ist. Andernfalls werden die Anweisungen zwischen dem zweiten Paar geschweiften Klammern (nach **ELSE**) ausgeführt. **ELSE** und das zweite Klammernpaar können weggelassen werden.

Der Ausdruck besteht in der Regel aus einem Vergleich numerischer Werte oder weiterer Ausdrücke, oder aus einem Vergleich von Flags, Texturen oder anderer Parameter. Nur Parameter des gleichen Typs können verglichen werden. Bei Flags kann mit 1 (gesetzt) oder 0 (nicht gesetzt) verglichen werden. Für den Vergleich nicht-numerischer Parameter kann der Wert **NULL** für Nichtexistenz benutzt werden. Die folgenden Operatoren stehen zum Vergleich zweier Parameter oder Ausdrücke zur Verfügung:

- || ▶ wahr, wenn einer wahr ist (ODER)
- && ▶ wahr, wenn beide wahr sind (UND)
- != ▶ wahr, wenn beide ungleich sind
- = ▶ wahr, wenn beide gleich sind
- < = ▶ wahr, wenn der erste Ausdruck gleich oder kleiner ist
- > = ▶ wahr, wenn der erste Ausdruck gleich oder größer ist
- < ▶ wahr, wenn der erste Ausdruck kleiner ist

- > ▶ wahr, wenn der erste Ausdruck größer ist

Die Vergleichsoperatoren ergeben einen Wert von 0 für falsch, und 1 für wahr. Vergleiche können mittels Klammern kombiniert bzw. verschachtelt werden.

Beispiele:

```
IF (x<0) {    // wenn x kleiner ist als 0, dann setze y und z auf -1
    RULE y=-1;
    RULE z=-1;
} ELSE {
    RULE y=1; // andernfalls setze y und z auf 1
    RULE z=1;
}
```

```
IF (((x+3)<9) || (y==0)) {
// setze z auf 10, falls x+3 kleiner ist als 9, oder y = 0 ist
    RULE z=10;
} ELSE {
// andernfalls setze z auf 5
    RULE z=5;
}
```

Bitte beachten Sie, daß der "ist gleich"-Vergleich mit "=" und nicht, wie man erwarten könnte, mit "==" durchgeführt wird! Eine **IF**-Anweisung darf zwischen den geschweiften Klammern einer anderen **IF**-Anweisung stehen; auf diese Weise können mehrere **IF**-Anweisungen 'verschachtelt' werden.

WHILE (Ausdruck) { Anweisungen... }

BREAK;

CONTINUE;

Die **WHILE** Anweisung wiederholt alle Anweisungen zwischen den geschweiften Klammern so lange, wie der Ausdruck wahr bzw. ungleich 0 ist. Der Ausdruck wird hierzu am Beginn jeder Wiederholung ausgewertet. **WHILE** wird oft benutzt, um einen Wert langsam zu verändern, z.B. um eine Tür zu öffnen oder einen Lift hochzufahren; hierbei steht in der Regel noch eine **WAIT**-Anweisung (s.u.) zwischen den geschweiften Klammern.

The **BREAK**-Anweisung beendet die Wiederholungen und fährt mit der ersten Anweisung nach der geschlossenen geschweiften Klammer fort. Die **CONTINUE**-Anweisung beginnt unmittelbar mit einer neuen Wiederholung.

Als **Beispiel** für eine Anwendung von **WHILE** hier eine Aktion, die bei Betreten einer Region diese wie einen Fahrstuhl auf- oder abwärtsfahren läßt. Die Fahrstuhl-Plattform soll sich zwischen den Höhen 0 und 10 bewegen:

```
REGION lift {  
    ....  
    IF_ENTER lift_start;  
}  
  
ACTION lift_start {  
    IF (lift.FLOOR_HGT>=10) {      // Lift schon oben?  
        WHILE (lift.FLOOR_HGT>0) { // dann runterfahren  
            RULE lift.FLOOR_HGT+=-0.2;  
            PLAY_SOUND lift_rumpel, 0.3; // Liftgeräusch  
            WAIT 1;  
        }  
    } ELSE {  
        WHILE (lift.FLOOR_HGT<10) { // rauffahren  
            RULE lift.FLOOR_HGT+=0.2;  
            PLAY_SOUND lift_rumpel, 0.3;  
            WAIT 1;  
        }  
    }  
}
```

GOTO *Label;*

Absoluter Sprung: Springt zu einer 'Zielmarke' in der Aktion und fährt ab dieser Marke mit den weiteren Anweisungen fort. *Label* kann ein beliebiges Schlüsselwort sein, das gefolgt von einem Doppelpunkt als Zielmarke irgendwo zwischen zwei Anweisungen in der Aktion steht.

Beispiel:

```
IF (energy<50) { // Wenn energy unter 50...  
    GOTO alarm;  
}  
...           // weitere Anweisungen  
alarm:  
    PLAY_SOUND alert, 0.3; // sonst gib akustischen Alarm  
    FADE_PAL red_pal,0.5; // und optischen Alarm.
```


CALL *Action;*

Führt sofort die angegebene Aktion aus und fährt dann - entweder nach dem Ende oder während einer **WAIT**-Anweisung (s.u.) in der neuen Aktion - mit der nächsten Anweisung in der aktuellen Aktion fort.

END;

Beendet diese Aktion.

EXCLUSIVE;

Beendet sämtliche noch laufenden Aktionen, die von dem auslösenden Objekt (Wall, Thing, Actor, Region) früher ausgelöst wurden. Diese Anweisung kann benutzt werden, um zu verhindern, dass mehrere Aktionen des gleichen Actors parallel laufen und sich gegenseitig stören.

WAIT *Zahl / Skill;*

Hält die Ausführung der Aktion für die angegebene Zahl von Bildzyklen an. Anstelle einer festen Zahl kann auch ein Skill angegeben werden, z.B. **TIME_FAC**. Alle anderen Aktionen laufen in dieser Zeit normal weiter. Das gilt auch für eine eventuelle 'Haupt'-Aktion, von der aus die aktuelle Aktion mit **CALL** gestartet wurde.

EACH_TICK-Aktionen dürfen keine **WAIT**-Anweisungen enthalten, ebenso wenig wie Aktionen, die direkt (**CALL**, **BRANCH**) oder indirekt (**IF_HIT**) von **EACH_TICK**-Aktionen ausgelöst wurden.

WAITT *Zahl / Skill;*

Wie **WAIT**, jedoch wird die Aktion für die angegebene Zahl von Ticks 'zeitkorrigiert' angehalten. Dieser Befehl ist dazu geeignet, eine feste Zeitspanne zu warten.

PLAY_SOUND *Sound, Volume, [Balance]/[Object];*

Startet einen Sound mit einer vorgegebenen Lautstärke. *Sound* ist ein vorher definiertes Schlüsselwort für eine Sounddatei, *Volume* eine Zahl zwischen 0 und 1 oder ein Skill. Mit dem optionalen Parameter *Balance* (Skill oder Zahl zwischen -1 und 1) kann der Sound bei Stereo-Soundkarten zwischen dem linken und rechten Kanal verschoben werden. Wird *Balance* nicht angegeben, so werden beide Kanäle gleich laut angesteuert.

Alternativ kann statt einem *Balance*-Wert das Schlüsselwort eines Objekts (Wall, Thing, Actor) angegeben werden. Der Sound ertönt dann aus Richtung des Objekts innerhalb der **SDIST** von dessen Textur.

Beispiel:

```
SOUND      crrahh,<crow.wav>;
TEXTURE    bird_tex  { BMAPS bird_map; SDIST 200; }
ACTOR      bird      { TEXTURE bird_tex; WAY bird_way; }
ACTION     bird_cry { PLAY_SOUND crrahh,0.5,bird; }
```

PLAY_SOUNDFILE < *Dateiname* >, *Volume, [Balance];*

Wie **PLAY_SOUND**, jedoch wird der Sound direkt von CD-ROM oder Harddisk gespielt, ohne daß Speicher dafür verbraucht wird. Allerdings darf kein Objekt anstelle des Balance-Parameters angegeben werden.

STOP_SOUND;

Bricht alle laufenden Sounds ab.

PLAY_SONG *Music, Volume;*

Startet einen neuen Hintergrundsong. *Music* ist ein vorher definiertes Schlüsselwort für eine Midi-Datei, *Volume* kann einen Wert zwischen 0 und 1 annehmen. Der Song wird bis zum nächsten **PLAY_SONG**-Befehl ständig wiederholt. Durch Abspielen eines beliebigen Songs mit dem Volume 0 wird die Hintergrundmusik abgeschaltet.

PLAY_SONG_ONCE *Music, Volume;*

Wie **PLAY_SONG**; der Song wird jedoch nur einmal abgespielt. Nach der letzten Note endet die Hintergrundmusik.

PLAY_CD *Start, End;*

Spielt Audio-Tracks einer CD ab. Voraussetzung ist DOS ab 6.0 und MSCDEX ab 2.2. Die Skills bzw. Zahlen *Start* und *End* geben den ersten und letzten zu spielenden Track an (min.1, max. 99). Ist *End* größer als die Anzahl der Tracks auf der CD, so wird die CD bis zum Ende abgespielt. Ist *Start* 0, wird eine gerade laufende CD angehalten; ist *End* 0, wird sie an der angehaltenen Stelle weitergespielt. Sind beide 0, passiert nichts.

Nach jeder Ausführung von **PLAY_CD** nimmt der vordefinierte Skill **CD_TRACK** die Nummer des Tracks an, der gerade gespielt wird, oder aber den Wert 0, wenn die CD nicht spielt. In der Zwischenzeit ändert sich **CD_TRACK** nicht. Zum ständigen Anzeigen des aktuellen Tracks muss **PLAY_CD 0,0;** wiederholt (z.B. jede Sekunde) ausgeführt werden.

PLAY_FLIC *Flic;*

Spielt eine bildschirmfüllende Animation (nur Profi-Version). *Flic* ist ein vorher via **FLIC** definiertes Schlüsselwort für eine 320x200-FLI- oder FLC-Datei mit eigener Palette. Während des Abspielens stoppen Bildaufbau und Bewegungen; die Palette der Animationsdatei wird aktiviert. Laufende Aktionen laufen weiter und können benutzt werden, um frame-genau zum Flic Sounds und Songs zu starten (s. Skill **FLIC_FRAME**).

PLAY_FLICFILE < *Flicdatei* >;

Wie **PLAY_FLIC**; die Animation wird jedoch direkt aus der angegebenen Datei gespielt, und der von der Animation belegte Speicherplatz wird nach dem Abspielen wieder freigegeben. Dieser Befehl erfordert einen Disk-Zugriff. Er ist dann vorzuziehen, wenn lange Animationen abgespielt werden sollen, ohne sie ständig im Speicher zu halten.

STOP_FLIC;

Bricht den augenblicklich laufenden Flic ab.

BEEP;

Spielt eine kurze Tonfolge aus dem PC-Lautsprecher. Nützlich während der Level-Entwicklung, um schnell zu erkennen, daß eine bestimmte Aktion ausgelöst wurde.

INPORT *Skill, adr;*

Der betreffende *Skill* wird aus dem I/O-Port Nr. *adr* gelesen (nur Profi-Version). Mit dieser Anweisung kann man direkt auf I/O-Ports zugreifen, um z.B. externe Geräte oder Tracker zu steuern.

Beispiel:

```
SKILL  input {}
SKILL  my_port { VAL 372; }
INPORT input,my_port; // set input skill to the content of
                       // port #372
```

OUTPORT *Zahl/Skill, adr;*

Die angegebene Zahl bzw. der *Skill* wird an den I/O-Port *adr* ausgegeben (nur Profi-Version). Nur Werte von 0..255 sind zulässig.

FADE_PAL *Palette, Zahl / Skill;*

Setzt bzw. wechselt die aktuelle Palette durch Überblendung. *Palette* ist das Schlüsselwort für eine neue Palettendefinition, auf die von der aktuellen Palette aus überblendet werden soll. Die Zahl bzw. der Skill geben den Überblendungsfaktor an (Wertebereich 0..1); durch successives Erhöhen des Faktors wird ein sanftes Überblenden erreicht. Ein Überblendungsfaktor von 1 oder mehr macht die angegebene Palette zur neuen aktuellen Palette.

Unterscheiden sich die Paletten nicht nur in ihren Farbtönen, sondern auch in ihren Farb-Ranges, kann mit einem Faktor ≥ 2 eine neue Shading-Tabelle berechnet werden, was allerdings bei gesetztem **AUTORANGE**-Flag ca. 0.3 Sekunden dauert. Bei einem Überblendungsfaktor ≥ 3 wird ggf. auch eine neue **BLUR**-Tabelle berechnet (ca. 2 Sekunden).

Beispiel:

```
PALETTE black_pal { PALFILE <BLACK.PCX>; } // all colors just black
PALETTE level_pal { PALFILE <MYPAL.PCX>; } // define it AFTER black_pal!!
SKILL  fade {}
ACTION fade_out { // fades the picture softly to black
    SET    fade,0;
    WHILE (fade<1) {
        RULE  fade += 0.1;
        FADE_PAL  black_pal,fade;
        WAIT  1;
    }
} // now everything is dark!
```

SETMIDI *Channel, status;*

GETMIDI *Channel, status;*

Typ eines Midi-Kanals setzen oder abfragen. *Channel* muß zwischen 0..15 liegen. Für *status* sind die folgenden Midi-Typen verfügbar: 0 = FM mode (default), 1 = digital mode (nur mit DDK-Option) und 2 = external mode (Midi-Output via Wave-Port oder externen Midi-Port der Soundkarte).

MIDI_COM *Statusb, data1, data2;*

Sendet Daten direkt an einen Midi-Kanal. Der Skill *Statusb* setzt sich aus der Summe von (Midi command*16) plus Kanalnummer (0..15) zusammen.

Beispiel:

```
SKILL stbef { VAL 11; } // command
SKILL statusb { VAL 0; } // status byte
SKILL channel { VAL 0; } // channel
SKILL data1 { VAL 123; } // 1. data byte
SKILL data2 { VAL 0; } // 2. data byte
ACTION PlayNoteTest {
    SET      stbef,9;      // command
    SET      data2,100;    // volume
    WHILE (data1<200) {
        RULE      statusb = 16*stbef+channel;
        MIDI_COM  statusb,data1,data2;
        RULE      data1 += 5;
        WAIT      1;
    }
}
```

Beachten Sie bitte, daß der Midi-Port nicht gepuffert ist und die Daten maximal mit 31 kBit/sec überträgt.

INKEY *String;*

Kopiert Tastatureingaben in den String mit dem angegebenen Namen oder Synonym. Hierbei wird automatisch die lokale Tastaturbelegung eingeschaltet. Die Anweisung wartet auf den Abschluß der Eingabe mit **[Return]** und fährt erst dann mit der Aktion fort, ähnlich wie **WAIT**. Allerdings werden während einer **INKEY**-Eingabe keine **SAVE/LOAD**-Anweisungen ausgeführt. Mit **[Esc]**, **[Up]**, **[Down]**, **[PgUp]** oder **[PgDn]** kann die Eingabe jederzeit abgebrochen werden, der vorherige String-Inhalt wird dann wiederhergestellt. Der eingegebene Text kann mit **[BackSpace]**, **[Del]**, **[Cursor rechts]** und **[Cursor links]** editiert werden. Die Tastaturbelegung entspricht dabei der jeweiligen Länderbelegung.

Steht der String in einem am Bildschirm angezeigten **TEXT** (s.u.), so sind die Eingaben und der Cursor (blinkendes Zeichen 127 aus dem zugehörigen **FONT**) sichtbar. Ist das Ende des Strings erreicht (ergibt sich aus der Länge des ursprünglichen Strings in der **STRING**-Definition), werden keine weiteren Tastatureingaben außer **[Return]** oder einer Abbruchtaste mehr angenommen. Leerzeichen am Schluss des eingegebenen Strings werden abgeschnitten. Nach Abschluß der Eingabe steht der Skill **RESULT** auf -1 bei Abbruch mit **[Esc]**, bei anderen Abbruch-Tasten auf 72 fuer **[Up]**, 73 fuer **[PgUp]**, 80 fuer **[Down]** und 81 fuer **[PgDn]**, und andernfalls auf 0. Die Länge des eingegebenen Strings lässt sich über den vordefinierten Skill **STR_LEN** auswerten.

SET_STRING *String1, String2;*

kopiert den Inhalt des zweiten in den ersten String. Die Länge des Ziel-Strings sollte größer oder gleich der Länge des Quell-Strings sein, sonst wird der String abgeschnitten.

ADD_STRING *String1, String2;*

Wie **SET_STRING**, jedoch wird der zweite String an den ersten angefügt. Die ursprüngliche Länge des ersten String wird dabei nicht überschritten.

TO_STRING *String, Skill;*

Kopiert den Zahlenwert des Skill in den betreffenden String.

SET_SKILL *Skill, String;*

Der Wert des Skills wird auf die erste im String stehende Zahl gesetzt.

FIND *Text,String;*

Findet den angegebenen String innerhalb aller Strings des angegebenen **TEXT**. Mit dieser Anweisung wird der **OFFSET_Y** und der **INDEX**-Parameter des Text auf denjenigen String gesetzt, dessen erste Buchstaben dem Inhalt des angegebenen String entsprechen. Der Skill **RESULT** wird ebenfalls auf den betreffenden **INDEX** gesetzt. Wird keine Entsprechung gefunden, so wird **RESULT** auf -1 gesetzt, und **OFFSET_Y** und **INDEX** werden nicht verändert. Nach der Anweisung enthalten - unabhängig von **RESULT** - die vordefinierten Skills **LINES** und **SIZE_Y** die Anzahl der Charakter- bzw. Pixel-Zeilen des gesamten **TEXT**.

SET_INFO *String,Object;*

Diese Anweisung ist der eingebaute "Debugger" für Objekt-Aktionen. Sie schreibt alle interessanten Parameter des Objekts (Wall, Thing, oder Actor) in den angegebenen String. Der String sollte mindestens 1000 Zeichen lang sein, um alle Informationen unterzubringen. Diese Anweisung steht nur im WED, nicht im Runtime-Modul zur Verfügung.

Beispiel für eine Debug-WDL-Datei, die bei Mausklick rechts das Game 'einfriert' und einen Mauszeiger erscheinen läßt. Die Parameter jedes mit der linken Maustaste angeklickten Objekts erscheinen dann auf dem Bildschirm:

```

////////////////////////////////////
// At first the code to switch on and move the mouse:
OVLY      mouse_ovr,<arrow.pcx>;
OVERLAY   mouse_sprite { OVLYS mouse_ovr; }
MSPRITE   mouse_sprite;

ACTION mouse_toggle {           // switches the mouse on and off
    RULE MOUSE_MODE += 2;
    IF (MOUSE_MODE > 2) {       // was it already on?
        RULE MOUSE_MODE=0;
        END;
    }
    WHILE (MOUSE_MODE > 0) {
        WAIT 1;                 // now move it over the screen
        RULE MOUSE_X += 2*MICKEY_X;
        RULE MOUSE_Y += 2*MICKEY_Y;
    }
};
////////////////////////////////////

```

```
// Now the WDL code for showing the debugging info:
STRING    info_strg,"
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
long string                                     "; // just a very

TEXT      info_txt {
        POS_X       4;
        POS_Y       4;
        FONT        debug_font;
        STRING      info_strg;
}

SYNONYM check_obj { TYPE ACTOR; }
SKILL running { VAL 0; }

ACTION scan_obj { // permanently update the object info
        IF (running != 0) {
                END;
        } // do nothing if action is already running
        RULE   running=1; // (remember, this is multitasking!)
        WHILE (running) {
                SET_INFO  info_strg,check_obj;
        WAIT          1;
        }
}

ACTION obj_click {
        BEEP;
        SET     check_obj,Touched;
        SET     info_txt.VISIBLE,1;
        CALL    scan_obj;
```



```
}  
ACTION freeze_unfreeze {  
    IF (MOUSE_MODE == 0) { // if the mouse is about to appear  
        SET MOVE_MODE,0.5; // freeze anything  
    } ELSE {  
        SET MOVE_MODE,1;  
    }  
    CALL mouse_toggle;  
}  
  
IF_RIGHT freeze_unfreeze;  
IF_KLICK obj_click;
```

PRINTFILE *"Name",Zahl/Skill;*

Definiert den Namen einer Textdatei, in welche die Werte von Skills und Strings zur Laufzeit ausgegeben werden können. Der Name setzt sich wie bei SAVE aus dem angegebenen String (max. 5 Buchstaben) plus einer maximal dreistelligen Zahl plus der Extension .TXT zusammen. Der Default-Name ist "PRINT0.TXT".

PRINT_VALUE *Skill;*

Schreibt den Skill als Zahl mit 3 Nachkommastellen in die per **PRINTFILE** angegebene Textdatei. Existiert die Datei schon, wird der ursprüngliche Datei-Inhalt nicht gelöscht, sondern die Zahl wird am Ende der Datei angehängt.

PRINT_STRING *String;*

Wie **PRINT_VALUE**, jedoch wird der Inhalt des String, der auch direkt als Buchstabenfolge zwischen Anführungszeichen angegeben werden kann, in die Datei geschrieben. "\n" innerhalb des Strings schreibt einen Zeilenumbruch in die Datei.

FREEZE *Bmap, Zahl/Skill;*

Kopiert das aktuelle 3D-Bild ohne Panels und Overlays verkleinert in eine bereits existierende Bitmap, die dann z.B. als **PICTURE** für einen gespeicherten Spielstand in einem Panel angezeigt werden kann. *Bmap* ist das Schlüsselwort, dem die Bitmap zugewiesen wurde, *Zahl/Skill* gibt den Verkleinerungsfaktor an (1..16). Da die Größe der Bild-Kopie vom eingestellten **MOTION_BLUR** abhängt, sollte dieser vorher auf 0 gestellt werden. Die modifizierte Bitmap wird mit der Anweisung **SAVE_INFO** abgespeichert; diese darf dann allerdings frühestens einen Framezyklus nach der **FREEZE**-Anweisung erfolgen (**WAIT**-Anweisung einfügen!).

SCREENSHOT *"Name", Zahl/Skill;*

Speichert den Inhalt des Bildschirms als PCX-Datei mit dem angegebenen Namen (max. 5 Stellen, in Anführungszeichen) plus einer dreistelligen Nummer (skill) und der Extension "PCX" ab. Die Größe und Auflösung des Bildes hängt vom aktuellen **MOTION_BLUR** ab, also sollte dieser einen Framezyklus zuvor vorübergehend auf 0 gesetzt werden.

NEXT_MY;**NEXT_THERE;****NEXT_MY_THERE;**

Diese Anweisungen verändern Synonyme (s.u.), um mehrere Objekte per **WHILE** oder **GOTO** zu manipulieren. **NEXT_MY** setzt das vordefinierte Synonym **MY** auf das nächste Objekt mit dem gleichen Namen. Wenn das letzte Objekt erreicht ist, wird **MY** wieder auf das erste plaziert. **NEXT_THERE** setzt das Synonym **THERE** auf die nächste Region mit dem gleichen Namen. **NEXT_MY_THERE** setzt das Synonym **MY** auf das nächste Objekt innerhalb der gleichen Region.

Beispiel (um alle Regionen mit Namen *lift1* um 1 Step anzuheben):

```
SET   THERE, lift1; // setzt THERE auf die erste Region lift1
loop1:
RULE   THERE.FLOOR_HGT += 1;
NEXT_THERE;         // setzt THERE auf die naechste Region
IF (THERE!=lift1) { GOTO loop1; }
```

LOCATE [*Object*];

Mit dieser Anweisung wird dem Player oder einem Actor die Region, in der er sich gemäß seiner Position befindet, neu zugewiesen. Dies funktioniert auch dann, wenn durch 'Beamen' oder Bewegen eines Actors ohne **CAREFULLY**-Flag dessen ursprüngliche Region 'verlorengegangen' ist. **LOCATE** mit einem Objekt-Parameter findet die Region des betreffenden Things oder Actors, **LOCATE** ohne Parameter die des Players (**HERE**).

DROP *Object*;

Das angegebene Thing oder der angegebene Actor wird in Blickrichtung des Players (**PLAYER_ANGLE**) in der Entfernung des Objekt-Parameters **DIST** platziert. Ist der Mauszeiger aktiv, wird es in dessen Richtung (**MOUSE_ANGLE**) platziert. Ist dies nicht erwünscht, so muß vor der Anweisung dem **MOUSE_ANGLE** der **PLAYER_ANGLE** zugewiesen werden. Damit die Kollisionserkennung funktioniert, muß die neue Position des Objekts in der gleichen Region liegen, in der sich der Player befindet; es ist also zu empfehlen, die **DIST** auf 0 zu setzen.

SHOOT [*Object*];

Eine wichtige Anweisung für Action-Spiele. Fehlt der optionale Parameter *Object*, so wird bei dem nächstgelegenen Objekt - Wand, Thing oder Actor - in Blickrichtung des Players die **IF_HIT**-Aktion ausgelöst. Über den vordefinierten Skill **SHOOT_FAC** wird die 'Schuß-Stärke', über **SHOOT_RANGE** die maximale Entfernung, über **SHOOT_X** und **SHOOT_Y** die Abweichung der **SHOOT**-Richtung vom Mittelpunkt des 3D-Fensters eingestellt (Wertebereich -1..+1). Nach der **SHOOT**-Anweisung enthält der Skill **HIT_DIST** die Entfernung des getroffenen Objektes, der Skill **RESULT** die Trefferstärke ($\text{SHOOT_FAC} * (1 - \text{HIT_DIST} / \text{SHOOT_RANGE})$) und die Skills **HIT_X** und **HIT_Y** die Pixelkoordinaten des getroffenen Texturpixels relativ zur oberen linken Textur-Ecke.

Es werden nur Objekte 'getroffen', deren **IMMATERIAL**-Flag nicht gesetzt ist. Bei Things und Actors werden die transparenten Bereiche der Textur ignoriert. **SHOOT** kann für Schüsse benutzt werden, aber auch z.B. dazu, um nahegelegene Schalter zu betätigen oder Türen zu öffnen.

Wird das Schlüsselwort oder Synonym eines Things oder Actors als *Object* angegeben, so läßt sich mit der Anweisung feststellen, ob der Player von dem

Objekt aus sichtbar ist. In diesem Falle werden die Skills **HIT_DIST** und **RESULT** gemäß der Entfernung von Player und Objekt gesetzt., der Skill **SHOOT_ANGLE** enthält den genauen Winkel vom Actor zum Player. Dieser Winkel kann dann einem abgefeuerten Geschoss oder dem Actor selbst zugewiesen werden (**SET actor.ANGLE, SHOOT_ANGLE;**). Ist der Player vom Objekt aus nicht sichtbar, erhalten **HIT_DIST** und **RESULT** den Wert 0.

Mit dem Skill **SHOOT_SECTOR** (default = 20) kann der Winkelbereich eingestellt werden, in dem der Actor die **SHOOT**-Anweisung durchführt. Der **ANGLE** des Actors liegt im Zentrum dieses Winkelbereichs. Befindet sich der Player außerhalb dieses Sektors, liefert **SHOOT** eine **HIT_DIST** und ein **RESULT** von 0. Hierdurch kann man z.B. erreichen, daß ein Monster den Player nur von vorne erblickt, während man sich von hinten 'heranpirschen' kann.

EXPLODE *Object;*

Löst bei allen Walls, Things oder Actors, deren **FRAGILE**-Flags gesetzt sind und deren nächster Vertex sich innerhalb der Distanz **SHOOT_RANGE** von der Position des Actors mit dem Namen oder Synonym *Object* befinden, die **IF_HIT**-Aktion aus. Der Skill **RESULT** wird je nach der Trefferstärke - wie bei **SHOOT** - auf einen Wert zwischen 0 und 1 gesetzt.

Nach der Anweisung sind die Skills **HIT_DIST** und **RESULT** wie bei **SHOOT** gesetzt, falls der Player selbst getroffen wurde. Der Skill **HIT_MINDIST** enthält entweder die Entfernung des nächstgelegenen getroffenen Objekts oder aber 0, falls sich kein Objekt mit einer **IF_HIT**-Aktion innerhalb der kritischen Entfernung befand.

Falls die **IF_HIT**-Aktion des getroffenen Objekts selbst wieder eine **EXPLODE**-Anweisung enthält, sollte vorher eine Wartepause (**WAIT 1;**) in die Aktion eingefügt werden, damit nicht zwei benachbarte Objekte durch ständige gegenseitige **EXPLODEs** die Engine blockieren.

PUSH *Zahl/Skill;*

Löst bei dem nächstgelegenen sichtbaren Objekt (Wall, Thing oder Actor) innerhalb der von *Zahl* bzw. *Skill* angegebenen Distanz die **IF_HIT**-Aktion aus. Hiermit können z.B. Türen geöffnet oder Aufzüge gestartet werden. Die Skills **HIT_DIST** (entweder 0 oder Entfernung) und **RESULT** werden wie bei **SHOOT** gesetzt.

SHAKE *Object;*

Teilt dem Objekt (Wall, Thing, Actor) mit, daß sich seine Position verändert hat. Eine **SHAKE**-Anweisung ist nach jedem direkten Setzen einer Koordinate erforderlich, wenn sich dadurch die Position oder Bodenhöhe eines Objekts oder die Länge einer Wall verändert.

Beispiel:

```
RULE   MY.X1 += 0.5;  
SHAKE  MY;
```

LIFT *Region, Zahl/Skill;*

Erhöht die Z-Werte aller zur Region *Region* gehörenden Vertices, Things und Actors um den angegebenen Betrag oder den Skill. Diese Anweisung kann benutzt werden, um z.B. die Höhe von Things und Actors auf die Bodenhöhe auf/abfahrender Lifte anzupassen.

Zu beachten ist, daß bei den Wänden und Objekten der betreffenden Region das Flag **SAVE** gesetzt sein muß, wenn Höhenänderungen beim Spielstand-Speichern mit abgespeichert werden sollen. Das gilt auch für die folgenden Regions-verändernden Anweisungen:

TILT *Region, Zahl/Skill;*

Multipliziert die Z-Werte aller zur Region gehörenden Vertices, Things und Actors mit dem angegebenen Betrag oder Skill. Diese Anweisung kann benutzt werden, um die Neigung von Regionen zu verändern.

SHIFT *Region, dx, dy;*

Verschiebt alle zur Region *Region* gehörenden Vertices, Things und Actors um die angegebenen Beträge oder Skills in X- und Y-Richtung des Koordinatensystems. Mit dieser Anweisung können z.B. Fahrzeuge oder Flöße in Bewegung gesetzt werden. Allerdings darf keine Wand der Region bei der Bewegung eine andere Wand berühren oder kreuzen!

ROTATE *Region, Zahl/Skill;*

Dreht sämtliche Wände und Objekte einer Region um den angegebenen Winkel. Das Drehzentrum ist der Regions-Parameter **GENIUS**. Der **GENIUS** darf auch außerhalb der Region liegen und gleichzeitig mehreren Regionen zugeteilt werden, so daß sich diese dann um ein gemeinsames Zentrum drehen. Beim Drehen darf keine Wand der Region eine andere Wand berühren oder kreuzen.

Beispiel:

```
ACTOR wheel_center { TEXTURE any_tex; FLAGS INVISIBLE; }
REGION wheel {
    FLOOR_HGT      2;
    CEIL_HGT       20;
    FLOOR_TEX      wood_tex;
    CEIL_TEX       stone_tex;
    GENIUS         wheel_center;
    EACH_TICK      wheel_rotate;
    FLAGS          SAVE;           // to keep any changes
}
ACTION wheel_rotate {
    ROTATE wheel,0.05;           // 0.05 Rad = ca. 3 Degrees
    RULE   wheel.FLOOR_ANGLE += 0.05; // rotate the texture also
}
```

SAVE *"Name", Zahl/Skill;*

Speichert den kompletten Spielstand unter einem vorgebbaren Namen plus einer Nummer im Spielstandverzeichnis (s. **SAVEDIR**) ab. Der Name der abgespeicherten Datei setzt sich aus den ersten fünf Buchstaben der Buchstabenfolge *Name* plus einer maximal dreistelligen Nummer, die dem Wert von *Skill* entspricht, plus der Extension .SAV zusammen.

LOAD *"Name", Zahl/Skill;*

Lädt, dekomprimiert und entschlüsselt den Spielstand, der unter dem Namen *Name* plus einer maximal dreistelligen Nummer, die dem Wert von *Skill* entspricht, abgespeichert wurde. Ist der Skill **LOAD_MODE** auf 1 gesetzt, so wird beim Laden ein eventueller Wechsel der Bildschirmauflösung unterdrückt.

SAVE_INFO *"Name", Zahl/Skill;*

Wie **SAVE**; speichert jedoch nur alle **LOCAL**- und **GLOBAL**-Skills, alle mit **FREEZE** veränderten Bitmaps und alle mit **INKEY** veränderten Strings im Spielstandverzeichnis ab.

LOAD_INFO *"Name", Zahl/Skill;*

Wie **LOAD**; lädt jedoch nur die mit **SAVE_INFO** abgespeicherten Werte.

SAVE_DEMO *"Name", Zahl/Skill;*

Zeichnet die Bewegung des Players in einer Demo-Datei im Spielstandverzeichnis auf. Der Name der abgespeicherten Datei setzt sich aus den ersten fünf Buchstaben der Buchstabenfolge *Name* plus einer maximal dreistelligen Nummer, die dem Wert von *Skill* entspricht, plus der Extension .REC zusammen.

PLAY_DEMO *"Name", Zahl/Skill;*

Spielt die Demo-Datei ab. Eine beliebige Taste stoppt das Abspielen.

STOP_DEMO;

Stoppt das Aufzeichnen oder Abspielen der aktuellen Demo-Datei.

MAP *< Dateiname > ;*

Levelwechsel; lädt eine neue Welt-Topographie aus der Datei *< Dateiname[.WMP] >* und setzt den Player und alle Actors auf die neuen Startpositionen. In der 'Lite' und 'Commercial'-Version ist die Anzahl der nachladbaren WMP-Dateien auf 100 beschränkt.

LEVEL < *Dateiname* > [, "*Resourcenname*";

Level- und WDL-Wechsel (nur Profi-Version); lädt eine neue Welt-Definition aus dem Script < *Dateiname*[.*WDL*] > , lädt die dort in **MAPFILE** angegebene Topographie und führt die neue Startaction aus. Befindet die WDL-Datei in einer neuen WRS-Resource, so kann diese ebenfalls angegeben werden (nicht in spitzen Klammern, da sie sonst zum Level dazu kompiliert würdel). Ist der Skill **LOAD_MODE** auf 1 gesetzt, so wird beim Level-Laden ein eventueller Wechsel der Bildschirmauflösung unterdrueckt. Sämtliche Regionen, Objekte und Skills - bis auf die **GLOBAL**-Skills, s.u. - werden durch die Objekte des neuen Levels ersetzt.

EXIT [*"Text"/String*];

Beendet das Spiel, kehrt zu DOS zurück. Der optionale Text, der entweder direkt oder als vorher definiertes String-Schlüsselwort angegeben werden kann, wird direkt nach Spielende auf dem DOS-Bildschirm ausgegeben.

Außer den von Objekt-Events ausgelösten Aktionen lassen sich über die folgenden Schlüsselworte Standard-Aktionen vorgeben:

► **EACH_TICK** *Action, Action...*;

Diese Liste von bis zu 16 Aktionen wird nach jedem Bildaufbau, d.h. ungefähr jede Sechzehntelsekunde einmal abgearbeitet. Dabei werden die Aktionen in der angegebenen Reihenfolge nacheinander ausgeführt. Mit der Anweisung **SET EACH_TICK.n, Action** (*n* = 1..16) können Aktionen an beliebigen Stellen in der Liste verändert werden; mit **SET EACH_TICK.n, NULL** wird die betreffende Aktion aus der Liste gelöscht. **EACH_TICK**-Aktionen dürfen keine **WAIT**-Anweisungen enthalten und auch keine **CALL**- oder **BRANCH**-Anweisungen zu Aktionen mit **WAIT**s durchführen.

Sinnvollerweise sollte den einzelnen Listenpositionen Aktionen festgelegter Bedeutung zugeordnet werden, die dann gezielt verändert oder entfernt werden können. **EACH_TICK.16** könnte z.B. für die Bewegung des Players vorgesehen werden

IF_START *Action*;

Die angegebene Aktion, die Palettenwechsel, Titel-Animationen, Songs etc. enthalten kann, wird beim Start des Spiels ausgeführt.

► **IF_LOAD** *Action;*

Die angegebene Aktion wird nach dem Laden eines gespeicherten Spielstands ausgeführt. Sie kann dazu benutzt werden, um die zum Speichern geänderten Panels oder Skills (**MOVE_MODE** etc.) wieder in den Normalzustand zu versetzen.

► **IF_LEFT** *Action;*

Die angegebene Aktion wird bei jeder Betätigung der linken Maustaste und bei Druck auf die Joystick-Taste 1 ausgeführt.

► **IF_MIDDLE** *Action;*

Die angegebene Aktion wird bei Betätigung der mittleren Maustaste oder bei Druck auf die Joystick-Taste 3 ausgeführt.

► **IF_RIGHT** *Action;*

Die angegebene Aktion wird beim Betätigen der rechten Maustaste und bei Druck auf die Joystick-Taste 2 ausgeführt.

► **IF_KLICK** *Aktion;*

Diese Aktion wird ausgeführt, wenn irgendwo im 3D-Fenster mit dem Mauszeiger links geklickt wird, ohne daß ein Objekt oder Panel getroffen wird.

► **IF_MSTOP** *Aktion;*

Diese Aktion wird ausgelöst, wenn der Mauszeiger aktiv ist und die Maus für 1/2 Sekunde stillgehalten wurde. Über den Skill **MOUSE_CALM** kann die Maximalstrecke in Maus-Pixels angegeben werden, die noch als Stillstand gilt (default 3). Durch Auswertung des Skills **MOUSE_MOVING** kann ebenfalls festgestellt werden, ob die Maus stillgehalten wird (0) oder in Bewegung (1) ist.

► **IF_ANYKEY** *Action;*

Die angegebene Aktion wird beim Betätigen irgendeiner Taste ausgeführt.

►IF_F1... *Action;*

Die angegebene Aktion wird beim Betätigen der **[F1]**-Taste ausgeführt. Den folgenden weiteren Tasten können auf die gleiche Weise Aktionen zugeordnet werden: **IF_F2...IF_F12, IF_ESC, IF_TAB, IF_CTRL, IF_ALT, IF_SPACE, IF_BKSP, IF_CUU, IF_CUD, IF_CUR, IF_CUL, IF_PGUP, IF_PGDN, IF_HOME, IF_END, IF_INS, IF_DEL, IF_PAUSE, IF_CAR** (Punkt), **IF_CAL** (Komma), **IF_ENTER, IF_PLUS, IF_MINUS, IF_0...IF_9, IF_A...IF_Z**. Tastenaktionen lassen sich während des Spiels mit der **SET**-Anweisung (z.B. **SET IF_F1, *Action;***) umdefinieren.

11. Synonyme

Häufig sollen mit der gleichen Aktion verschiedene Objekte oder Regionen verändert werden. Um nicht für jedes dieser Objekte eine eigene Aktion definieren zu müssen, benutzt man in der Aktion anstelle des Schlüsselwortes für das Objekt oder die Region ein **Synonym**. Es gibt einige vordefinierte Synonyme (**MY**, **HERE**, **THERE**). Beliebige weitere Synonyme lassen sich vom Anwender definieren:

SYNONYM *Keyword* { ... }

Eine solche Synonym-Definition kann - in der angegebenen Reihenfolge - die folgenden Schlüsselwörter enthalten:

TYPE *Keyword*; Bezieht sich auf die Art des Synonyms. *Keyword* steht für **OVERLAY**, **TEXTURE**, **WALL**, **THING**, **ACTOR**, **REGION**, **PANEL**, **TEXT**, **STRING** oder **ACTION**.

DEFAULT *Keyword*; Optional: Objekt, für das das Synonym bei Spielstart steht.

Mit Synonymen läßt sich arbeiten wie mit normalen Schlüsselworten. Synonyme für Aktionen können mit der Anweisung **CALL** gestartet werden. Synonyme können einander zugewiesen werden:

```
REGION lift_1 { ... }
SYNONYM lift_syn1 { TYPE REGION; }
SYNONYM lift_syn2 { TYPE REGION; }
SYNONYM lift_floor { TYPE TEXTURE; }

ACTION lift1_zu_lift2 {
    SET    lift_syn1, lift_1;           // Region zuweisen
    SET    lift_syn2, lift_syn1;
    SET    lift_floor, lift_syn2.FLOOR_TEX;
}
```

Es gibt folgende vordefinierten Synonyme:

- **HERE** Dieses vordefinierte Synonym steht für die Region, in der sich der Player befindet; wichtig für die Kollisionserkennung und die Höhenanpassung. Bei direktem Versetzen der Playerposition (Skills **PLAYER_X**, **PLAYER_Y**) muß ggf. auch **HERE** neu zugewiesen werden.

- **THERE** Vordefiniertes Synonym für diejenige Region, die entweder die gerade ablaufende Aktion mit **IF_ENTER**, **EACH_TICK** o.ä. ausgelöst hat oder in der sich das Objekt (**MY**, s.u.) befindet, welches die aktuelle Aktion ausgelöst hat.

- **MY** Synonym für dasjenige Objekt, das die gerade ablaufende Aktion ausgelöst hat (über **IF_NEAR**, **IF_FAR**, **EACH_TICK** o.ä.). Auf diese Weise lassen sich gezielt einzelne Objekte ansprechen, ohne deren Index zu kennen (z.B. mit **SET MY.TEXTURE**, **crash_tex**;). Wurde die aktuelle Aktion nicht von einem Objekt ausgelöst, so ist dieses Schlüsselwort undefiniert.

- #HIT** Synonym für das zuletzt vom Player mit der **SHOOT**-Anweisung getroffene oder mit der Maus angeklickte Objekt oder für dasjenige mit der letzten **EXPLODE**-Anweisung getroffene Objekt, das dem Explosionszentrum am nächsten liegt.

- #TOUCHED** Objekt, welches zuletzt mit der Maus berührt wurde.

- #TOUCH_TEX** Textur, die zuletzt mit der Maus berührt wurde.

- #TOUCH_REG** Region, deren Boden oder Decke zuletzt mit der Maus berührt wurde.

►TOUCH_TEXT

Ist diesem Synonym ein **TEXT** zugewiesen, erscheinen **TOUCH**-Texte im Font und mit den Flags des betreffenden Text. Hat der Text das Flag **VISIBLE**, so erscheinen die Texte auf dessen **POS_X/POS_Y**, ansonsten an der Maus.

12. Rollenspiel-Funktionen: Skills

'Rollenspiel' ist ein traditoneller Ausdruck dafür, daß Player und Actors wie im wahren Leben individuelle Eigenschaften (*Skills*) besitzen. Art und Wirkung dieser Eigenschaften lassen sich frei bestimmen. Skills entsprechen ungefähr den Variablen in einer Programmiersprache. Es gibt eine Reihe von vordefinierten Skills, mit denen z.B. das Bewegungsverhalten des Players bestimmt werden kann. Zusätzlich können beliebig viele Skills frei definiert werden - Fitneß, Kampfkraft, Zauberfaktor oder Blutdruck, pH, Alkoholpegel... das bleibt ganz der Phantasie des Autors überlassen.

Ein Skill wird folgendermaßen definiert:

SKILL *Keyword* { ... }

Eine solche Skill-Definition kann - in der angegebenen Reihenfolge - die folgenden Schlüsselwörter enthalten:

- TYPE** *Keyword*; Bezieht sich auf die Art des Skills. *Keyword* steht für **LOCAL**, **GLOBAL** oder **PLAYER** (default). Die Unterschiede betreffen nur das Verhalten des Skills beim Speichern und Laden: **GLOBAL**-Skills werden mit den Anweisungen **SAVE** und **SAVE_INFO** abgespeichert, **LOCAL**-Skills jedoch nur mit **SAVE_INFO** und **PLAYER**-Skills nur mit **SAVE**.
- VAL** *Zahl*; Fixkommazahl, Anfangswert für den Skill. Wird kein Anfangswert angegeben, so erhält der Skill den Anfangswert 0.
- MIN** *Zahl*; Fixkommazahl, untere Grenze des Skill. Der Skill-Wert kann diese Grenze niemals unterschreiten.
- MAX** *Zahl*; Fixkommazahl, obere Grenze des Skill. Der Skill-Wert kann diese Grenze niemals überschreiten.

GLOBAL-Skills eignen sich für alle Werte, die von Level zu Level eines Spiels 'mitgenommen' werden sollen, wie Kampfkraft, Punktezahl oder Schwierigkeitsgrad. Sie können nach Spielstart oder nach dem Wechsel eines Levels (Anweisung **LEVEL**) mit einer **LOAD_INFO**-Anweisung aus dem letzten gespeicherten Spiel-Status zurückgeladen werden.

Bei **LOCAL**-Skills bleibt der letzte Wert beim Spielstand-Laden immer erhalten. Sie sind für User-Einstellungen wie z.B. Lautstärke oder Bildschirm-auflösung geeignet, die sich bei Laden eines alten Spielstands nicht ändern sollen.

Sind die Skills definiert, können ihre Werte ebenso wie bei jedem anderen Schlüsselwort durch Aktionen mit **SET**-, **RULE**- oder anderen Anweisungen verändert werden.

Beispiel: Der Player soll eine Eigenschaft besitzen, die sich HEALTH nennt. Die Health nimmt normalerweise kontinuierlich ab. Bekommt der Player jedoch von einem böartigen Actor einen Hieb verpaßt, so sinkt die HEALTH-Eigenschaft ruckartig um einen bestimmten Betrag, um dann langsamer wieder auf den vorherigen Wert anzusteigen. Das Ganze kann folgendermaßen definiert werden:

```
SKILL health      { VAL 100; }
SKILL recover     { VAL 0; }
SKILL rec_factor  { VAL 0.5; }

ACTION permanent {
    WAITT 16;      // 1 Sek. warten
    RULE health += rec_factor * recover - 0.005;
    RULE recover -= 1; // Wert verringern bis auf 0
}
ACTION aua {
    RULE health -= 25; // Getroffen - Health sinkt ab
    RULE recover = 10; // Für 10 Sekunden Erholungszeit
}

EACH_TICK permanent;
```

Zu beachten ist, daß die Zahlenwerte so gewählt sind, daß sich der Player nach jedem Treffer nach genau 10 Sekunden erholt hat - falls ich mich nicht verrechnet habe...

Außer den selbstdefinierten Skills stehen folgende vordefinierten Skills zur Verfügung, die in Aktionen ausgewertet und - sofern sie mit dem Dreieck ► gekennzeichnet sind - auch verändert werden können:

► **SCREEN_WIDTH**► **SCREEN_HGT**

Breite und Höhe des 3D-Fensters in Pixeln. Der Vorgabe- und Maximalwert ergibt sich aus der mit **VIDEO** eingestellten Bildschirmauflösung. Die Fenster-Größe läßt sich während des Spiel verändern. Zu beachten ist, daß die Breite durch 16 und die Höhe durch 2 teilbar sein muß.

Durch Neudefinition dieser Skills läßt sich die Anfangs-Bildschirmgröße reduzieren, was Speicher spart. Allerdings darf die neudefinierte Breite und Höhe dann während des Spiels nicht mehr überschritten werden.

► **SCREEN_X**► **SCREEN_Y**

Horizontaler und vertikaler Abstand des 3D-Fensters von der oberen linken Bildschirmkante in Pixeln; Vorgabe jeweils 0. Der maximale Abstand ergibt sich aus der mit **VIDEO** eingestellten Bildschirmauflösung abzüglich **SCREEN_WIDTH** bzw. **SCREEN_HGT**. Der horizontale Abstand muß durch 4 teilbar sein.

► **ASPECT**

Höhen-zu-Breiten-Verhältnis der Bilddarstellung im 3D-Fenster, Wertebereich 0.1..10. Der Vorgabewert von 1 stellt ein Höhen-zu-Breiten-Verhältnis von 1:1 ein.

► **EYE_DIST**

Augen-Abstand für 3D-Brillen (default 0.5 Steps). Hierzu muss mit der Befehlszeilenoption **-3D-SI-MEYE** gestartet werden. Wird dieser Skill auf 0 gesetzt, ist der Stereoscopic Effekt abgeschaltet. Bei aktivem Stereoscopic Mode halbiert sich die effektive Frame-Rate.

► **SKY_OFFS_X**► **SKY_OFFS_Y**

Horizontale und vertikale Verschiebung aller Sky-Texturen in Pixeln. **SKY_OFFS_X** darf nicht negativ werden. Durch periodisches Erhöhen von **SKY_OFFS_X** in einer Aktion kann ein 'ziehende Wolken'-Effekt erzielt werden.

►MOTION_BLUR

Parameter für die Bewegungsunschärfe (0..1). Bei der Bewegungsunschärfe reduziert sich automatisch die Bildschirm-Auflösung während der Player-Bewegung, was die Bewegung auf langsamen Rechnern 'flüssiger' macht. Der Vorgabewert ist 0 (keine Bewegungsunschärfe).

►BLUR_MODE

Wird dieser Skill (default 0) auf 1 gesetzt, so ist die Bewegungsunschärfe ständig eingeschaltet, auch wenn sich der Player nicht bewegt. Wird **BLUR_MODE** auf 0.5 gesetzt, so wird die Unschärfe bei Player-Bewegung und bei Bewegung der Maus eingeschaltet.

►RENDER_MODE

Aktivität der 3D-Darstellung; Vorgabe = 0.5. Folgende Abstufungen sind möglich:

- 2: Das 3D-Bild wird komplett neu berechnet; erforderlich nach direkter Versetzung der Player-Position.
- 1: Das 3D-Bild wird teilweise neu berechnet; erforderlich nach direkter Veränderung einer Objekt- oder Wandposition oder Regionshöhe.
- 0.5: Normalzustand. Das Bild wird je nach Playerbewegung automatisch neu berechnet.
- 0: Das 3D-Bild wird nicht generiert (für Titelbilder, Vor- und Nachspänne und Ganzbild-Menüs).

RENDER_MODE wird nach jeder Bildberechnung automatisch auf den Vorgabewert 0.5 zurückgesetzt.

►MOVE_MODE

Bewegungsverhalten von Player und Actors; Vorgabe=1. Mit diesem Skill können Aktionen und Bewegungen im Level 'eingefroren' werden. Folgende Abstufungen sind möglich:

- 1: Normalzustand
- 0.5: Die Bewegungen der Actors sowie die Objekt-**EACH_TICK**-Aktionen werden gestoppt.
- 0: Auch die Bewegung des Players sowie sämtliche Objekt- und Regions-Events werden nicht mehr ausgeführt.
- 0.5: Alle Aktionen, bis auf die Tastatur-Events, werden gestoppt.

►CLIPPING

Darstellung von Objekten außerhalb der **CLIP_DIST**. Folgende Abstufungen sind möglich:

- 0: Normalzustand: Walls sind nicht sichtbar, wenn einer ihrer Vertices ausserhalb der **CLIP_DIST** liegt. Das Flag **FAR** wird ignoriert.
- 0.5: Walls sind nicht sichtbar, wenn beide Vertices außerhalb der **CLIP_DIST** liegen und **FAR** nicht gesetzt ist. Bei großen Außenbereichen kann **FAR** bei den umgebenden **SKY**-Wänden gesetzt werden.
- 1: Walls sind nicht sichtbar, wenn beide Vertices außerhalb der **CLIP_DIST** liegen, oder irgendeine Wand ihrer Region nicht sichtbar ist. Wände mit Flag **FAR** sind in jedem Fall sichtbar. Dieser Modus ist sinnvoll, wenn das Level in viele kleine Regionen eingeteilt wurde.

- **LOAD_MODE** Wird dieser Skill (default: 0) auf 1 gesetzt, so wird bei der **LOAD**- und der **LEVEL**-Anweisung ein eventueller Wechsel der Bildschirmauflösung unterdrückt.
- **THING_DIST**
► **ACTOR_DIST** Verhältnis der **CLIP_DIST** von Things bzw. Actors zu Walls. **THING_DIST** muss zwischen 0 und 1 (default) liegen. **ACTOR_DIST** muss kleiner oder gleich **THING_DIST** sein.
- **MAP_OFFSX**
► **MAP_OFFSY** Horizontale und vertikale Abweichung des Zentrums der Landkarte vom Mittelpunkt des 3D-Fensters in Pixeln; default 0.
- **MAP_CENTERX**
► **MAP_CENTERY** Horizontale und vertikale Abweichung des Zentrums der Landkarte wie **MAP_OFFSX** und **MAP_OFFSY**, jedoch nicht in Pixeln, sondern in Steps. Dies hat den Vorteil, dass unabhängig vom Maßstab der Karte (**MAP_SCALE**) das Zentrum immer an der gleichen Stelle bleibt.
- #**MAP_MAXX**
#**MAP_MINX**
#**MAP_MAXY**
#**MAP_MINY** Größte und kleinste X- und Y-Koordinaten aller Objekte des Levels; werden bei Spielstart automatisch berechnet.
- **MAP_EDGE_X1**
► **MAP_EDGE_X2**
► **MAP_EDGE_Y1**
► **MAP_EDGE_Y2** Linke und rechte Begrenzung der Kartendarstellung auf dem Bildschirm, gemessen in Pixelabständen von der oberen linken Bildschirmkante. Die Vorgaben entsprechen der aktuellen Bildschirmgröße.
- **MAP_SCALE** Abbildungsmaßstab der Landkarte relativ zur Größe des 3D-Fensters; Vorgabe 0.9. Bei dem Wert 1 paßt die Landkarte exakt in das Fenster.

- **MAP_MODE** Darstellungstyp der Landkarte; Wertebereich 0..1, Vorgabe 0. Bei Werten > 0 wird eine Landkarte aller bereits erforschten und gesehenen Regionen und Objekte im 3D-Fenster angezeigt. Bei Werten >= 1 werden sämtliche Objekte - auch die noch nicht gesehenen - angezeigt.
- **MAP_ROT** Wird dieser Skill auf 1 gesetzt, so rotiert die Landkarte mit dem Player-Winkel um das Player-Symbol herum, ähnlich wie ein Radarbild. Allerdings werden nur die Objekte innerhalb der **CLIP_DIST** angezeigt.
- **COLOR_PLAYER** Farbnummer, in der das Player-Symbol auf der Landkarte gezeichnet wird (Default = 7).
- **COLOR_ACTORS** Default-Farbnummer für Actors auf der Landkarte (Default = 3).
- **COLOR_THINGS** Default-Farbnummer für Things auf der Landkarte (Default = 13).
- **COLOR_WALLS** Default-Farbnummer für Zwischenwände auf der Landkarte (Default = 244). Liegt die Farbe innerhalb eines Shading-Bereichs, werden die Wände 'antialiased' gezeichnet.
- **COLOR_BORDER** Default-Farbnummer für Randwände auf der Landkarte (Default = 17). Liegt die Farbe innerhalb eines Shading-Bereichs, werden die Wände 'antialiased' gezeichnet.
- **MAP_LAYER** Nummer der Overlay-Ebene (s. **LAYERS**), über der die Landkarte erscheint. Wertebereich 0..16, Vorgabe 0. Alle höheren **LAYERS** werden über die Landkarte darübergezeichnet.

- **MOUSE_MODE** Mausdarstellung. Bei Zuweisung von 0 (default) wird die Maus nicht am Bildschirm gezeigt, bei Zuweisung von 1 erscheint der Mauszeiger (**MSPRITE**, s.u.) am Bildschirm. Bei Zuweisung von 2 wirkt sich die Mausbewegung nicht mehr auf die **FORCE-Skills** aus, so daß die Maus unabhängig vom Player bewegt werden kann.
- #MOUSE_-MOVING** Maus in Bewegung (1) oder seit 1/4 Sekunde stillstehend (0).
- **MOUSE_CALM** Maximalstrecke, die noch als Stillstand für **MOUSE_MOVING** gilt (default 3).
- **MOUSE_TIME** Zeit in Ticks, über die die Mausstrecke **MOUSE_CALM** zur Ermittlung von **MOUSE_MOVING** gemessen wird; default 4.
- **TOUCH_MODE** Bei Zuweisung von 1 (default) werden die Textur-Strings (**TOUCH**) von berührten Objekten an der Position des Mauszeigers angezeigt; bei Zuweisung von 2 wird der Mauszeiger noch zusätzlich angezeigt.
- #TOUCH_STATE** Dieser Skill (default 0) nimmt den Wert 1 an, wenn sich der Mauszeiger auf einem Objekt mit **TOUCH**-Text befindet, den Wert 2, wenn der Objekt-Textur eine **IF_KLICK**-Aktion zugeordnet ist, und 3, wenn beides der Fall ist.
- #MICKEY_X**
#MICKEY_Y Relative Mausbewegung seit dem letzten Frame; kann benutzt werden, um die Skills **MOUSE_X** und **MOUSE_Y** zu verändern.

►**MOUSE_X**►**MOUSE_Y**

Horizontale und vertikale Position des Mauszeigers in Pixeln, bezogen auf die linke obere Bildschirmcke. Wenn der Mauszeiger aktiviert wurde (Skill **MOUSE_MODE**), kann er durch Verändern dieser beiden Skills am Bildschirm bewegt werden (**Beispiel**):

```
RULE MOUSE_X += MICKEY_X;
```

```
RULE MOUSE_Y += MICKEY_Y;
```

Die **MIN**- und **MAX**-Werte der Skills sollten so eingestellt werden, dass der Mauszeiger nicht über den Bildschirmrand hinaus geraten kann.

►**MOUSE_ANGLE**

Absoluter Richtungswinkel des Mauszeigers im Level, z.B. zum Fortwerfen eines Gegenstands am Mauszeiger (**DROP**); wird automatisch aus **MOUSE_X** und aus dem **PLAYER_ANGLE** berechnet. Kann zum Einstellen des **DROP**-Winkels auch direkt verändert werden.

►**TOUCH_DIST**

Maximale Entfernung eines Objekts zur Auslösung der Mausektionen **TOUCH**, **IF_TOUCH**, **IF_RELEASE**, **IF_KLICK**. Default = 100 steps.

#**JOYSTICK_X**#**JOYSTICK_Y**

Auslenkungen der Joystick-Achsen im Bereich von -255.. + 255.

#**STR_LEN**

Anzahl der Zeichen eines **STRING**, der per **INKEY** eingegeben wurde.

#**LINES**

Anzahl der Zeilen eines **TEXT**; wird durch die **FIND**-Anweisung gesetzt.

#**SIZE_Y**

Vertikale Größe eines **TEXT** in Pixeln; wird durch die **FIND**-Anweisung gesetzt.

- **SOUND_VOL** Generelle Lautstärke für die Geräusche, Wertebereich 0..1; wird mit den individuellen Geräuschlautstärken multipliziert.
- **MUSIC_VOL** Lautstärke für die Musik, Wertebereich 0..1. Mit dem Wert 0 wird die Musik abgeschaltet.
- **CDAUDIO_VOL** CD Audio Lautstärke im Bereich 0..1. Dies allerdings funktioniert nur unter DOS; unter Windows 95 wird die CD Audio Lautstärke mit dem zusammen mit der Soundkarte installierten Mixer-Tool eingestellt.
- **PSOUND_VOL** Relative Lautstärke des Player-Sounds (Sound der Bodentextur) im Bereich 0..2 (Vorgabe = 1). Die Lautstärke, mit der der Sound abgespielt wird, ergibt sich aus dem Produkt (**PSOUND_VOL** * *texture.SVOL*).
- **PSOUND_TONE** Tonhöhe des Player-Sounds (Sound der Bodentextur) im Bereich 0..4 (Vorgabe = 1).
- #CHANNEL** Nummer des Soundkanals (0..7), dem der letzte Sound z.B. durch eine **PLAY_SOUND**-Anweisung zugewiesen wurde. Wird der Sound aus irgendeinem Grund nicht gespielt (wenn z.B. alle Kanäle durch **SLOOP**-Sounds belegt sind), so enthält **CHANNEL** den Wert -1.
- #CHANNEL_0..**
#CHANNEL_7 Mit diesen Skills kann ständig der Zustand der 8 Soundkanäle abgefragt werden:
- 0 = Kein Sound, Kanal frei
 - 1 = Normaler Sound spielt
 - 2 = **SLOOP**-Sound spielt

- **AMBIENT** Globaler Lichtpegel (Bereich -1.. +1, default 0), der auf alle Textur- und Regions-**AMBIENTs** aufaddiert wird.
- **PLAYER_LIGHT** Stärke der Lichtquelle, die der Player mit sich führt (0..1, 16 Stufen, Vorgabe 1).
- **LIGHT_DIST** Entfernung des Beginns des Schattierungsbereiches von Player (Vorgabe 10 Steps). Objekte, die näher sind als **LIGHT_DIST**, werden vom **PLAYER_LIGHT** nicht mehr beeinflusst.
- # **DARK_DIST** Entfernung der Dunkelgrenze vom Player; wird bei jeder Veränderung des **PLAYER_LIGHT** vom Programm Neuberechnet.
- **PLAYER_WIDTH** Minimaler Abstand des Players zu Wänden und Objekten; relevant für die Kollisionserkennung. Vorgabe- und Mindestwert 1.2 Steps.
- **PLAYER_SIZE** Abstand zwischen Füßen und Augen des Players in Steps (Vorgabe = 3 Steps). Relevant für die Augenhöhe des Players sowie für die Kollisionserkennung.
- **PLAYER_CLIMB** Maximaler Höhenunterschied in Steps, der vom Player beim Wechseln der Region - z.B. Betreten einer Treppenstufe - überwunden werden kann. Relevant für die Kollisionserkennung. Vorgabe 1.5 Steps.
- **WALK_PERIOD** Anzahl der Steps für eine Periode einer rhythmischen streckenabhängigen Auf/Ab- bzw. Vor/Rückbewegung des Players, zur Darstellung von Geh- oder Schwimmbewegungen. Vorgabe = 4 Steps. Gleichzeitig ertönt in diesem räumlichen Abstand der Sound der Bodentextur als Schrittg Geräusch des Players.

- **WALK_TIME** Zeitkonstante für die **WALK**-Bewegung. Vorgabe = 4 Ticks.
- **WAVE_PERIOD** Anzahl der Ticks für eine Periode einer rhythmischen zeitabhängigen Bewegung des Players. Vorgabe = 16 Ticks.
- # **WALK** Augenblickliche Auslenkung der rhythmischen Player-Gehbewegung (-1.0 .. +1.0).
- # **WAVE** Augenblickliche Auslenkung der rhythmischen zeitabhängigen Player-Bewegung (-1.0 ... +1.0).
-
- **PLAYER_VX**
► **PLAYER_VY** Geschwindigkeit des Players in X- und in Y-Richtung des Koordinatennetzes in Steps pro Tick. Durch Verändern dieser Skills kann der Player horizontal bewegt werden; dabei erfolgt eine automatische Anpassung bei Kollisionen mit Wänden oder Objekten.
- **PLAYER_VZ** Geschwindigkeit des Players in Z-Richtung (vertikal).
- **PLAYER_VROT** Rotationsgeschwindigkeit des Player-Blickwinkels (**PLAYER_ANGLE**) in Radiant pro Tick. Durch Verändern dieses Skills wird der Player um die Hochachse gedreht.
- **PLAYER_TILT** Tangens des vertikalen Blickwinkels des Players (Drehung um die Querachse). Durch Verändern dieses Skills wird der Blick des Players gehoben oder gesenkt.

- **PLAYER_ARC** Brennweite des Player-Auges in Blickwinkel-Radiant (0.2..2.0). Kann während des Spiels für 'Fernglas'- oder 'Stoned'-Effekte verändert werden. Der Vorgabewert 1 entspricht etwa dem Blickwinkel des menschlichen Auges (ca. 55°).
- **FRICTION** Reibungskoeffizient für die **ACCEL**-Anweisung; Wertebereich 0..1, Vorgabe 0.5.
- **INERTIA** Trägheitskoeffizient für die **ACCEL**-Anweisung; Wertebereich > 0, Vorgabe 1.
- **SHOOT_RANGE** Maximale Objektentfernung für die **SHOOT**-Anweisung in Steps; Vorgabe = 500 Steps.
- **SHOOT_SECTOR** Winkelbereich (in Radiant) für die **SHOOT**-Anweisung von Objekt zu Player; default = 2ð. Muss vor jeder **SHOOT**-Anweisung auf den gewünschten Wert gesetzt werden. Befindet sich der Player außerhalb dieses Sektors (relativ zur Blickrichtung des Actors), liefert **SHOOT** immer eine **HIT_DIST** und ein **RESULT** von 0.
- **SHOOT_FAC** 'Trefferstärke'-Faktor für die **RESULT**-Berechnung bei der **SHOOT**- und **EXPLODE**-Anweisung; Wertebereich 0..1, Vorgabe 1.
- **SHOOT_X**
► **SHOOT_Y** Horizontale und vertikale Abweichung der **SHOOT**-Richtung vom Mittelpunkt des 3D-Fens- ters bzw. vom Mittelpunkt des Objekts im Wertebereich von 0..1.

#HIT_DIST	Entfernung zum letzten mit der SHOOT - oder EXPLODE -Anweisung getroffenen Objekt in Steps. Wenn kein Objekt getroffen wurde, wird dieser Skill auf 0 gesetzt.
#HIT_MINDIST	Entfernung des von EXPLODE am härtesten getroffenen Objekts zum Explosionszentrum.
#RESULT	Dieser Skill steht für die 'Stärke' eines Events, die Schwere eines Treffers oder das Ergebnis einer Aktion.
#SHOOT_ANGLE	Winkel vom SHOOT -auslösenden Actor zum Player; kann benutzt werden, um z.B. den Startwinkel eines Geschosses zu setzen.

#HIT_X**#HIT_Y**

Koordinaten des von **SHOOT** getroffenen Texturpixels einer Wall, eines Things oder Actors, relativ zur oberen linken Ecke der Textur. Auch während Textur-Mausaktionen (**IF_KLICK**, **IF_TOUCH**) sind diese Skills gültig. Bei Walls gibt **HIT_X** die horizontale Pixelkoordinate des **SHOOT**-Treffers oder des Mausklicks inklusive des Textur-Offsets (**OFFSET_X**) an. Kann benutzt werden, um Einschusslöcher in Wänden zu setzen, z.B. mit folgender Aktion:

```
BMAP    hole_map,<hole.pcx>;
TEXTURE hole_tex { BMAP hole_map; }
SYNONYM last_hitobj { TYPE WALL; }
ACTION set_hole {
    SHOOT;
    IF (HIT == NULL) { // nothing hit?
        END;
    }
    // something hit before?
    IF (last_hitobj != NULL) {
        SET last_hitobj.ATTACH,NULL;
    }
    // remove hole from last wall
    SET HIT.ATTACH,hole_tex;
    // and place it onto new
    SET last_hitobj,HIT;
    SET hole_tex.POS_X,HIT_X;
    SET hole_tex.POS_Y,HIT_Y;
}
```

►SKIP_FRAMES

Actors mit **TARGET**, die vom Player nicht gesehen werden und weiter als ihre **DIST** vom Player entfernt sind, bewegen sich normalerweise nur jeden 6. Frame (um eine entsprechend längere Strecke), um Rechenzeit zu sparen. Mit diesem Skill (default 5) wird die Anzahl der übersprungenen Frames vorgegeben.

- **ACTOR_CLIMB** Mit diesem Skill (default 1) kann die Maximalhöhe von Stufen angegeben werden, die Actors bei ihrer **TARGET**-Bewegung überwinden können (analog zu **PLAYER_CLIMB**, s.u.).
- **ACTOR_WIDTH**
► **THING_WIDTH** Mit diesen Skills (default = 1) kann ein Faktor angegeben werden, mit dem die Textur-Breite aller Actors oder Things für die Kollisionserkennung multipliziert wird.
- #ACTOR_-**
IMPACT_VX
#ACTOR_-
IMPACT_VY
#ACTOR_-
IMPACT_VZ Rückprall eines Actors an einem Hindernis in X- und Y-Richtung bzw. durch Kollision mit dem Boden oder der Decke seiner Region. Diese Skills enthalten nur bei Actors gesetztem **CAREFULLY**-Flag, nur innerhalb ihrer **EACH_TICK**-Aktion und nur unmittelbar nach einer Actor-Bewegung (gesetztes **MOVED**-Flag) gültige Werte.
- #ACTOR_-**
FLOOR_HGT
#ACTOR_-
CEIL_HGT Boden- und Deckenhöhe an der Actor-Position. Diese Skills enthalten nur bei Actors mit **TARGET**-Bewegung und **CAREFULLY**-Flag, nur innerhalb ihrer **EACH_TICK**-Aktion und nur unmittelbar nach einer Actor-Bewegung (gesetztes **MOVED**-Flag) gültige Werte.
- **PLAYER_X**
► **PLAYER_Y** Position des Players auf der X- und Y-Achse des Koordinatensystems. Bei direktem Verändern dieser Werte erfolgt keine Kollisionserkennung!
- **PLAYER_Z** Absolute Höhe der Player-Augenposition in Steps, bezogen auf die Nullhöhe des Level (s. **PLAYER_HGT**). Bei direktem Verändern dieses Wertes erfolgt keine Kollisionserkennung!

►PLAYER_- ANGLE	Horizontale Blickrichtung des Players in Radiant (0..6.28). Die Blickrichtung 0 entspricht der Richtung der positiven X-Achse des Koordinatensystems (Osten).
#PLAYER_SIN #PLAYER_COS	Sinus und Cosinus des horizontalen Player-Blickwinkels; erforderlich für die Rules, die die Komponenten PLAYER_VX und PLAYER_VY aus der Player-Blickrichtung und seiner Vorwärtsgeschwindigkeit berechnen.
#PLAYER_SPEED	Geschwindigkeitskomponente in Blickrichtung des Players; wird aus PLAYER_VX , PLAYER_VY und dem Blickwinkel berechnet. Bei ausschließlicher Seitwärtsbewegung des Players ist der Wert dieses Skills 0; bewegt sich der Player rückwärts, wird PLAYER_SPEED negativ.
#ACCELERATION	Beschleunigung in Blickrichtung des Players; wird aus der Veränderung von PLAYER_SPEED berechnet. Wird der Player rückwärts beschleunigt, so wird ACCELERATION negativ.
#PLAYER_HGT	Relative Höhe der 'Füße' des Players über oder unter der Bodenhöhe der Region; wichtig für die Korrektur der Vertikalbewegung. Der Wert ergibt sich aus $(\text{PLAYER_Z} - \text{PLAYER_SIZE} - \text{FLOOR_HGT})$.
#FLOOR_HGT #CEIL_HGT	Reale Boden- und Deckenhöhe in Steps an der Player-Position; Z-Werte der Vertices und Steigung des Bodens sind berücksichtigt.
#PLAYER_- DEPTH	Tiefe einer Unterwasser- BELOW -Region unter dem Player, deren Deckenhöhe der Bodenhöhe der Player-Region entspricht.

- ▶ **PLAYER_LAST_X,**
▶ **PLAYER_LAST_Y** Diese Skills enthalten die letzte Player-Position im vorigen Framezyklus und werden zur internen Berechnung der Skills **STEPS, WALK** usw. benutzt. Soll sich der Player ohne **WALK**-Veränderung bewegen - z.B. wenn er sich mit einem Fahrzeug mitbewegt - so können diese Werte per Aktion verändert werden.

- ▶ **IMPACT_VX**
▶ **IMPACT_VY** Kollisionsvektor; Geschwindigkeitsveränderung durch Wandkollision des Players oder eines **STICK**-Actors in X- und Y-Richtung. Diese Werte werden bei jeder Kollision automatisch erhöht und müssen bei der Auswertung manuell wieder auf 0 gesetzt werden. Sie können zur Generierung eines verstärkten Abprallens (Flipper-Effekt) verwendet werden.

- ▶ **IMPACT_VZ** Kollisionsvektor, Geschwindigkeitsveränderung in vertikaler Richtung, z.B. durch Kollision mit Boden oder Decke.

- ▶ **IMPACT_VROT** Prallt ein Actor mit Target **STICK** gegen ein Hindernis, so gibt dieser Skill seine durch den Aufprall bewirkte Winkeländerung relativ zum Player an. **IMPACT_VROT** kann benutzt werden, um dem Player eine Drehbewegung durch Abprallen zu verleihen.

- #BOUNCE_VX**
#BOUNCE_VY Diese Skills enthalten bei Kollisionen des Players mit einer Wand den Geschwindigkeitsvektor, projiziert auf die Wand-Normale. Addiert man sie auf die Player-Geschwindigkeit, ergibt sich ein realistisches Abprallverhalten.

#SLOPE_AHEAD
#SLOPE_SIDE

Steigung des Region-Bodens in Blickrichtung und in Seitwärtsrichtung des Players (Höhenzunahme pro Step). Kann z.B. für die Bewegungs-Aktion des Players verwendet werden, um den Blick zu heben oder den Player über eine Schräge hinabrutschen zu lassen.

#SLOPE_X
#SLOPE_Y

Steigung des Bodens der Player-Region in X- und Y-Richtung.

#MOVE_ANGLE
#DELTA_ANGLE

Der Skill **MOVE_ANGLE** enthält die reale Bewegungsrichtung des Players, der Skill **DELTA_ANGLE** die Winkeldifferenz zwischen **MOVE_ANGLE** und **PLAYER_ANGLE** im Bereich von -PI.. + PI.

#NODE

Nummer des aktuellen Node bei Multiplayerspielen; ergibt sich aus der Befehlszeile (nur Commercial- und Profi-Version).

►REMOTE_0
►REMOTE_1

Skills zum Übertragen von Informationen und Ereignissen an den verbundenen PC bei Multiplayer-Spielen (nur Commercial- und Profi-Version)

#TIME_CORR

Zeitkorrekturfaktor. Bei einer Framerate von 16 Bilder/sec hat dieser Skill den Wert 1.0; bei höheren Frameraten sinkt sein Wert, bei niedrigeren steigt er proportional an. Kann zur automatischen Anpassung von Geschwindigkeiten an unterschiedliche Frameraten verwendet werden.

#TIME_FAC

Zeitkorrekturfaktor, Kehrwert von **TIME_CORR**. Dieser Skill enthält die Anzahl der Framezyklen pro Tick; er ist umso höher, je schneller der Rechner ist.

- **TICKS** Der Wert dieses Skills erhöht sich jede sechzehntel Sekunde um 1.
- **SECS** Der Wert dieses Skills erhöht sich jede Sekunde um 1.
- **STEPS** Der Wert dieses Skills erhöht sich mit dem vom Player zurückgelegten Weg um die entsprechende Anzahl von Steps.
- #FLIC_FRAME** Frame-Nummer des gerade ablaufenden Flic. Kann benutzt werden, um in einer globalen **EACH_TICK**-Aktion Sprache, Soundeffekte oder Musik frame-genau zu starten, mit Anweisungen wie:
- ```
IF (FLIC_FRAME==50) {
 PLAY_SOUND bang,0.7;
}
```
- Solange kein Flic läuft, hat dieser Skill den Wert 0.
- #CD\_TRACK** Nummer des gerade spielenden Track einer Audio-CD; wird durch die Anweisung **PLAY\_CD** aktualisiert.
- #MOUSE\_LEFT**  
**#MOUSE\_MIDDLE**  
**#MOUSE\_RIGHT**  
**#JOY\_4** Zustand der betreffenden Maus- bzw. Joysticktaste; 0 = nicht gedrückt, 1 = gedrückt. Die drei Maustasten entsprechen der ersten, dritten und zweiten Joystick-Taste.
- #KEY\_ANY** Dieser Skill hat den Wert 1, wenn irgendeine Taste gedrückt ist, und sonst 0.

#KEY\_F1...  
#KEY\_F12,  
#KEY\_ESC,  
#KEY\_TAB,  
#KEY\_SHIFT,  
#KEY\_CTRL,  
#KEY\_ALT,  
#KEY\_SPACE,  
#KEY\_BKSP,  
#KEY\_CUU,  
#KEY\_CUD,  
#KEY\_CUR,  
#KEY\_CUL,  
#KEY\_PGUP,  
#KEY\_PGDN,  
#KEY\_HOME,  
#KEY\_END,  
#KEY\_INS,  
#KEY\_DEL,  
#KEY\_PAUSE,  
#KEY\_CAR,  
#KEY\_CAL,  
#KEY\_PLUS,  
#KEY\_MINUS,  
#KEY\_ENTER,  
#KEY\_0...KEY\_9,  
#KEY\_A...KEY\_Z

Zustand der betreffenden Taste; 0 = nicht gedrückt,  
1 = gedrückt.

#FORCE\_-  
AHEAD

Analogwert der Eingabe via Maus, Joystick oder Tastatur (Cursor-auf/ab-Tasten) für die Vorwärts-/Rückwärtsbewegung; Wertebereich ca. -1 .. +1. Der Wertebereich kann durch die Schlüsselworte **KEY\_SENSE**, **JOY\_SENSE**, **MOUSE\_SENSE** (s.u.) abhängig von jeweiligen Eingabegerät beeinflusst werden.

|                       |                                                                                                                                                                                                                                                                 |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>#FORCE_-STRAFE</b> | Eingabewert für lineare Links-/Rechtsbewegung, eingegeben mit den [,]- und [.] -Tasten oder durch Drücken von [Alt] und gleichzeitiges Seitwärtsbewegen von Maus oder Joystick.                                                                                 |
| <b>#FORCE_ROT</b>     | Eingabewert für die horizontale Rotation des Blickwinkels, eingegeben mit den Cursor-links/rechts-Tasten oder durch Seitwärtsbewegen von Maus oder Joystick.                                                                                                    |
| <b>#FORCE_TILT</b>    | Eingabewert für das Heben und Senken des Blickwinkels mit den Bild-auf/ab-Tasten.                                                                                                                                                                               |
| <b>#FORCE_UP</b>      | Eingabewert für die Vertikalbewegung (z.B. Drücken/Springen) mit den Tasten [Pos1] und [Ende].                                                                                                                                                                  |
| <b>►KEY_SENSE</b>     | Empfindlichkeit der Bewegung des Players durch Tastenbetätigung mit den Cursortasten, [,], [.] , [Bild auf]/[Bild ab], [Pos1] und [Ende]. Der Skill gibt den Betrag des Eingabewertes (z.B. <b>FORCE_AHEAD</b> , s.u.) bei Tastenbetätigung an (Vorgabe = 0.7). |
| <b>►SHIFT_SENSE</b>   | Faktor, um den alle Eingabewerte bei Betätigung der [Shift]-Taste erhöht werden (Vorgabe = 2).                                                                                                                                                                  |
| <b>►MOUSE_SENSE</b>   | Empfindlichkeit der Bewegung des Players mit der Maus. Der Wert 1 entspricht mittlerer Empfindlichkeit (Vorgabe = 1).                                                                                                                                           |
| <b>►JOY_SENSE</b>     | Empfindlichkeit der Bewegung des Players mit dem Joystick. Der Wert 1 entspricht mittlerer Empfindlichkeit (Vorgabe = 1).                                                                                                                                       |

Die folgenden Skills stehen zu Testzwecken im Editor WED.EXE zur Verfügung, nicht jedoch im Runtime-Modul VRUN.EXE:

**#ERROR**

Fehlercode für das aktuelle Bild; kann z.B. in einem Panel angezeigt werden. Folgende Fehlercodes sind möglich:

- 0 = kein Fehler,
- 1 = zu kleine **CLIP\_DIST**,
- 3 = zu kleiner **NEXUS**  
(zu viele Wände sichtbar),
- 5 = fehlende Bitmap,
- 6 = fehlende Textur,
- 7 = fehlerhafter Sound,
- 8 = falscher Synonym-Gebrauch,
- 9 = fehlende Region.

Der letzte Fehlercode bleibt so lange gespeichert, bis ein neuer Fehler auftritt. Das Auftreten eines Fehlers wird zusätzlich von einem Geräusch des PC-Lautsprechers signalisiert.

**►DEBUG\_MODE**

Wird dieser Skill auf 1 gesetzt, so hält das Programm nach jedem Bildaufbau an, bis die **[S]**-Taste gedrückt wird (Single Step Mode). Kann auch durch **[Ctrl][Alt][S]** oder mit der Kommandozeilenoption **-SST** aktiviert werden.

**#MAX\_DIST**

Entfernung des fernsten sichtbaren Vertex innerhalb der **CLIP\_DIST**; zur einfacheren Optimierung der Regionen-**CLIP\_DIST**.

**#ACTIONS**

Anzahl der gerade laufenden Aktionen.

**#ACTIVE\_NEXUS**

Auslastung des Nexus.

**#ACTIVE\_  
TARGETS**

Anzahl der Actors mit **TARGET**.

### 13. Levelwechsel, Spielstände, Multiplayer-Modus

Der Wechsel eines Levels mit der Anweisung **LEVEL** entspricht dem Neustart eines Spiels. Sämtliche Skills und Parameter werden aus der neuen WDL-Datei geladen, mit Ausnahme der **LOCAL**- und **GLOBAL**-Skills. Der neue und der alte Level müssen hierzu exakt die gleichen **LOCAL**- und **GLOBAL**-Skills in der gleichen Reihenfolge enthalten; die 'normalen' Skills dürfen sich unterscheiden. Über die **GLOBAL**-Skills lassen sich Parameter zwischenspeichern und in den neuen Level 'mitnehmen', z.B. Kampfkraft und Ausrüstung des Players.

Bei Wechsel der Topographie mit der Anweisung **MAP** bleiben die **LOCAL**-, **GLOBAL**- und Player-Skills erhalten, mit Ausnahme der Player-Position. Walls, Regions, Things und Actors werden aus der WMP-Datei geladen.

Mit den Anweisungen **SAVE** und **LOAD** lassen sich Spielstände abspeichern und wieder laden. Folgende Parameter des Spiels werden gespeichert:

- # Der Name der aktuellen .WDL- und .WMP-Datei,
- # alle Player- und **GLOBAL**-Skills (**LOCAL**-Skills werden nicht gespeichert),
- # alle Synonyme,
- # die **EACH\_TICK**-Aktionsliste
- # alle Key-Aktionen (**IF\_F1** etc.),
- # der aktuelle Stand aller gerade laufenden Aktionen,
- # die Positionen und sonstigen veränderbaren Parameter jedes **VIEW**, **PANEL**, **TEXT** oder **OVERLAY**,
- # die Positionen, Skills und sonstigen Parameter sämtlicher Actors,
- # die Flags aller Regionen, Walls und Things,
- # die Positionen, Skills und sonstigen Parameter jeder Region, jeder Wall, jedes Things und jeder Textur, deren Flag **SAVE** gesetzt ist.

Damit die **ATTACH**-Textur eines Objekts gespeichert wird, muß das **SAVE**-Flag dieser Textur gesetzt sein.

Läuft zum Zeitpunkt des Ladens ein anderer Level, so wird automatisch der Level gewechselt. Das Laden selbst wird - ebenso wie **LEVEL** und **MAP** - nicht unmittelbar in der Aktion, in der die entsprechende Anweisung steht, sondern erst

nach Ende der Aktion durchgeführt. Anschließend wird die **IF\_LOAD**-Aktion ausgeführt.

Mit den Anweisungen **SAVE\_INFO** und **LOAD\_INFO**, die ausschließlich auf die **GLOBAL**- und **LOCAL**-Skills wirken, lassen sich allgemeine Spielparameter wie z.B. Soundlautstärke oder Bildschirmauflösung unabhängig von den Spielständen speichern. Auch per **FREEZE** geänderte Bitmaps werden mit diesen Befehlen gespeichert. Im Tutorial wird auf die Realisierung von Aktionen zum Levelwechsel und zum Speichern von Spielständen näher eingegangen.

Für das Spiel zweier Player gegeneinander (*Deathmatch*-Mode) können zwei PCs entweder über ein Nullmodem-Kabel - z.B. ein seriellcs Laplink-, Norton Commander- oder Interlink-Übertragungskabel - verbunden werden, oder beide müssen an einem lokalen Netzwerk (LAN) mit IPX-Protokoll hängen. Auf beiden Computern muß der gleiche Level laufen.

Zur Einrichtung der Übertragung muß beim Programmstart mit Kommandozeilen-Optionen die Nummer des seriellen Ports für die Kommunikation (Option **-COM n**) und die 'Netzwerknummer' (**-NODE 0** oder **-NODE 1**) angegeben werden. Wird der **-COM**-Parameter weggelassen, so wird eine IPX-Verbindung angenommen. Der beim Start angegebene **NODE** kann in Aktionen über den Skill **NODE** abgefragt werden.

Nach dem Start ueberprüft die Engine, ob bereits eine ACKNEX-Session auf einem seriell verbundenen PC oder auf dem Netzwerk läuft. Wenn ja, meldet sie sich bei der betreffenden Session an. Andernfalls wird eine neue Session eröffnet.

Zur Repräsentation des 'gegnerischen' Players im Level dient ein Actor mit dem **TARGET NODE0** oder **NODE1**. Ein solcher Actor führt die Bewegungen des Player am angeschlossenen PC mit der entsprechenden Netzwerknummer **NODE0** oder **NODE1** aus.

Zwischen des verbundenen PCs wird die jeweilige Player-Position und zusätzlich der vordefinierte Skill **REMOTE\_0** oder **REMOTE\_1** übertragen, wobei der PC mit **NODE0** den Wert von **REMOTE\_0** zum anderen sendet und der mit **NODE1** den Wert von **REMOTE\_1**. Mit Hilfe dieser Remote-Skills können z.B. Treffer ausgewertet werden. Die Positionen von Actors oder die Veränderungen von Objekten und Regionen werden in diesem einfachen seriellen Multiplayer-Modus nicht übertragen. Jedoch können einfache Aktionen (Feuern, oder Bewegung von Liften oder Türen) durch Auswertung von **REMOTE**-Skills dem jeweils anderen PC übermittelt werden.

## 14. Bedieneroberfläche: Views, Panels, Texte, Overlays

Bei Spielstart nimmt das 3D-Fenster den ganzen Bildschirm ein; ein Menü ist zunächst nicht sichtbar. Die Bedieneroberfläche läßt sich jedoch mit Hilfe von weiteren 3D-Fenstern (Views), Overlays und Bedienungspanels frei definieren.

Grafische Elemente innerhalb des 3D-Fensters wie z.B. Cockpits, Werkzeuge oder Waffen werden als **Overlay** definiert. Overlays werden wesentlich schneller dargestellt als Panels; sie besitzen aber keine Knöpfe oder Anzeigen und verbrauchen etwa das Dreifache an Speicherplatz. Es können beliebig viele Overlays übereinander dargestellt werden.

### **OVERLAY** *Keyword { ... }*

Dies weist dem Schlüsselwort eine Overlay-Definition zu. Diese kann - in der angegebenen Reihenfolge - wiederum die folgenden Schlüsselwort-Zuweisungen enthalten:

#### **LAYER** *Zahl;*

Bestimmt die Reihenfolge des Overlay auf dem Bildschirm, wenn es sich mit anderen Bildschirm-Elementen überlappt. Elemente mit größerem **LAYER**-Wert werden über Elementen mit kleinerem **LAYER** dargestellt. **LAYER** kann während des Spiels nicht verändert werden.

#### ►**POS\_X** *x;*

#### ►**POS\_Y** *y;*

Abstand der oberen linken Ecke des Overlay von der oberen linken Ecke des 3D-Fensters bzw. - wenn das Flag **ABSPOS** gesetzt ist (s.u.) - von der oberen linken Bildschirmecke in Pixeln. Durch Verändern dieser Werte kann das Overlay während des Spiels über den Bildschirm verschoben werden.

#### **OVLYS** *Ovly;*

Die eigentliche Overlay-Bitmap, der vorher mit dem Schlüsselwort **OVLY** eine Bitmap-Datei zugewiesen sein muß.

**#SIZE\_X**

**#SIZE\_Y**

Größe der Overlay-Bitmap in Pixeln; läßt sich in Aktionen auswerten.

**FLAGS**      ***Keyword1, Keyword2...;***

Hier kann eine Liste von Schlüsselworten (Flags) angegeben werden, die die Eigenschaften des Overlays bestimmen. Folgende Flags lassen sich 'setzen':

**ABSPOS;**

Wenn dieses Flag gesetzt ist, bezieht sich die Overlay-Position (**POS\_X**, **POS\_Y**) nicht auf die obere linke Ecke des 3D-Fensters, sondern auf die obere linke Ecke des Bildschirms.

► **VISIBLE**

Nur wenn dieses Flag gesetzt ist, erscheint das Overlay auf dem Bildschirm.

---

Ein vordefiniertes Synonym für ein Overlay ist der Mauszeiger:

► **MSPRITE** ***Overlay;***

Dieses Overlay erscheint als Mauszeiger auf dem Bildschirm. Der Mauszeiger wird mit den vordefinierten Skills **MOUSE\_X** und **MOUSE\_Y** bewegt. Die Overlay-Parameter **POS\_X** und **POS\_Y** geben den 'Hot Spot' des Mauszeigers an.

---

Zur Darstellung von Bedienelementen, Anzeigen und Bildern dienen **Panels**. Sie sollten möglichst außerhalb des 3D-Fensters liegen, da die Anzeigeelemente sonst bei jedem Bildaufbau neu generiert werden müssen, was Rechenzeit kostet.

**PANEL** ***Keyword { ... }***

Dies definiert ein Panel mit dem Namen *Keyword*. Eine Panel-Definition kann - in der angegebenen Reihenfolge - die folgenden Zuweisungen enthalten:



**PAN\_MAP**     *Bmap;*

Bitmap-Schlüsselwort für den Panel-Hintergrund. Die Größe dieser Bitmap bestimmt zugleich die Größe des Panel. Bei jeder Änderung der Panel-Position (**POS\_X**, **POS\_Y**) wird die Hintergrund-Bitmap neu gezeichnet.

**LAYER**     *Zahl;*

Bestimmt die Reihenfolge des Panels auf dem Bildschirm, wenn es sich mit anderen Bildschirm-Elementen überlappt. Elemente mit größerem **LAYER**-Wert werden über Elementen mit kleinerem **LAYER** dargestellt. **LAYER** kann während des Spiels nicht verändert werden.

► **POS\_X**     *x;*► **POS\_Y**     *y;*

Abstand der oberen linken Kante des Panel von der oberen linken Kante des Bildschirms. Werden diese Werte durch eine **SET**-Anweisung während des Spiels geändert, so wird das komplette Panel neu gezeichnet.

**FLAGS**     *Keyword1,Keyword2...;*

Hier kann eine Liste von Schlüsselworten (Flags) angegeben werden, die die Eigenschaften des Panels bestimmen. Folgende Flags lassen sich 'setzen':

**TRANSPARENT**

Wird dieses Flag gesetzt, so wird der Panel-Hintergrund sowie die Bitmaps von **PICTURE**- und **WINDOW**-Anzeigeelementen (s.u.) transparent gezeichnet, d.h. die Farbe 0 wird unterdrückt.

**REFRESH**

Wird dieses Flag gesetzt, so werden Panel und Anzeigen bei jedem Bildaufbau regeneriert. Dies ist erforderlich, um ein Panel über dem 3D-Fenster darzustellen oder z.b. eine Schrift über das Panel rollen zu lassen, ohne dabei den Hintergrund zu überschreiben. Ohne **REFRESH** wird ein Anzeigeelement nur bei einer Änderung des betreffenden Skills neu aufgebaut. Besonders bei großen Panels kostet der ständige Bildaufbau allerdings erhebliche Rechenzeit; hier sollte für den Panel-Hintergrund besser ein **OVERLAY** eingesetzt werden.

## RELPOS

Wenn dieses Flag gesetzt ist, bezieht sich die Panel-Position (**POS\_X**, **POS\_Y**) nicht auf die obere linke Ecke des Bildschirms, sondern auf die obere linke Ecke des 3D-Fensters. Panels mit Flag **RELPOS** werden an den Kanten des 3D-Fensters 'abgeschnitten'.

Die folgenden Anzeige-Elemente können im Panel dargestellt werden, wobei sich die *x,y*-Positionen immer auf den Abstand der oberen linken Ecke des jeweiligen Elements zur oberen linken Ecke des Panels beziehen:

### **BUTTON** *x,y,BmapOn,BmapOff,ActionOn,ActionOff;*

Taste im Panel. Die Abmessungen entsprechen der Bitmap mit dem per *BmapOn* angegebenen Schlüsselwort. Normalerweise erscheint die Bitmap *BmapOff* auf dem Panel; beim Anklicken (linke Maustaste) wechselt die Bitmap zu *BmapOn*, und die Aktion *ActionOn* wird ausgelöst. Loslassen der Maustaste wechselt die Bitmap wieder zu *BmapOff* und löst *ActionOff* aus. Soll der Button im unbetätigten Zustand nicht sichtbar sein, kann man der *BmapOff* auch das **NULL**-Schlüsselwort zuweisen.

### **VSLIDER** *x,y,len,Bmap,Skill;*

### **HSLIDER** *x,y,len,Bmap,Skill;*

Vertikaler und horizontaler Schieberegler zur Eingabe von Skills. Die Zahl *len* entspricht der Höhe bzw. Breite des Schiebereglerwegs in Pixeln. Die Bitmap *Bmap* wird für den Schiebereglerknopf verwendet, der mit der linken Maustaste 'gezogen' werden kann. Die Grenzen (**MIN**, **MAX**) des Skills bestimmen den Wertebereich des Reglers: Am Startanschlag hat der Skill den Wert **MIN**, am Endanschlag den Wert **MAX**.

### **VBAR** *x,y,len,Bmap,fac,Skill;*

### **HBAR** *x,y,len,Bmap,fac,Skill;*

Vertikale bzw. horizontale Balkenanzeige zur grafischen Darstellung eines Skills im Panel, z.B. für eine Kompaß-Skala. Die Bitmap wird abhängig vom Skillwert vertikal verschoben dargestellt. *len* ist die vertikale Größe der Anzeige in Pixeln, *fac* eine Festkommazahl, die multipliziert mit dem Skill-Wert die vertikale Verschiebung in Pixeln ergibt. Die angezeigte Bitmap muß mindestens ( $len + fac * (\text{Skill-Maximalwert})$ ) Pixel hoch bzw. breit sein.

**DIGITS** *x,y,len,Font,fac,Skill;*

Zahlenanzeige. *Font* ist ein Zeichensatz entweder aus 11 Zeichen (Zahlen 0..9 und Leerzeichen) oder aus 128 Zeichen in ASCII-Reihenfolge. Der ganzzahlige Anteil des Skill wird mit *len* Stellen numerisch im Panel dargestellt. Der Skill-Wert (hier kann auch ein Objekt-Parameter o.ä. angegeben werden) wird vorher mit dem Faktor *fac* multipliziert. Führende Nullen werden unterdrückt. Ist im *Font* kein Minuszeichen enthalten, so werden negative Werte nicht dargestellt. Mit einer einstelligen Anzeige können natürlich statt Zahlen auch Bilder oder Symbole angezeigt werden.

**Beispiel:**

```
DIGITS 115,15,6,TestFont,1000,HERE.FLOOR_HGT;
```

**PICTURE** *x,y,Texture,Skill;*

Animierte Bildanzeige. Die *Texture* kann bis zu 32 **SIDES** enthalten, die mit ihren **CYCLES** animiert werden und je nach dem ganzzahligen Wert des Skills gewechselt werden (1 = erste Seite, 2 = 2. Seite usw.). Die Bitmaps der Textur werden unskaliert angezeigt.

**WINDOW** *x,y,dx,dy,Bmap,Skillx,Skill;*

Anzeige eines Ausschnitt-'Fensters' aus einer Bitmap. Die Zahlen *dx* und *dy* bestimmen die Größe des Ausschnitts in Bildpunkten. Die Quell-Bitmap *Bmap* darf nicht kleiner als *dx* und *dy* sein. Die beiden Skills *Skillx* und *Skill* geben die Position des Ausschnitt-Fensters über der Bitmap in Pixeln an, bezogen auf die obere linke Ecke. Das Fenster kann sich nur innerhalb der Bitmap-Grenzen bewegen.

**MSPRITE** *Overlay;*

Dieses Overlay erscheint anstelle des 'allgemeinen' **MSPRITE** als Mauszeiger innerhalb des Panel.

**►IF\_KLICK Aktion;**

Diese Aktion wird anstelle der 'allgemeinen' **IF\_KLICK**-Aktion ausgeführt, wenn irgendwo auf dem Panel-Background mit dem Mauszeiger links geklickt wird.

**Texte** dienen der Ausgabe von Meldungen oder dem Dialog mit Actors. Sie sind ähnlich hierarchisch aufgebaut wie Bitmaps und Texturen: Der 'rohe' Text wird mit dem **STRING**-Schlüsselwort definiert, der formatierte Text, der mehrere solcher Strings enthalten kann, mit dem **TEXT**-Schlüsselwort.

### **STRING** *Keyword*, "*Text*";

Definition einer Buchstabenfolge mit dem Namen *Keyword* und dem Inhalt *Text*. Dieser Text steht zwischen Anführungszeichen; Zeilenumbrüche werden entweder durch die Zeichenfolge "\n" oder durch Zeilenumbrüche in der WDL-Datei angegeben.

### **TEXT** *Keyword* { ... }

Dies definiert einen formatierten Text mit dem Namen *Keyword*. Eine Text-Definition kann - in der angegebenen Reihenfolge - die folgenden Schlüsselworte enthalten:

### **LAYER** *Zahl*;

Bestimmt die Reihenfolge des Textes auf dem Bildschirm, wenn er sich mit anderen Bildschirm-Elementen überlappt. Elemente mit größerem **LAYER**-Wert werden über Elementen mit kleinerem **LAYER** dargestellt. **LAYER** kann während des Spiels nicht verändert werden.

### ► **POS\_X** *x*;

### ► **POS\_Y** *y*;

Abstand der oberen linken Kante des Textes von der oberen linken Ecke des Bildschirms. Die Position darf außerhalb des Bildschirms liegen; es wird dann nur der sichtbare Teil des Textes dargestellt. Die Textposition kann während des Spiels verändert werden. Auf diese Weise lassen sich Texte horizontal oder vertikal über den Bildschirm verschieben.

**►SIZE\_Y** *n*;

Höhe des angezeigten Textes in Pixeln (Default- und Maximalwert = Bildschirmhöhe). Der Text wird nur im vertikalen Bereich von **POS\_Y** und **POS\_Y + SIZE\_Y** dargestellt. Buchstaben, die ganz oder teilweise ausserhalb dieses Bereiches liegen, werden unterdrückt.

**►OFFSET\_Y** *n*;

Nummer der ersten Pixelzeile des Textes, die an der **POS\_Y**-Position dargestellt wird. Mit diesem Parameter lässt sich der Text vertikal pixelweise scrollen.

**STRINGS** *n*;

Maximalzahl an Strings, die dieser **TEXT** enthalten kann. Die Strings werden ohne Zwischenräume hintereinander dargestellt.

**FONT** *Font*;

Der vorher definierte Zeichensatz für den Text oder für Symbole; 128 oder 256 Buchstaben in ASCII-Reihenfolge.

**#SCALE\_X****#SCALE\_Y**

Größe eines einzelnen Buchstaben des **FONT** in Pixeln; kann in Aktionen ausgewertet werden.

**►STRING** *Keyword1, Keyword2...*;

Der eigentliche Text, bestehend aus einem oder mehreren Strings (Maximalanzahl vorhier in **STRINGS** definieren!). *Keyword* ist das Schlüsselwort für einen vorher definierten String. Es kann während des Spielablaufs in Aktionen verändert werden. Wird als String-Schlüsselwort **NULL** zugewiesen, so wird kein Text dargestellt.

**►INDEX** *n*;

Nummer des Strings, der mit dem **STRING**-Schlüsselwort per Aktion verändert wird; Wertebereich 1..**STRINGS**, default 1.

**FLAGS**     *Keyword1,Keyword2...;*

Hier kann eine Liste von Schlüsselworten (Flags) angegeben werden, die die Eigenschaften des Textes bestimmen. Folgende Flags lassen sich 'setzen':

**CENTER\_X****CENTER\_Y**

Bei Setzen dieser Flags wird der Text horizontal um **POS\_X** bzw. vertikal um **POS\_Y** zentriert. Ist das entsprechende Flag nicht gesetzt, so wird der Text bündig nach links bzw. oben ausgegeben.

**CONDENSED**

Bei Setzen dieses Flags wird der Text horizontal um 1 Pixel pro Buchstabe gestaucht. Besonders bei schrägen **FONTs** sehen gestauchte Texte besser aus.

**NARROW**

Wie **CONDENSED**, jedoch wird der Text stärker gestaucht.

**►VISIBLE**

Nur wenn dieses Flag gesetzt wird, erscheint der Text auf dem Bildschirm.

---

Zur Darstellung von Rückspiegeln, Raketen-Kameras und ähnlichem, aber auch für Multiplayer-Spiele können mehrere 3D-Fenster neben- oder untereinander auf dem Bildschirm dargestellt werden (nur mit der Commercial- oder Profi-Version). Jedes Fenster - außer dem ersten - wird durch einen **VIEW** definiert:

**VIEW** *Keyword { ... }*

Dies definiert ein weiteres 3D-Fenster mit dem Namen *Keyword*. Eine View-Definition kann - in der angegebenen Reihenfolge - die folgenden Schlüsselworte enthalten:

**LAYER**     *Zahl;*

Bestimmt die Reihenfolge des View auf dem Bildschirm, wenn es sich mit anderen Views überlappt. Views mit größerem **LAYER**-Wert werden über Views mit kleinerem **LAYER** dargestellt. Zu beachten ist, dass Panels, Overlays und Texte unabhängig von ihrem **LAYER** immer über den 3D-Fenstern dargestellt werden.

► **POS\_X**     *x;*► **POS\_Y**     *y;*

Abstand der oberen linken Ecke des View von der oberen linken Ecke des Bildschirms in Pixeln. Durch Verändern dieser Werte kann der View während des Spiels über den Bildschirm verschoben werden.

► **SIZE\_X**     *dx;*► **SIZE\_Y**     *dy;*

Größe des View in Pixeln. Für Position und Größe des Fensters gelten die gleichen Einschränkungen wie für **SCREEN\_X**, **SCREEN\_Y**, **SCREEN\_WIDTH**, **SCREEN\_HGT**.

► **GENIUS**     *Keyword;*

Das Schlüsselwort eines Things oder Actors. Der Bildinhalt des View entspricht einem Blick durch dessen Augen. Wenn sich der betreffende Actor bewegt, sollte er das **FAR**-Flag gesetzt haben, damit die Bewegung auch in weiter Entfernung von der Player-Position kontinuierlich verläuft.

**FLAGS**     *Keyword...;*

Hier kann eine Liste von Schlüsselworten (Flags) angegeben werden, die die Eigenschaften des View bestimmen. Folgende Flags lassen sich setzen:

► **VISIBLE**

Nur wenn dieses Flag gesetzt wird, erscheint das 3D-Fenster auf dem Bildschirm.

**Beispiel** für eine horizontale Unterteilung eines SVGA-Bildschirms in zwei 3D-Fenster:

```
VIDEO S640x480;
ASPECT 0.5; // to get the right proportions
SKILL SCREEN_X { VAL 0; } // 1st View, upper half of screen
SKILL SCREEN_Y { VAL 0; }
SKILL SCREEN_WIDTH { VAL 640; } // SVGA mode only!
SKILL SCREEN_HGT { VAL 240; }

VIEW my_view {
 POS_X 0; // 2nd window, lower half
 POS_Y 240;
 SIZE_X 640;
 SIZE_Y 240;
 GENIUS my_actor; // You'll look through his eyes
 FLAGS VISIBLE;
}
```



## Publikation eines ACKNEX-Games

Ihr Spiel oder Ihre 3D-Applikation ist nun fertig und funktioniert wie gewünscht, wenn Sie mit WED -RUN starten. Wie wird daraus jetzt ein vertriebsfertiges Produkt auf einer CD-ROM?

### Runtime-Module generieren

Seit der Version 3.9 unterstützt ACKNEX zwei Betriebssysteme, DOS und Windows 95/98 mit DirectX 5. Sie haben also die Wahl, ob Sie Ihr Game mit dem Runtime-Modul für DOS ausliefern oder für Windows (oder für beides).

Um die Runtime-Module zu erzeugen, laden Sie den Start-Level Ihres Spiels in den WED. Als Besitzer einer Lite- oder Commercial-Version klicken Sie nun auf **PUBLISH**, als Besitzer einer Profi-Version auf **COMPILE & PUBLISH**. Danach werden Sie in Ihrem Game-Verzeichnis einige neue Dateien vorfinden: WRUN.EXE und WWRUN.EXE (bei **PUBLISH**) bzw. VRUN.EXE, WVRUN.EXE und eine .WRS-Datei (bei **COMPILE & PUBLISH**). Kopieren Sie zusätzlich WWRUN.MDF und WWRUN.WDF aus dem WED-Verzeichnis in Ihr Game-Verzeichnis. Bitte beachten Sie, dass beim Compilieren der Resource-Dateien für die weiteren Level jedesmal neue Runtime-Module in Ihrem Game-Verzeichnis generiert werden - diese können Sie löschen, da nur die Runtime-Module für das Startlevel relevant sind.

Jetzt können Sie Ihr Game testen, und zwar mit folgenden DOS-Befehlen:

|                        |                                              |
|------------------------|----------------------------------------------|
| WRUN <i>levelname</i>  | zum Starten der DOS-Version                  |
| WWRUN <i>levelname</i> | zum Starten der Windows-Version              |
| VRUN <i>levelname</i>  | zum Starten der DOS-Version mit Resource     |
| WVRUN <i>levelname</i> | zum Starten der Windows-Version mit Resource |

Wenn Ihr Game per Runtime-Modul nicht startet, hier die häufigsten Fehlerursachen:

# Einer Ihrer Level genügt nicht den Einschränkungen für lizenzfreie Publikation (s.u.).

- # Sie haben vergessen, Ihre WDL-Datei per **BIND**-Schlüsselwort zur Resource hinzuzufügen.
- # WWRUN.MDF und WWRUN.WDF befinden sich nicht im Verzeichnis.
- # Sie haben der Resource-Datei einen anderen Namen als Ihrer WDL-Datei gegeben.
- # Auf Ihrem PC läuft eine der Windows-95B Versionen, bei denen aufgrund eines Bugs keine Windows-Programme direkt aus einem DOS-Fenster gestartet werden können. Erzeugen Sie in diesem Fall eine Desktop-Verknüpfung, um das Windows-Modul zu starten.
- # Auf Ihrem PC ist DirectX 5 nicht installiert.

Sie können nun die zum Starten Ihres Games per Doppelklick erforderlichen Verknüpfungen und Icon-Dateien erzeugen - wir setzen voraus, dass Sie sich mit Windows auskennen und wissen, wie das geht - und alles in ein gesondertes Verzeichnis kopieren, welches dann komplett auf die CD-ROM gebrannt werden kann. Das Game startet direkt von CD-ROM. Ein separates Setup-Programm ist nicht erforderlich. Wenn Sie ein DirectX 5 -Installationsprogramm beifügen wollen, so können Sie dies von der Microsoft-Site herunterladen. Zur Distribution von DirectX 5 beachten Sie bitte die Lizenzbestimmungen von Microsoft, Inc.

## Features

Falls Sie Ihrem Spiel weitere Features hinzufügen wollen, können Sie einfach auf die nächsthöhere GameStudio-Version upgraden - die drei Versionen sind aufwärtskompatibel. Größere **Updates** gibt es etwa einmal pro Jahr. Um zu erfahren, ob sich das Upgrade oder Update lohnt, können Sie vorher die aktuelle Liste der neuen Features von unserer WWW-Site abrufen. In der folgenden Tabelle finden Sie alle Features aufgelistet, in denen sich die drei GameStudio-Versionen zur Zeit (V3.9) unterscheiden:

| Feature / Version                                   | Lite    | Commercial  | Profi                   |
|-----------------------------------------------------|---------|-------------|-------------------------|
| Max. Objekte pro Level<br>(Walls + Things + Actors) | 1500    | 1500        | 10000                   |
| 2-Player-Modus                                      | nein    | Nullmodem   | Nullmodem<br>& Netzwerk |
| Mehrere 3D-Views                                    | nein    | ja          | ja                      |
| Maximale Auflösung                                  | 320x400 | 640x480     | 800x600                 |
| Shutterbrille                                       | nein    | nein        | ja                      |
| FLIC-Player                                         | nein    | nein        | ja                      |
| Polygon-Objekte                                     | nein    | Drahtgitter | Texturiert              |
| CD Audio                                            | nein    | nein        | ja                      |
| OUTPORT/INPORT                                      | nein    | nein        | ja                      |
| LEVEL-Anweisung                                     | nein    | nein        | ja                      |
| Resource-Compiler                                   | nein    | nein        | ja                      |
| Runtime-Module                                      | ja      | ja          | ja                      |
| Email-Support                                       | nein    | 90 Tage     | unbegrenzt              |

Besonders bei kommerziellen Games werden Sie nicht selten Features, Funktionen oder Effekte wünschen, die noch nicht in ACKNEX integriert sind. Einiges davon - z.B. Feuer, Partikel-Effekte, besondere Bewegungsarten - können Sie durch WDL-Aktionen realisieren, anderes wiederum nicht oder nur auf umständliche Weise. Vielleicht wollen Sie auch ein besonderes Eingabegerät - HMD, Tracker oder Fußpedale - verwenden, welches zur Zeit noch nicht unterstützt wird. Conitec liefert Ihnen kurzfristig **speziell für Sie erweiterte Game-Engines** mit Ihren gewünschten Features. Fragen Sie uns! Vielleicht steht das, was Sie benötigen, bereits auf unserer Update-Liste.

## Lizenzgebühren

Mit 3D GameStudio erstellte Games oder sonstige Produkte können Sie im allgemeinen **gebührenfrei** einsetzen und vertreiben. Ausnahmen sind TV-Ausstrahlungen, Einsätze an öffentlichen Plätzen oder in Spielhallen sowie 'große' Games mit Polygon-Objekten oder mit mehr als 1500 Objekten pro Level. Für solche Spiele reicht das von WED generierte Runtime-Module nicht aus; Sie erhalten von uns ein auf Ihr Spiel angepaßtes spezielles Runtime-Modul gegen Zahlung einer Einmal-Lizenzgebühr. Geben Sie uns dafür einfach den 'Magic Key' der Resource-Datei Ihres Startlevels an; Sie erhalten das Runtime-Modul per Post oder Email innerhalb eines Werktags. Bitte beachten Sie, dass das Runtime Modul dann nur die betreffende Resource-Datei startet; wenn Sie sie neu compilieren - auch wenn nichts an den einzelnen Dateien geändert wurde - kann ein anderer 'Magic Key' generiert werden.

Entnehmen Sie die Lizenzgebühren bitte der folgenden Tabelle. Alternative stückzahlabhängige Lizenzgebühren sind in Sonderfällen möglich.

|                                                   |               |            |
|---------------------------------------------------|---------------|------------|
| Endverbraucherpreis                               | unter DM 50,- | ab DM 50,- |
| Demo                                              | DM 150.-      | - -        |
| Shareware-Spiel (Selbstvertrieb)                  | DM 2000.-     | DM 3000.-  |
| Werbespiel/Präsentation                           | DM 4000.-     | DM 4000.-  |
| Kommerzielles Spiel<br>(Distributor-Vertrieb)     | DM 6000.-     | DM 10000.- |
| Kommerzielles Spiel<br>(Internationaler Vertrieb) | DM 12000.-    | DM 20000.- |

Zusammen mit 3D GameStudio haben Sie Grafikdateien für einen Demo-Level (im **DEMO**-Verzeichnis) oder für ein mitgeliefertes Actionspiel (im **SKAPH**-Verzeichnis) erhalten. Die **DEMO**-Grafiken sind nicht lizenzfrei. Sie dürfen sie zum Üben im Rahmen des Tutorials benutzen, aber nicht weitergeben. Die **SKAPH**-Grafikdateien hingegen können Sie frei verwenden und auch lizenzfrei

vertreiben - jedoch ausschließlich als Bestandteil von Spielen, die Sie mit 3D GameStudio produziert haben. Jede andere Verwendung ist lizenzpflichtig.

Wenn Sie für Ihr Projekt Level-Designer oder WDL-Programmierer suchen oder eine Game-Entwicklung in Auftrag geben wollen, hier eine Adresse:

**oP group Germany**  
**Habsburger Allee 20**  
**D-60385 Frankfurt/Main**  
**Tel 069-492426**  
**Fax 069-436389**  
**<http://www.op-group.com>**

## Probleme & Hinweise

### Update von einer früheren Version

Falls Sie GameStudio neu erworben haben, können Sie diesen Abschnitt überspringen. Alle anderen seien auf folgendes hingewiesen: Version 3.9 enthält zwei Unterschiede zu früheren Versionen, welche in seltenen Fällen - abhängig von Ihren WDL-Dateien - zu unerwünschten Effekten führen können.

Arithmetik-Fehler in **RULE**-Anweisungen wurden von früheren Versionen ignoriert, jedoch nicht mehr ab V3.9. Wenn Sie also eine "division by zero"-Fehlermeldung beobachten, überprüfen Sie die angegebene Aktion. Begrenzen Sie die Werte von Dividend und Divisor so, dass das Resultat einer Division niemals einen Wert von  $\pm 2,000,000$  überschreitet.

Der zweite Unterschied ist ein Bugfix. Nehmen wir an, in Ihrem Level schießt ein Monster eine Rakete auf den Player. Normalerweise muss hierzu die Rakete als Actor definiert sein, an die Position des Monsters plziert und dann mit **TARGET BULLET** in Richtung Player losgelassen werden. Um zu verhindern, dass die Racket beim Start mit dem Monster selbst kollidiert und explodiert, sollte sie entweder außerhalb des Monster-Kollisionsradius plziert werden, oder das Monser muss kurzzeitig während des Abschusses **PASSABLE** gesetzt werden.

Falls Sie dies vergessen haben, wird die alte Version trotzdem manchmal funktionieren, die neue jedoch niemals. Warum? Aufgrund eines Bugs in der alten Version überspringt ein Actor, dessen **INVISIBLE**-Flag gelöscht wurde, mehrere Frames (s. Skill **SKIP\_FRAMES**). Auf diese Weise kann er auch massive Objekte durchdringen. Die Beseitigung des Bugs kann nun bewirken, dass unsauber definierte **BULLET**s beim Start 'hängenbleiben'... Also vergessen Sie nicht das **PASSABLE**-Flag!

### Probleme der Windows-Version

Das Windows 95/98 Runtime-Modul hat eine Menge Vorzüge, aber auch einige Nachteile gegenüber der DOS-Version. Wenn in Ihren WDL-Skripten irgendwo eine ungültige mathematische Operation durchgeführt wird - z.B. eine Division durch Null, oder die Quadratwurzel aus einer negativen Zahl - wird Ihnen die DOS-Version beim Abbruch die betreffende Aktion und Fehlerursache nennen.

Die Windows-Version hingegen bricht hier sang-und klanglos ab. Testen Sie also all Ihre Level vorher mit der DOS-Version!

Das Windows Runtime-Modul benötigt die DirectX5-Bibliothek. Leider unterstützen die DirectX-Treiber einiger Grafik-Karten die 320x240- und 320x400-Modi nicht. Wenn Sie also ein Game in der 320x400-Auflösung entwickeln, achten Sie darauf, daß Sie auch eine alternative Auflösung - z.B 640x480 - mit berücksichtigen! Außerdem unterstützen die Treiber besonders einiger älterer Grafik-Karten den Stereo-Modus nicht, der für den Shutterbrillen-Betrieb erforderlich ist. Und wenn Sie im Bild weiße Flecken sehen, die in der DOS-Version nicht auftauchen, dann haben Sie wahrscheinlich vergessen, die letzte Farbe Ihrer Palette auf weiß zu setzen...

### Probleme der DOS-Version

Die im **WED** integrierte Engine ist die DOS-Version. Bedingt durch das Multitasking und die Bildschirm-Emulation kann es vorkommen, daß **WED** - und auch das DOS-Runtime-Modul - unter Windows 95 deutlich langsamer läuft als das Windows-Runtime-Modul. Außerdem kann es zu Problemen mit der Joystick-Abfrage kommen, die sich durch spontanes 'Wackeln' und 'Driften' des Players bemerkbar machen. Starten Sie also die DOS-Version unter Windows immer mit abgeschaltetem Joystick (Option **-NJ**).

Bei sehr großen Leveln mit vielen Bitmaps reicht manchmal der für ein DOS-Fenster automatisch reservierte virtuelle Speicher nicht aus. In diesem Fall bricht **ACKNEX** beim Start mit der Meldung '*Out of memory*' ab. Sofern jedoch noch genug Platz auf der Harddisk frei ist, kann man mit den DOS-Einstellungen (Rechtsklick auf DOS-Icon ► *Eigenschaften* ► *Speicher* ► *Arbeitsspeicher für DPMI*) den virtuellen Speicher manuell auf den erforderlichen Wert erhöhen.

Bei einigen Soundkarten werden in DOS-Fenstern Midi-Dateien nicht mehr abgespielt, wenn vorher z.B. mit dem Multimedia-Player Musik gespielt wurde. Dies ist ein Fehler von Windows 95A und hat nichts mit **ACKNEX** zu tun.

Einige Chipsätze, die in Notebook-PCs integriert sind, unterstützen den SVGA-Modus nicht. Auch bei 'normalen' PCs gibt es Probleme mit S3Trio-Karten mit Produktionsdatum von Mitte 1996 bis Mitte 1997; einige dieser Karten funktionieren aufgrund eines Bugs im VESA-BIOS nicht im SVGA-Modus. In all diesen

Fällen benutzen Sie zum Level-Testen mit WED entweder den X320x400-Modus, oder generieren Sie ein Runtime-Modul für Windows.

Zwischen WED und anderen Windows-95-Applikationen kann mit **[Alt]-[Return]** umgeschaltet werden, wenn WED mit der Option **-VGA** gestartet wurde.

## Restriktionen

Hier sind noch einmal die Einschränkungen der Software zusammengefaßt, die beim Level-Design unbedingt beachtet werden sollten:

|                                                                |       |
|----------------------------------------------------------------|-------|
| Gesamtzahl der Objekte (Walls, Things, Actors) pro Level ..... | 10000 |
| Actors pro Level .....                                         | 500   |
| Regionen pro Level .....                                       | 1000  |
| Walls und Things pro Region .....                              | 200   |
| Maximale Wall-Länge (Steps) .....                              | 250   |
| Maximale Höhendifferenz zwischen 2 Regionen (Steps) .....      | 250   |

Bei der Lite- und Commercial-Version ist zusätzlich die Gesamtzahl der Objekte auf 1500 pro Level beschränkt.

## Geschwindigkeit

Die Frame-Rate (Geschwindigkeit des Bildaufbaus), die für die 'Weichheit' der Bewegungen verantwortlich ist, kann je nach Levelgestaltung und Playerposition auf dem gleichen Rechner um bis zu 200% schwanken. Sie wird außer von der Rechner-Performance und der Größe und Auflösung des 3D-Fensters von folgenden Faktoren beeinflusst:

|                                                          |              |
|----------------------------------------------------------|--------------|
| Zahl der Objekte (Walls, Things, Actors) im Level: ..... | ca. 25%      |
| Zahl der Objekte innerhalb der <b>CLIP_DIST</b> : .....  | bis ca. 30%  |
| Gleichzeitig ablaufende Aktionen und Ways: .....         | bis ca. 100% |
| Zahl der sichtbaren Walls und Objekte: .....             | bis ca. 150% |
| Verhältnis Böden, Decken, Things zu Walls im Bild: ..... | bis ca. 40%  |
| Verhältnis geneigter zu geraden Bodenflächen: .....      | bis ca. 100% |



Außerdem hat die Bewegungsgeschwindigkeit des Players (Move-Aktion) Einfluß auf die subjektiv empfundene Geschwindigkeit des Spiels. Für eine gute Frame-Rate auf alten 486er-PCs empfehlen wir, pro Level nicht mehr als 2000 Walls und Objekte zu implementieren, von denen nicht mehr als 50 gleichzeitig sichtbar sein sollten. Nicht mehr als 20 Actors sollten gleichzeitig mit einem **WAY** und gesetztem Flag **CAREFULLY** im Level herumlaufen. Vor allem sollte vermieden werden, viele Actors über geneigte Böden laufen zu lassen. Ein großer Level mit weit auseinanderliegenden Räumen ist für die Performance günstiger als ein kleiner, enger Level mit dicht zusammenstehenden Räumen.

## Index

- + = ..... 82
- = ..... 82
- 3D\_SIMEYE ..... 22, 109
- C ..... 6, 22
- COM ..... 131
- COM ..... 21
- CT ..... 6
- d ..... 21, 35
- DIR ..... 21
- E ..... 22
- GOD ..... 22
- I ..... 4
- IPX ..... 21
- NC ..... 22
- NJ ..... 22
- NM ..... 22
- NODE ..... 21, 71, 131
- OCD ..... 21
- OM ..... 21
- OS ..... 21
- OT ..... 21
- RUN ..... 5
- S ..... 4
- SST ..... 22, 129
- VESA ..... 4
- VGA ..... 5, 149
- W1 ..... 22
- W2 ..... 22
- WMP ..... 21
- WND ..... 22
- X ..... 6
- XT ..... 6
- \* = ..... 82
- / = ..... 82
- ^ ..... 82
- % ..... 82
- & ..... 82
- | ..... 82
- 3D-Brillen ..... 109
- ABOUT ..... 13
- ABS ..... 82
- ABSPOS ..... 132, 133
- ACCEL ..... 81, 119
- ACCELERATION .... 123
- ACOS ..... 82
- ACTION ..... 80
- ACTIONS ..... 129
- ACTIVE\_NEXUS .... 129
- ACTIVE\_TARGETS .. 129
- ACTOR ..... 20, 68, 76
- ACTOR\_CEIL\_HGT .. 122
- ACTOR\_CLIMB ..... 122
- ACTOR\_DIST ..... 112
- ACTOR\_FLOOR\_HGT  
..... 122
- ACTOR\_IMPACT\_VX  
..... 122
- ACTOR\_IMPACT\_VY  
..... 122
- ACTOR\_IMPACT\_VZ  
..... 122
- ACTOR\_WIDTH ..... 122
- ADD\_STRING ..... 91
- ALBEDO ... 33, 44, 63, 67
- ALIGN ..... 11, 55
- AMBIENT . 44, 63, 67, 117
- ANGLE ..... 71, 73, 97
- ASCII ..... 30
- ASIN ..... 82
- ASPECT ..... 109, 141
- ASPEED ..... 73
- ATTACH ... 46, 47, 49, 55,  
69, 130
- ATTACHED ..... 46
- AUTORANGE .... 39, 89
- BASE ..... 65
- BBM ..... 29, 53
- BEEP ..... 88
- BEHIND ..... 48
- BELOW . 40, 60, 62, 65-67,  
123
- BERKELEY ..... 59, 76
- BIND ..... 34, 143
- BLUR ..... 39, 48
- BLUR\_MODE ..... 110
- BMAP ..... 29, 34
- BMAPS ..... 41
- BOUNCE\_VX ..... 124
- BOUNCE\_VY ..... 124
- BRANCH ..... 101, 104
- BREAK ..... 84
- BULLET ..... 70, 75, 147
- BUTTON ..... 135
- CALL ..... 86, 101, 104
- CANDELABER ..... 77
- CAREFULLY ... 68-70, 75,  
78, 79, 96, 150
- CD\_TRACK ..... 88, 126
- CDAUDIO\_VOL ..... 116
- CEIL\_ANGLE ..... 62
- CEIL\_ASCEND .... 64, 66
- CEIL\_DESCEND ... 64, 66
- CEIL\_HGT ..... 61, 123
- CEIL\_OFFS\_X ..... 62
- CEIL\_OFFS\_Y ..... 62
- CEIL\_TEX ..... 61, 80
- CENTER\_X ..... 139
- CENTER\_Y ..... 139
- CHANNEL ..... 116
- CHANNEL\_0 ..... 116
- CHECK ..... 13
- CLIP ..... 50
- CLIP\_DIST .. 22, 33, 60, 63,  
75, 76, 111-113, 149
- CLIPPING ..... 111
- CLUSTER ..... 50
- COLOR\_ACTORS .... 73,  
113
- COLOR\_BORDER ... 56,  
113
- COLOR\_PLAYER .... 113
- COLOR\_THINGS .... 73,  
113
- COLOR\_WALLS .. 56, 113
- COMPILE & PUBLISH  
..... 142
- COMPILE & PUBLISH . 8
- CONDENSED .... 50, 139
- CONTINUE ..... 84



|                                 |            |                                |             |                                |                |
|---------------------------------|------------|--------------------------------|-------------|--------------------------------|----------------|
| CONTINUE; . . . . .             | 84         | FADE_PAL . . . . .             | 89          | HARD . . . . .                 | 38             |
| COPY . . . . .                  | 9          | FAR . . . 60, 65, 77, 111, 140 |             | HBAR . . . . .                 | 135            |
| COS . . . . .                   | 82         | FENCE . . . . .                | 47, 60, 65  | HEIGHT . . . . .               | 71, 73, 77     |
| CREATE . . . . .                | 9          | FILE . . . . .                 | 7           | HELP . . . . .                 | 13             |
| CURTAIN . . . . .               | 59         | FIND . . . . .                 | 92, 115     | HERE . . . . .                 | 65, 96, 105    |
| CYCLE . . . . .                 | 55, 69     | FLAG1 . . . . .                | 60, 79      | HIT . . . . .                  | 105            |
| CYCLES . 41, 43, 45, 46, 52,    |            | FLAG8 . . . . .                | 60, 79      | HIT_DIST . 96-98, 119, 120     |                |
| 55, 69, 136                     |            | FLAGS . . 38, 48, 58, 59, 76,  |             | HIT_MINDIST . . . . 97, 120    |                |
| DARK_DIST . . . . .             | 117        | 133, 134, 139, 140             |             | HIT_X . . . . .                | 48, 96, 121    |
| Deathmatch . . . . .            | 131        | FLI . . . . .                  | 31          | HIT_Y . . . . .                | 48, 96, 121    |
| DEBUG_MODE . . . . .            | 129        | FLIC . . . . .                 | 31, 42, 88  | HOLD . . . . .                 | 71             |
| DEFINE . . . . .                | 35         | FLIC_FRAME . . . . 88, 126     |             | HSLIDER . . . . .              | 135            |
| DELAY . . . . .                 | 43, 46, 52 | FLIP . . . . .                 | 10          | IBANK . . . . .                | 33             |
| DELETE . . . . .                | 9          | FLOOR_ANGLE . . . . 62         |             | IF . . . . .                   | 83, 84         |
| DELTA_ANGLE . . . . 125         |            | FLOOR_ASCEND . . 64, 66        |             | IF_ANYKEY . . . . .            | 102            |
| DEMO . . . . .                  | 145        | FLOOR_DESCEND . . 64,          |             | IF_ARISE . . . . .             | 64, 66         |
| DIAPHANOUS . 39, 46, 48         |            | 66                             |             | IF_ARRIVED . . 69, 70, 72,     |                |
| DIGITS . . . . .                | 136        | FLOOR_HGT . . . . 61, 123      |             | 75                             |                |
| DirectX . . . . .               | 32, 142    | FLOOR_OFFS_X . . . . 62        |             | IF_BKSP . . . . .              | 103, 127       |
| DISPLAY . . . . .               | 12         | FLOOR_OFFS_Y . . . . 62        |             | IF_DIVE . . . . .              | 64, 66         |
| DIST . . 56, 57, 72, 74-76, 79, |            | FLOOR_TEX . . . . .            | 61          | IF_ENTER . . . . .             | 63, 79         |
| 96, 121                         |            | FOLLOW . . . . .               | 70          | IF_F1... . . . .               | 103            |
| DISTANCE . . . . .              | 56, 74     | FONT . . . . .                 | 30, 47, 138 | IF_FAR . . . . .               | 56, 57, 74, 75 |
| DITHER . . . . .                | 33         | FOOT_HGT . . . . .             | 123         | IF_HIT . . 57, 60, 70, 75, 78, |                |
| DROP . . . . .                  | 96, 115    | FORCE_AHEAD . . . . 127,       |             | 96, 97                         |                |
| DRUMBANK . . . . .              | 33         | 128                            |             | IF_KLICK . 48, 69, 102, 114,   |                |
| DUPLICATES . . . . .            | 13         | FORCE_ROT . . . . .            | 128         | 115, 121, 136                  |                |
| DXF . . . . .                   | 30         | FORCE_STRAFE . . . . 128       |             | IF_LEAVE . . . . .             | 64, 79         |
| EACH_CYCLE 57, 59, 65,          |            | FORCE_TILT . . . . .           | 128         | IF_LEFT . . . . .              | 102            |
| 75, 78                          |            | FORCE_UP . . . . .             | 128         | IF_LOAD . . . . .              | 102, 131       |
| EACH_TICK . . 58, 64, 75,       |            | FRAGILE . . . 60, 75, 78, 97   |             | IF_MIDDLE . . . . .            | 102            |
| 86, 101                         |            | FRAME . . . . .                | 41, 43      | IF_MSTOP . . . . .             | 102            |
| EDIT . . . . .                  | 9          | FREEZE . . . . .               | 95, 131     | IF_NEAR . . . 56, 57, 60, 74,  |                |
| ELSE . . . . .                  | 83         | FRICTION . . . . .             | 81, 119     | 75, 78, 79                     |                |
| END . . . . .                   | 86         | GENIUS . . . 44, 62, 63, 99,   |             | IF_RELEASE . . . . 47, 115     |                |
| ENDIF . . . . .                 | 35         | 140                            |             | IF_RIGHT . . . . .             | 102            |
| ERROR . . . . .                 | 129        | GETMIDI . . . . .              | 90          | IF_SPACE . . . . .             | 103, 127       |
| Event . . . . .                 | 80         | GHOST . . . . .                | 46, 48      | IF_TOUCH . . 47, 115, 121      |                |
| EXCLUSIVE . . . . .             | 86         | GLOBAL . . . 100, 101, 107,    |             | IFDEF . . . . .                | 6, 8, 21, 35   |
| EXIT . . . . .                  | 8, 101     | 130, 131                       |             | IFELSE . . . . .               | 35             |
| EXP . . . . .                   | 82         | God mode . . . . .             | 22          | IFNDEF . . . . .               | 6, 8, 35       |
| EXPLODE . . 60, 75, 78, 97,     |            | GOTO . . . . .                 | 85          | IMMATERIAL . 47, 58, 78,       |                |
| 105, 119, 120                   |            | GRID . . . . .                 | 12          | 96                             |                |
| EYE_DIST . . . . .              | 109        | GROUND . . . . .               | 65, 73, 77  | IMPACT_VROT . . . . 124        |                |

|                      |                    |                   |                         |                       |                            |
|----------------------|--------------------|-------------------|-------------------------|-----------------------|----------------------------|
| IMPACT_VX .....      | 124                | LOCAL ..          | 100, 107, 108, 130, 131 | MOUSE_CALM ..         | 102, 114                   |
| IMPACT_VY .....      | 124                | LOCATE .....      | 95, 96                  | MOUSE_LEFT .....      | 126                        |
| IMPASSABLE .....     | 58                 | LOG .....         | 82                      | MOUSE_MIDDLE ....     | 126                        |
| IMPORT .....         | 10                 | LOG10 .....       | 82                      | MOUSE_MODE .....      | 114                        |
| INCLUDE .....        | 34                 | LOG2 .....        | 82                      | MOUSE_MOVING ..       | 102, 114                   |
| INDEX .....          | 92, 138            | Magic Key .....   | 8, 145                  | MOUSE_RIGHT .....     | 126                        |
| INERTIA .....        | 81, 119            | MAP .....         | 94, 130                 | MOUSE_SENSE ..        | 127, 128                   |
| INFINITY .....       | 37                 | MAP_CENTERX ..... | 112                     | MOUSE_TIME .....      | 114                        |
| INKEY .....          | 90, 91, 115        | MAP_CENTERY ..... | 112                     | MOUSE_X .....         | 115, 133                   |
| INPORT .....         | 89                 | MAP_COLOR .....   | 56, 73                  | MOUSE_Y .....         | 115, 133                   |
| INT .....            | 82                 | MAP_EDGE_X1 ..... | 112                     | MOVE .....            | 69                         |
| INV_DIST .....       | 97                 | MAP_EDGE_X2 ..... | 112                     | MOVE_ANGLE .....      | 125                        |
| INVISIBLE ....       | 58, 76, 147        | MAP_EDGE_Y1 ..... | 112                     | MOVE_MODE ...         | 102, 111                   |
| IPX .....            | 131                | MAP_EDGE_Y2 ..... | 112                     | MOVED .....           | 78, 122                    |
| JOIN .....           | 10                 | MAP_LAYER .....   | 113                     | MPU401 .....          | 22                         |
| JOY_4 .....          | 126                | MAP_MAXX .....    | 112                     | MSPRITE ....          | 69, 114, 133, 136          |
| JOY_SENSE ....       | 127, 128           | MAP_MAXY .....    | 112                     | MUSIC .....           | 30                         |
| JOYSTICK_X .....     | 115                | MAP_MINX .....    | 112                     | MUSIC_VOL .....       | 116                        |
| JOYSTICK_Y .....     | 115                | MAP_MINY .....    | 112                     | MY .....              | 95, 105                    |
| KEY_A...KEY_Z ....   | 127                | MAP_MODE .....    | 113                     | NARROW .....          | 50, 139                    |
| KEY_ANY .....        | 126                | MAP_OFFSX .....   | 112                     | NEXT .....            | 10                         |
| KEY_F1...KEY_F12 ... | 127                | MAP_OFFSY .....   | 112                     | NEXT_MY .....         | 95                         |
| KEY_SENSE ....       | 127, 128           | MAP_ROT .....     | 113                     | NEXT_MY_THERE ..      | 95                         |
| KEYWORDS .....       | 13                 | MAP_SCALE .....   | 112                     | NEXT_THERE .....      | 95                         |
| LAYER ..             | 132, 134, 137, 140 | MAPCOLOR .....    | 50                      | NEXUS .....           | 32                         |
| LAYERS .....         | 113                | MAPFILE .....     | 34, 101                 | NO_CLIP .....         | 50                         |
| LBM .....            | 29, 53             | MASTER .....      | 79                      | NODE .....            | 71, 125, 131               |
| LEFT .....           | 57                 | MAX .....         | 107, 135                | NODE0 .....           | 131                        |
| LENGTH .....         | 12, 56             | MAX_DIST .....    | 129                     | NODE1 .....           | 131                        |
| LEVEL ..             | 101, 107, 112, 130 | MDL .....         | 30, 41                  | NULL .....            | 69, 80, 83, 138            |
| LIBER .....          | 75, 76             | MICKEY_X .....    | 114                     | OFFSET_X ..           | 41, 43, 46, 55, 121        |
| LIFT .....           | 98                 | MICKEY_Y .....    | 114                     | OFFSET_Y ..           | 41, 43, 46, 55, 92, 138    |
| LIFTED .....         | 64                 | MIDI_COM .....    | 90                      | ONESHOT ..            | 48, 57, 59, 64, 65, 75, 78 |
| LIGHT_ANGLE ..       | 33, 44, 63         | MIDI_PITCH .....  | 33                      | oP group Germany .... | 146                        |
| LIGHT_DIST .....     | 117                | MIN .....         | 107, 135                | OPEN WMP .....        | 7                          |
| LIGHTMAP .....       | 49                 | MIRROR .....      | 43, 52                  | OUTPORT .....         | 89                         |
| LINES .....          | 92, 115            | MODE .....        | 8                       | OVERLAY ...           | 69, 132, 134               |
| LOAD .....           | 91, 100, 112       | MODEL ....        | 30, 41, 43, 50          | OVLY .....            | 29, 132                    |
| LOAD_GAME .....      | 130                | MODIFY .....      | 9                       | OVLYS .....           | 132                        |
| LOAD_INFO ...        | 100, 107, 131      | MOTION_BLUR ..    | 95, 110                 |                       |                            |
| LOAD_MODE ..         | 100, 101, 112      | MOUSE .....       | 115                     |                       |                            |
|                      |                    | MOUSE_ANGLE ..    | 96, 115                 |                       |                            |

|                       |                                    |                        |                                         |                       |                                         |
|-----------------------|------------------------------------|------------------------|-----------------------------------------|-----------------------|-----------------------------------------|
| PALETTE . . . . .     | 89                                 | POS_Y . . .            | 46, 106, 132-135,<br>137, 139, 140      | SCALE_XY . . . . .    | 52                                      |
| PALETTE . . . . .     | 37                                 | POSITION . .           | 47, 56, 59, 60                          | SCALE_Y . . . . .     | 44, 138                                 |
| PALFILE . . . . .     | 38                                 | PREV . . . . .         | 10                                      | SCREEN_HGT . .        | 109, 140                                |
| PAN_MAP . . . . .     | 134                                | PRINT_STRING . . . . . | 94                                      | SCREEN_WIDTH . .      | 109,<br>140                             |
| PANEL . . . . .       | 133                                | PRINT_VALUE . . . . .  | 94                                      | SCREEN_X . . . . .    | 109, 140                                |
| PASSABLE . . . .      | 58, 76, 147                        | PRINTFILE . . . . .    | 94                                      | SCREEN_Y . . . . .    | 109, 140                                |
| PATH . . . . .        | 34                                 | PSOUND_TONE . . . .    | 116                                     | SCREENSHOT . . . . .  | 95                                      |
| PCX . . . . .         | 29                                 | PSOUND_VOL . . .       | 61, 116                                 | SCROLL . . . . .      | 13                                      |
| PICTURE . . . .       | 95, 134, 136                       | PUBLISH . . . . .      | 142                                     | SCYCLES . . . . .     | 45, 52                                  |
| PLAY . . . . .        | 48, 59, 65, 78                     | PUSH . . . . .         | 98                                      | SDIST . . . . .       | 45, 52, 87                              |
| PLAY_CD . . . . .     | 88, 126                            | RADIANCE . . . . .     | 44                                      | SECS . . . . .        | 126                                     |
| PLAY_DEMO . . . . .   | 100                                | RANDOM . . . . .       | 44, 46, 82                              | SEEN . . . . .        | 58, 64, 77                              |
| PLAY_FLIC . . . . .   | 88                                 | RANGE . . . . .        | 38, 39                                  | SENSITIVE . . . . .   | 60, 78                                  |
| PLAY_FLICFILE . . . . | 88                                 | RECTANGLE . . . . .    | 12                                      | SET . . . . .         | 80                                      |
| PLAY_SONG . . . . .   | 87                                 | REFRESH . . . . .      | 134                                     | SET_ALL . . . . .     | 80, 81                                  |
| PLAY_SONG_ONCE .      | 87                                 | REGION . . . . .       | 61                                      | SET_INFO . . . . .    | 92                                      |
| PLAY_SOUND . . .      | 87, 116                            | REGIONS . . . . .      | 8                                       | SET_SKILL . . . . .   | 91                                      |
| PLAY_SOUNDFILE . .    | 87                                 | REL_ANGLE . .          | 70, 72, 133                             | SET_STRING . . . . .  | 91                                      |
| PLAYER_ANGLE . . .    | 96,<br>118, 123                    | REL_DIST . . . . .     | 72                                      | SETMIDI . . . . .     | 90                                      |
| PLAYER_ARC . . . . .  | 119                                | RELPOS . . . . .       | 135                                     | SHADOW . . . . .      | 48                                      |
| PLAYER_CLIMB . . . .  | 117                                | REMOTE_0 . . . .       | 125, 131                                | SHAKE . . . . .       | 57, 74, 98                              |
| PLAYER_COS . . . . .  | 123                                | REMOTE_1 . . . .       | 125, 131                                | SHIFT . . . . .       | 65, 99                                  |
| PLAYER_DEPTH .        | 66, 123                            | RENDER_MODE . . .      | 110                                     | SHIFT_SENSE . . . . . | 128                                     |
| PLAYER_HGT . .        | 65, 122,<br>123                    | REPEL . . . . .        | 70                                      | SHOOT . .             | 57, 58, 75, 78, 96,<br>98, 105, 119-121 |
| PLAYER_LIGHT . .      | 63, 67,<br>117                     | RESET HOR . . . . .    | 11                                      | SHOOT_ANGLE . .       | 97, 120                                 |
| PLAYER_SIN . . . . .  | 123                                | RESULT .               | 70, 91, 92, 96-98,<br>119, 120          | SHOOT_FAC . . . .     | 96, 119                                 |
| PLAYER_SIZE . . . . . | 117                                | RIGHT . . . . .        | 57                                      | SHOOT_RANGE .         | 96, 97,<br>119                          |
| PLAYER_SPEED . . . .  | 123                                | ROTATE . . . . .       | 63, 65                                  | SHOOT_SECTOR . . . .  | 97,<br>119                              |
| PLAYER_TILT . . . . . | 118                                | ROTATE & SCALE . . . . | 10                                      | SHOOT_X . . . . .     | 96, 119                                 |
| PLAYER_VROT . . . .   | 118                                | ROTATE . . . . .       | 99                                      | SHOOT_Y . . . . .     | 96, 119                                 |
| PLAYER_VX . . . .     | 118, 123                           | RULE . . . . .         | 82, 147                                 | SIDES . .             | 40, 42, 43, 51, 52,<br>79, 136          |
| PLAYER_VY . . . .     | 118, 123                           | SAVE .                 | 51, 60, 64, 79, 91, 98,<br>99, 107, 130 | SIGN . . . . .        | 82                                      |
| PLAYER_VZ . . . . .   | 118                                | SAVE AS . . . . .      | 7                                       | SIN . . . . .         | 82                                      |
| PLAYER_WIDTH . . . .  | 117                                | SAVE WMP . . . . .     | 7                                       | SIZE_X . .            | 56, 74, 133, 140                        |
| PLAYER_X . . . . .    | 122                                | SAVE_ALL . . . . .     | 65                                      | SIZE_Y . .            | 74, 92, 115, 133,<br>138, 140           |
| PLAYER_Y . . . . .    | 122                                | SAVE_DEMO . . . . .    | 100                                     | SKAPH . . . . .       | 145                                     |
| PLAYER_Z . .          | 64, 122, 123                       | SAVE_GAME . . . . .    | 130                                     | SKILL . . . . .       | 107                                     |
| POLYGON . . . . .     | 12                                 | SAVE_INFO .            | 95, 100, 107,<br>131                    | SKIP_FRAMES . . .     | 77, 121,                                |
| PORTCULLIS . .        | 47, 59, 65                         | SAVEDIR . . . . .      | 34, 99, 100                             |                       |                                         |
| POS_X . .             | 46, 106, 132-135,<br>137, 139, 140 | SCALE_X . . . . .      | 44, 66, 138                             |                       |                                         |

|                         |                                |                       |                           |                       |              |
|-------------------------|--------------------------------|-----------------------|---------------------------|-----------------------|--------------|
|                         | 147                            | THING . . . . .       | 19, 68, 76                | WALK . . . . .        | 61, 118, 124 |
| SKY . . . .             | 40, 41, 49, 60, 111            | THING_DIST . . . . .  | 112                       | WALK_PERIOD . . . . . | 117          |
| SKY_OFFSETS_X . . . . . | 42, 109                        | THING_WIDTH . . . . . | 122                       | WALK_TIME . . . . .   | 118          |
| SKY_OFFSETS_Y . . . . . | 109                            | THINGS . . . . .      | 8                         | WALKTHROUGH . . . . . | 9            |
| SKY_OFFSET_Y . . . . .  | 42                             | TICKS . . . . .       | 126                       | WALL . . . . .        | 10, 40, 55   |
| SLOOP . . . . .         | 50, 61, 116                    | TILT . . . . .        | 98                        | WALLS . . . . .       | 8, 13        |
| SLOPE_AHEAD . . . . .   | 125                            | TIME_CORR . . . . .   | 125                       | WAV . . . . .         | 30, 54       |
| SLOPE_SIDE . . . . .    | 125                            | TIME_FAC . . . . .    | 125                       | WAVE . . . . .        | 61, 118      |
| SLOPE_X . . . . .       | 125                            | TITLE . . . . .       | 42, 47                    | WAVE_PERIOD . . . . . | 118          |
| SLOPE_Y . . . . .       | 125                            | TO_STRING . . . . .   | 91                        | WAY . . . . .         | 20, 68, 72   |
| SOUND . . . . .         | 30, 45                         | TOP . . . . .         | 62                        | WAYPOINT . . . . .    | 72           |
| SOUND_VOL . . . . .     | 116                            | TOUCH . . . . .       | 47, 106, 114, 115         | WAYS . . . . .        | 8            |
| SPEED . . . . .         | 69, 70, 73                     | TOUCH_DIST . . . . .  | 47, 115                   | WED . . . . .         | 4            |
| SPLIT . . . . .         | 10                             | TOUCH_MODE . . . . .  | 47, 114                   | WEX . . . . .         | 26, 29-31    |
| SQRT . . . . .          | 82                             | TOUCH_RANGE . . . . . | 47                        | WHILE . . . . .       | 84           |
| STARTTEXT . . . . .     | 101                            | TOUCH_REG . . . . .   | 105                       | WINDOW . . . . .      | 134, 136     |
| STATISTICS . . . . .    | 13                             | TOUCH_STATE . . . . . | 114                       | Windows 95 . . . . .  | 147          |
| STEPS . . . . .         | 124, 126                       | TOUCH_TEX . . . . .   | 105                       | WIRE . . . . .        | 50           |
| STICK . . . . .         | 70, 72, 124                    | TOUCH_TEXT . . . . .  | 47, 106                   | WRS . . . . .         | 142          |
| STOP_DEMO . . . . .     | 100                            | TOUCHED . . . . .     | 105                       | WRUN.EXE . . . . .    | 7, 142       |
| STOP_FLIC . . . . .     | 88                             | TRANSPARENT . . . . . | 134                       | WVRUN.EXE . . . . .   | 8, 142       |
| STOP_SOUND . . . . .    | 87                             | TYPE . . . . .        | 104, 107                  | WVRUN.EXE . . . . .   | 7, 142       |
| STR_LEN . . . . .       | 115                            | UNDEF . . . . .       | 35                        | WVRUN.MDF . . . . .   | 142          |
| STRAIGHT . . . . .      | 75                             | UNDO . . . . .        | 9                         | WVRUN.WDF . . . . .   | 142          |
| STRING . . . . .        | 91, 115, 137, 138              | Update . . . . .      | 143                       | X . . . . .           | 74           |
| STRINGS . . . . .       | 138                            | VAL . . . . .         | 107                       | X1 . . . . .          | 57           |
| SVDIST . . . . .        | 45                             | VBAR . . . . .        | 135                       | X2 . . . . .          | 57           |
| SVOL . . . . .          | 45                             | Vertex . . . . .      | 19, 70                    | Y . . . . .           | 74           |
| SYNONYM . . . . .       | 104                            | VERTICES . . . . .    | 8                         | Y1 . . . . .          | 57           |
| TAN . . . . .           | 82                             | VIDEO . . . . .       | 32, 109                   | Y2 . . . . .          | 57           |
| TARGET . . . . .        | 69, 73, 77, 121, 122, 131, 148 | VIEW . . . . .        | 139                       | Z1 . . . . .          | 57           |
| TARGET_X . . . . .      | 70-72                          | VISIBLE . . . . .     | 58, 64, 76, 133, 139, 140 | Z2 . . . . .          | 57           |
| TARGET_Y . . . . .      | 70-72                          | VRUN.EXE . . . . .    | 8, 142                    | ZOOM . . . . .        | 12           |
| TEXT . . . . .          | 91, 92, 106, 115, 137          | VSLIDER . . . . .     | 135                       | # . . . . .           | 28           |
| TEXTURE . . . . .       | 40, 51, 55, 68                 | VSPEED . . . . .      | 69, 70, 72, 73            | ► . . . . .           | 28           |
| THERE . . . . .         | 95, 105                        | WAIT . . . . .        | 84, 86, 97                |                       |              |
|                         |                                | WAITT . . . . .       | 86                        |                       |              |